GEN REPORT | JULIA-HOWTOS

Julia Integration in a Hybrid R and Python Research Repository

Project skeleton, environment management and cross-language calls for adding Julia to a uv and renv based research project

datalab@unimib.it

Project:	dve-sample-py
Document:	Julia-Q3-Project-Specification.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py/notes/howtos/
Date:	October 14, 2026

Keywords: GEN, Julia, R, Python, Interoperability, Pkg-environments, Research-software

Abstract

This document is an hands-on tutorial on introducing Julia into a hybrid multi-language research repository. The target reader is an expert Python and R programmer who is a Julia beginner, working in statistics, mathematics, machine learning and economics.

The repository layout follows the standard R package structure, with non-standard entries excluded through `.Rbuildignore`, so that CRAN compliance is preserved. The Julia package `Dve` is added alongside the Python package `dvepy` (managed by `uv`) and the R package `dvesimpler` (managed by `renv`), targeting Julia `1.13.1` without legacy constraints.

The specification covers a minimal Julia package skeleton, including a `hello_world` function, an executable command-line script, unit tests, Jupyter kernel support and language server configuration for VS Code and Emacs. It further addresses dependency management, `Project.toml` and `Manifest.toml` semantics, dev, test and LSP dependency specification, and Pkg workspaces.

Integration examples are required for R to Julia, Julia to Python and Python to Julia calls, covering primitive types and data frames, with explicit treatment of copy versus zero-copy semantics, interpreter discovery and first-run bootstrapping. Accuracy rules require unverified claims to be explicitly marked.

Contents

TOC **7**

Q:1 **8**
Q:1 - **Julia integration in a hybrid R + Python research project** 8
ROLE 8
CONTEXT 8
Repository Layout 8
RULES FOR ACCURACY 9
OBJECTIVES 9
1. Julia project skeleton 9
2. Julia project structure 10
3. Language integration examples 10
OUTPUT FORMAT 10

A:1 (Claude) **12**
Repository tree and assumptions 12
Deviations from the assumed layout 12
1. Julia project skeleton 13
Package and project files 13
Unit test 14
Executable script outside the package 15
Jupyter kernel bound to the project 17
LSP support 17
2. Julia project structure 19
Dependency management, UUIDs, compat, registries 19
Dev, test and LSP dependencies 20
What is configured and what is convention 20
Workspaces 20
3. Language integration examples 21
Choice of integration packages 21
Shared environment rules 21
Project package `dvepy` 22
3.1 R to Julia 22
3.2 Julia to Python 23
3.3 Python to Julia 25
Python/R to Julia equivalents 26
Acceptance checklist 26
Common pitfalls 26

Unverified items and documentation to check . . . 27
Additional Notes . . . 28
Revision: in-process R to Julia, large data, dual runtime . . . 28
1. R to Julia reconsidered: JuliaCall . . . 28
What changes with the in-process model . . . 28
In-process example . . . 29
2. What zero-copy means between R and Julia . . . 29
Memory layouts . . . 29
Direction by direction . . . 30
Hazards of shared memory . . . 30
3. Columnar and lazy models (Polars-style) . . . 30
Why borrowing numeric vectors does not scale to data frames . . . 30
Three levels . . . 31
4. Dual runtime and the optional RCall dependency . . . 32
Differences between the runtimes . . . 32
Why not put RCall in Project.toml or a workspace member . . . 32
Layout additions . . . 32
Container-only environment . . . 32
Optional code-level integration (only if Dve must call R) . . . 33
Runtime configuration . . . 34
Acceptance checklist (additions) . . . 35
Additional pitfalls . . . 36
Additional unverified items . . . 36

A:1 (Gemini) **37**
1. Julia Project Skeleton . . . 37
Project Configuration and Main Module . . . 37
Jupyter Kernel Support . . . 38
Executable Script . . . 38
Unit Testing . . . 39
LSP Editor Support . . . 39
2. Julia Project Structure . . . 40
Dependency Management and UUIDs . . . 40
Dev, Test, and LSP Dependencies . . . 40
Source, Test, and Script Folders . . . 40
Workspaces . . . 41
3. Language Integration Examples . . . 41
1. R to Julia (`RCall.jl`) . . . 41
2. Julia to Python (`PythonCall.jl`) . . . 42
3. Python to Julia (`juliacall`) . . . 43
Acceptance Tests Checklist . . . 44
Common Pitfalls . . . 44
List of Unverified Items . . . 44
Additional Notes . . . 45
References . . . 45

A:1 (ChatGPT) **46**
1. Julia project skeleton . . . 46
1.1 Proposed repository tree . . . 46
1.2 Minimal Julia package . . . 47
1.3 CLI script with a shebang and `--help` . . . 48
1.4 Unit test . . . 50
1.5 Jupyter kernel support . . . 51
1.6 LSP support: VS Code and Emacs . . . 52
2. Julia project structure . . . 53
2.1 Dependencies, UUIDs, and reproducibility . . . 53
2.2 Runtime, test, development, and LSP dependencies . . . 55
2.3 Source, test, and script directories . . . 58
2.4 What does "workspace" mean? . . . 58
3. Language integration examples . . . 59
3.1 Integration architecture . . . 59
3.2 Environment ownership and the no-duplicate-installation rule . . . 59
3.3 R to Julia . . . 60
3.4 Julia to Python . . . 63
3.5 Python to Julia . . . 67
4. Acceptance tests and operational validation . . . 71
4.1 Acceptance checklist . . . 71
4.2 R package compliance . . . 71

A:1 (DeepSeek) **72**
Proposed Repository Layout and Julia Version . . . 72
Objective 1: Julia Project Skeleton . . . 72
Minimal `Dve` Module . . . 72
Jupyter Kernel Support . . . 73
Executable Script . . . 73
Unit Test . . . 74
LSP Editor Support . . . 75
Objective 2: Julia Project Structure . . . 75
Dependency Management and UUIDs . . . 75
Test and Dev Dependencies . . . 76
Source, Test, and Script Folders . . . 76
Workspaces . . . 76
Objective 3: Language Integration Examples . . . 77
R to Julia . . . 77
Julia to Python . . . 77
Python to Julia . . . 78
Calls to Project Packages . . . 79
Acceptance Tests Checklist . . . 79
Common Pitfalls . . . 79
Unverified Items . . . 79
Additional Notes . . . 80

References 80

A:1 (Kimi) **81**
0. Repository tree and first-time bootstrap 81
0.1 Proposed tree 81
0.2 Concept map: Python/R to Julia equivalents 82
0.3 Deviations from the assumed layout (CRAN compliance) 82
0.4 Clean-checkout bootstrap 83
1. Julia project skeleton 83
1.1 Project files 83
1.2 Jupyter kernel 84
1.3 Executable script with command-line interface 85
1.4 Unit test 87
1.5 LSP editor support 88
2. Julia project structure 89
2.1 Dependency management and UUIDs 89
2.2 Dev, test and LSP dependencies 90
2.3 How source, test and script folders are determined 90
2.4 Workspaces 91
3. Language integration examples 91
3.1 R calls Julia (`JuliaCall`) 92
3.2 Julia calls Python (`PythonCall.jl`) 93
3.3 Python calls Julia (`juliacall`) 95
3.4 Conversion semantics at a glance 97
3.5 Interpreter and environment discovery, summarized 97
4. Acceptance checklist 97
5. Common pitfalls 98
6. Additional notes 99
7. Unverified items (documentation to check) 99
References 100

A:1 (Perplexity) **101**
Proposed Repository Tree 101
1. Julia Project Skeleton 102
Package Entry Point 102
Project File 102
Install Dependencies 103
Jupyter Kernel 103
Executable Script 104
Unit Test 107
Editor Support 107
VS Code 107
Emacs 108
2. Julia Project Structure 109
Dependencies and UUIDs 109

Test and Development Dependencies . . . 109
Source, Tests, and Scripts . . . 110
Pkg Workspaces . . . 110
3. Language Integration Examples . . . 110
Python/R to Julia Equivalents . . . 110
Julia to Python . . . 111
Environment Binding: Julia to Python . . . 112
Python to Julia . . . 112
Environment Binding: Python to Julia . . . 114
R to Julia . . . 114
Environment Binding: R to Julia . . . 116
First-Run Bootstrapping . . . 116
Acceptance Tests . . . 116
Common Pitfalls . . . 117
Additional Notes . . . 117
Unverified Items . . . 117
References . . . 118

A:1 (Vibe) **119**
Proposed repository tree . . . 119
Corrections to the assumed layout . . . 119
Python–R–Julia equivalents . . . 120
Objective 1 — Julia project skeleton . . . 120
`Project.toml` and the `Dve` module . . . 120
Jupyter kernel . . . 121
Executable script . . . 122
Unit test . . . 123
LSP support . . . 124
Objective 2 — Julia project structure . . . 126
Dependency management and UUIDs: is `Pkg.add` enough? . . . 126
Dev, test and LSP dependencies . . . 126
How source, test and script folders are determined . . . 127
Workspaces . . . 127
Objective 3 — Language integration . . . 127
Package choices and justification . . . 127
R to Julia (JuliaConnectoR) . . . 128
Julia to Python (PythonCall) . . . 129
Python to Julia (juliacall + juliapkg) . . . 130
Acceptance tests . . . 132
Common pitfalls . . . 132
Unverified items . . . 133
Additional notes . . . 133

Q:2 **135**
Q:2 - **TODO:(q2-title)** . . . 135

Question Prompt 2 135

A:2 (Claude) **136**

A:2 (Gemini) **137**

A:2 (ChatGPT) **138**

A:2 (DeepSeek) **139**

A:2 (Perplexity) **140**

A:2 (Vibe) **141**

A:a **142**

A:a - **TODO:(aa-title)** 142

Appendix a 142

TOC

1. Q:1 - Julia integration in a hybrid R + Python research project
    - see: TODO:(a1-ref-claude) (Claude)
    - see: Julia Repository Layout (Gemini)
    - see: Julia integration tutorial for a hybrid research repository (ChatGPT)
    - see: Julia integration tutorial (DeepSeek)
    - see: Adding Julia to a hybrid R + Python research project (Kimi)
    - see: Julia in a Multi-Language Research Repository (Perplexity)
    - see: Cross Language Integration Bridge (Vibe)
2. Q:2 - TODO:(q2-ref)
    - see: TODO:(a2-ref-claude) (Claude)
    - see: TODO:(a2-ref-gemini) (Gemini)
    - see: TODO:(a2-ref-chatgpt) (ChatGPT)
    - see: TODO:(a2-ref-perplexity) (Perplexity)
    - see: TODO:(a2-ref-deepseek) (DeepSeek)
    - see: TODO:(a2-ref-vibe) (Vibe)
3. A:a - TODO:(appendix-a)

Q:1

Q:1 - Julia integration in a hybrid R + Python research project

^

ROLE

You are a senior Julia developer and academic research software engineer. You are writing a hands-on tutorial for an expert Python and R programmer who is a Julia beginner. Explain Julia concepts by contrasting them with Python and R equivalents wherever that shortens the learning curve. Be concise and precise, and avoid generic introductions to programming.

CONTEXT

- The repository is a hybrid, multi-language research project with Python (managed by `uv`) and R (managed by `renv`). Julia is being added.
- Research fields: statistics, mathematics, machine learning and economics.
- Goal: introduce Julia through working examples of cross-language integration.
  - R calls Julia.
  - Julia calls Python.
  - Python calls Julia.
  - Calls go both to third-party packages and to the project's own packaged code.
- Julia target: version: "1.13.1", the latest stable release, with no legacy-compatibility constraints.
- Package names:
  - Julia: `Dve`
  - Python: `dvepy`
  - R: `dvesimpler`

Repository Layout

Repository follows standard R project layout, with non-standard entries explicitly ignored in `.Rbuildignore` file. If Julia support requires modifications, justify any deviation from this assumption, but keep CRAN compliance as a contraint.

```
<project-root>
  pyproject.toml
  DESCRIPTION
  uv.lock
```

```
renv.lock
Project.toml
Manifest.toml

R/                              # <- R "dvesimpler" package sources
inst/include                    # <- Rcpp C++ "dvesimpler" package interface
inst/python                     # <- Python "dvepy" package source base
inst/python/dve/__init__.py     # <- Python "dvepy.dve" `dve` module in "dvepy" package
src/Makevars                    # <- Rcpp C++ build specification
src/*.cpp                       # <- Rcpp C++ package sources specification
src/Dve.jl                      # <- Julia Dve main module source
src/*.jl                        # <- Other Julia sources, ignored in `.Rbuildignore`
tests/pytest                    # <- Python "dvepy" package test base (`pytest`)
tests/testthat                  # <- R "dvesimpler" package test base (`devtools::test()`)
tests/*.jl                      # <- Test Julia sources, ignored in `.Rbuildignore`
```

RULES FOR ACCURACY

- Do not invent APIs, package names, configuration keys or `Project.toml` syntax. If you are not certain, mark the statement as `[unverified]` and name the official documentation to check.
- If a requirement below rests on a mistaken assumption, say so explicitly and explain the correct mechanism.
- Every command and code snippet must be runnable from a clean checkout. State the working directory and the expected output.
- Each code block must be preceded by its file path.
- When comparing alternatives, use these criteria: purpose, maintenance status, setup effort, and when to prefer it.

OBJECTIVES

1. Julia project skeleton

Provide a minimal Julia package skeleton with:

- A module named `Dve`, importable after activating the project.
- Jupyter kernel support (explain installation and how the kernel binds to the project environment).
- A function `hello_world(salutation, who)` that returns a formatted greeting string.
- An executable Julia script, located outside the package, that:
  - starts with a shebang line, and explain how it finds the project environment;
  - parses command-line arguments, with `--help` documentation;
  - imports `Dve` with `using` and prints the result of `hello_world`.
- A unit test for `hello_world`, and the command that runs it.
- LSP editor support for VS Code and Emacs. Explain the relationship between `LanguageServer`, `JuliaWorkspaces` and `SymbolServer`, and whether they are alternatives or components. Verify

any deprecation claims. For Emacs, compare the available clients (for example `lsp-mode` with `lsp-julia`, and `eglot` with `eglot-jl`) and provide a current `use-package` configuration snippet.

2. Julia project structure

Explain the following, with minimal examples:

- Dependency management and UUIDs: is `Pkg.add` enough? Cover `Project.toml`, `Manifest.toml`, `[compat]`, `Pkg.develop` for local packages, and registries.
- How to specify dev, test and LSP dependencies. Cover `[extras]`/`[targets]`, `test/Project.toml` and separate environments, and state which option is recommended and why.
- How the source, test and script folders are determined. Clarify what is set in `Project.toml` and what is only a convention.
- What *workspaces* are. Cover Pkg workspaces (the `[workspace]` table) and, if relevant, `JuliaWorkspaces` and editor workspaces. Give the use cases and a minimal example.

3. Language integration examples

Provide minimal, runnable examples for each direction. Name the integration package you use, and justify the choice against alternatives.

1. R to Julia
   - Primitive argument and return types.
   - Complex types: an R `data.frame` passed to Julia as a `DataFrames.DataFrame` and returned.
2. Julia to Python
   - Primitive argument and return types.
   - Complex types: a Julia `DataFrame` converted to a `pandas.DataFrame` and back.
   - Calls to an imported Python package (`numpy`) and to the project package `dvepy`.
3. Python to Julia
   - Primitive argument and return types.
   - Complex types: a `pandas.DataFrame` converted to a Julia `DataFrame` and back.
   - Calls to an imported Julia package (`Statistics`) and to the project package `Dve`.

For every example, state how the conversion works, including copy versus zero-copy semantics. Explain how each integration finds the right interpreter and environment (`uv` for Python, `renv` for R) without duplicating installations. Explain the first-run bootstrapping steps.

OUTPUT FORMAT

- Markdown with one ## section per objective. Start with the proposed repository tree and the assumed Julia version.
- Use short explanations followed by code. Add a table of Python/R to Julia equivalents where helpful.
- End with a checklist of acceptance tests (what to run, what to expect) and a list of common pitfalls.

- Finish with a list of all `[unverified]` items and the documentation to check.
- Reply in clear formatted ”GitLab Flavored Markdown (GLFM)” Markdown, with precise (lint) validation:
  - codeblock delimiters “‘ placed atline start). Avoid codeblock nesting.
  - use *underscore markup* for emphasys
  - prefer nested headings to text markup with asterisks
  - use only ”dash” for unordered lists, with correct indentation
  - insert appropriate blank line separation after headings, list and codeblocks
- Ignore document formatting markup, like:
  -
  - {=latex} codeblocks
  - [!tip] [!note] block quotes
  - code folding tags (”three curly braces pairs”)
  - internal links: e.g. [^]
- At the end, provide, as Markdown footnotes, a list of references to online documentation resources, linked to answer text where appropriate. To avoid reference clashing with other part of the document, prefix references with the string ”rf-”.
- Add any additional important information not explicitly required in an ”Additional Notes” section.

A:1 (Claude)

^ *Julia for the Dve research repository*

- see: Claude

Repository tree and assumptions

Assumed Julia version: 1.13.1 (`julia = "1.13"` in `[compat]`). The 1.13 release blog is dated 10 Sep 2026.[1] Whether the 1.13.1 patch is already tagged is `[unverified]`; install with `juliaup add 1.13`.[2]

Proposed tree. Entries marked `(new)` are my additions, and `(moved)` marks a deviation from your layout, justified below.

```
<project-root>
  pyproject.toml
  DESCRIPTION
  uv.lock
  renv.lock
  Project.toml                      # Julia package Dve + [workspace]
  Manifest.toml                     # one shared Julia lockfile, committed
  .Rbuildignore                     # extended, see below
  R/                                # dvesimpler
  inst/include, inst/python/dve/__init__.py
  src/Makevars, src/*.cpp           # Rcpp
  src/Dve.jl, src/*.jl              # Julia sources (ignored by R)
  test/runtests.jl                  # (moved) from tests/*.jl
  test/Project.toml                 # (new) test environment, workspace member
  scripts/hello.jl                  # (new) executable CLI, outside the package
  scripts/Project.toml              # (new) script environment, workspace member
  examples/*.jl, *.py, *.R          # (new) integration examples
  examples/Project.toml             # (new) workspace member
  tests/pytest, tests/testthat      # unchanged
```

Deviations from the assumed layout

Two requirements rest on a mistaken assumption.

- `tests/*.jl` does not work with `Pkg.test()`.

[1] Julia 1.13 Highlights, 10 Sep 2026, https://julialang.org/blog/2026/09/julia-1.13-highlights/
[2] juliaup, the Julia installer and version manager, https://github.com/JuliaLang/juliaup

  - `Pkg.test` hard-codes `test/runtests.jl`, and no `Project.toml` key changes that.[3]
  - Keep R's `tests/` and add a Julia `test/` directory (no `s`).
  - Running `julia tests/x.jl` by hand loses the test sandbox, `[extras]/[targets]` handling and the `Pkg.test` entry point.
- `src/*.jl` inside R's `src/` is fine for R, as long as the files stay out of the tarball.
  - R compiles only known source extensions, and `.Rbuildignore` entries are regexes matched against paths relative to the root.[4]
  - CRAN compliance does not require removing the files. It requires that the R package never fails when Julia is missing (see the pitfalls).

File: `.Rbuildignore` (append these lines)

```
^Project\.toml$
^Manifest(-v[0-9.]+)?\.toml$
^src/.*\.jl$
^test$
^scripts$
^examples$
^\.julia
```

1. Julia project skeleton

Package and project files

Python/R analogy: `Project.toml` is `pyproject.toml`/`DESCRIPTION`, `Manifest.toml` is `uv.lock`/`renv.lock`. The module name must equal `name`, and the entry file must be `src/<name>.jl`; there is no key to move it.

File: `Project.toml`

```
name = "Dve"
uuid = "5b3e2a8c-91d4-4f06-8a7e-2c1d9e6f4b30"
version = "0.1.0"

[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"

[compat]
DataFrames = "1"
Statistics = "1"
julia = "1.13"

[workspace]
```

[3] Pkg.jl manual, "Creating Packages" (layout, tests), https://pkgdocs.julialang.org/v1/creating-packages/

[4] Writing R Extensions, `.Rbuildignore` and package structure, https://cran.r-project.org/doc/manuals/r-release/R-exts.html

```
projects = ["test", "scripts", "examples"]
```

The UUID above is arbitrary but valid. Generate your own with `using UUIDs; uuid4()`.

File: `src/Dve.jl`

```
module Dve

using DataFrames: DataFrame
using Statistics: mean

export hello_world, scaled_mean, add_total

"""
    hello_world(salutation, who) → String

Return a formatted greeting, e.g. `hello_world("Hello", "Julia") == "Hello, Julia!"`.
"""
function hello_world(salutation::AbstractString, who::AbstractString)
    isempty(strip(salutation)) && throw(ArgumentError("salutation must not be empty"))
    return string(salutation, ", ", who, "!")
end

"Mean of `v` multiplied by `factor` (primitive/vector example for the bridges)."
scaled_mean(v::AbstractVector{<:Real}, factor::Real = 1) = mean(v) * factor

"""
    add_total(tbl) → DataFrame

Accept any Tables.jl-compatible table with numeric columns `x` and `y`,
return a new `DataFrame` with an extra column `total = x + y`.
"""
function add_total(tbl)
    df = DataFrame(tbl)          # copies columns by default
    df.total = df.x .+ df.y
    return df
end

end # module
```

Multiple dispatch replaces Python's `isinstance` branching and R's S3/S4 dispatch. The `::AbstractString` annotation is a dispatch constraint, not a declaration.

Unit test

File: `test/Project.toml`

```
[deps]
Dve = "5b3e2a8c-91d4-4f06-8a7e-2c1d9e6f4b30"
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"
```

File: `test/runtests.jl`

```
using Test
using Dve

@testset "hello_world" begin
    @test hello_world("Hello", "Julia") == "Hello, Julia!"
    @test hello_world("Ciao", "mondo") == "Ciao, mondo!"
    @test_throws ArgumentError hello_world("", "x")
end
```

Shell, working directory `<project-root>`. The first command resolves and installs everything on a clean checkout:

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
julia --project=. -e 'using Pkg; Pkg.test()'
```

Expected output ends with a summary like the following (timing differs):

```
Test Summary: | Pass  Total  Time
hello_world   |    3      3  0.1s
     Testing Dve tests passed
```

Executable script outside the package

The script lives in `scripts/`, a workspace member with its own dependency (`ArgParse`, the closest match to Python's `argparse`).

File: `scripts/Project.toml`

```
[deps]
ArgParse = "c7e460c6-2fb9-53a9-8c5b-16f535851c63"
Dve = "5b3e2a8c-91d4-4f06-8a7e-2c1d9e6f4b30"

[compat]
ArgParse = "1"
```

File: `scripts/hello.jl`

```
#!/usr/bin/env julia
# Activate the environment that lives next to this file, independent of the
# caller's working directory. A plain `julia` shebang has no --project flag.
import Pkg
Pkg.activate(@__DIR__; io = devnull)

using ArgParse
```

```julia
using Dve

function parse_cli(args)
    s = ArgParseSettings(description = "Print a greeting built by Dve.hello_world.")
    @add_arg_table! s begin
        "--salutation", "-s"
            help = "greeting word"
            default = "Hello"
        "who"
            help = "who to greet"
            required = true
    end
    return parse_args(args, s)
end

function main(args = ARGS)
    opts = parse_cli(args)
    println(hello_world(opts["salutation"], opts["who"]))
    return 0
end

abspath(PROGRAM_FILE) == @__FILE__ && exit(main())
```

How the shebang finds the environment:

- `#!/usr/bin/env julia` only locates the interpreter.
- `Pkg.activate(@__DIR__)` (the `scripts/` folder, a workspace member) then makes `using Dve` resolve through the shared root `Manifest.toml`.
- A shebang with `-S julia --project=@script` may also work in recent versions. I could not verify it, so it is `[unverified]`.
- Alternatively, `#!/usr/bin/env -S julia --project=.` depends on the caller's current directory, which is fragile.

Shell, working directory `<project-root>`:

```
chmod +x scripts/hello.jl
julia --project=. -e 'using Pkg; Pkg.instantiate()'
./scripts/hello.jl --salutation Ciao mondo
./scripts/hello.jl --help
```

Expected output: `Ciao, mondo!`. The help text is ArgParse's usage line (approximately `usage: hello.jl [-s SALUTATION] [-h] who`) followed by the description and the argument list.

Jupyter kernel bound to the project

Install IJulia in the global environment, not in `Project.toml`. It is tooling, and the kernel finds it through the load-path stack (`@` → `@v1.13` → `@stdlib`).[5] The kernel is bound to your project by the `--project=@.` option, which makes Julia search upward from the kernel's working directory (the notebook folder) for a `Project.toml`.

Shell, working directory `<project-root>`:

```
julia -e 'using Pkg; Pkg.add("IJulia")'
julia -e 'using IJulia; installkernel("Julia Dve", "--project=@."; specname = "julia-dve")'
uv run jupyter kernelspec list
```

- Expected: a `julia-dve…` kernelspec appears in the list.
- IJulia chooses the Jupyter installation through the `JUPYTER` environment variable. To register the kernel with the uv-managed Jupyter, run the second command as `JUPYTER="$(uv run which jupyter)" julia -e '…'` (`[unverified]`, see IJulia's README).
- For Jupytext, a pairing such as `jupytext --set-kernel julia-dve notebook.md` should then work.

LSP support

How the three packages relate

They are components of one server, not alternatives.

- `LanguageServer.jl` is the process editors talk to. It implements the LSP wire protocol.[6]
- `JuliaWorkspaces.jl` is the analysis engine.[7] Its README states the target design: `LanguageServer.jl` keeps only the protocol layer and all functionality lives in `JuliaWorkspaces`.
- `SymbolServer.jl` indexed the symbols of installed packages without loading them, and `StaticLint.jl` provided scope and error analysis.
  - Verified: both GitHub repositories are archived (13 Jul 2026), with banners saying their functionality moved into `JuliaWorkspaces.jl`.[8][9]
  - They are now legacy components, not alternatives you choose between.
- Breaking change: a downstream project reports that `LanguageServer.jl` 6.0 (registered 2026-09-12) removed the `depot_path` argument of `runserver()`. Clients written against the 5.x call crash at startup.[10]
  - The same report mentions a SymbolServer patch needed for Julia 1.13 on the 5.x line.
  - I verified this only through that one report, so treat the details as `[unverified]`.

[5] IJulia.jl, https://github.com/JuliaLang/IJulia.jl
[6] LanguageServer.jl, https://github.com/julia-vscode/LanguageServer.jl
[7] JuliaWorkspaces.jl, https://github.com/julia-vscode/JuliaWorkspaces.jl
[8] SymbolServer.jl (archived 13 Jul 2026), https://github.com/julia-vscode/SymbolServer.jl
[9] StaticLint.jl (archived 13 Jul 2026), https://github.com/julia-vscode/StaticLint.jl
[10] positron-julia pull request 46, pinning LanguageServer.jl to 5.x after the 6.0 release, https://github.com/TidierOrg/positron-julia/pull/46

VS Code

Install the ”Julia” extension. It manages its own language-server environment, so you do not install `LanguageServer` yourself.

File: `.vscode/settings.json`

```
{
  "julia.environmentPath": "${workspaceFolder}"
}
```

Whether the extension picks up workspace members such as `test/` automatically is `[unverified]`.

Emacs: comparison of clients

Criterion	`eglot` + `eglot-jl`	`lsp-mo`
Purpose	Thin glue starting `LanguageServer.jl` under Emacs's built-in LSP client	Same s
Maintenance	`[unverified]`, check the repository's last commits[11]	`[unver`
Setup effort	Low: `eglot` is built into Emacs 29+, and `eglot-jl` bundles its own server environment	Mediun
Prefer when	You want minimal configuration	You alr

Both bundle or point to a pinned `LanguageServer` environment, so the 5.x/6.x break above applies to their defaults. If the server crashes at startup, pin `LanguageServer` to 5.x in that environment, or update the client.

File: `~/.emacs.d/init.el` (eglot variant, my recommendation)

```
(use-package julia-mode
  :ensure t
  :mode "\\.jl\\'")

(use-package eglot-jl
  :ensure t
  :after (eglot julia-mode)
  :hook (julia-mode . eglot-ensure)
  :init
  ;; use the environment bundled with eglot-jl
  (setq eglot-jl-language-server-project eglot-jl-base)
  ;; first start precompiles and indexes; the default 30 s is too short
  (setq eglot-connect-timeout 300)
  :config
  (eglot-jl-init))
```

File: `~/.emacs.d/init.el` (lsp-mode variant)

[11] eglot-jl, https://github.com/non-Jedi/eglot-jl
[12] lsp-julia, https://github.com/gdkrmr/lsp-julia

```
(use-package lsp-julia
  :ensure t
  :after lsp-mode
  :hook (julia-mode . lsp-deferred)
  :init
  ;; own environment holding LanguageServer, instead of the bundled one
  (setq lsp-julia-package-dir nil
        lsp-julia-default-environment "~/.julia/environments/lsp"))
```

Shell, working directory anywhere (creates the `lsp` environment used above):

```
julia --project="$HOME/.julia/environments/lsp" -e 'using Pkg; Pkg.add("LanguageServer")'
```

Keep editor tooling in named global environments like this. It stays out of `Project.toml` and out of the `uv`/`renv` lockfiles.

2. Julia project structure

Dependency management, UUIDs, compat, registries

Is `Pkg.add` enough? For adding a dependency, yes.

- `Pkg.add("X")` writes `X = "<uuid>"` into `[deps]`, resolves versions and records them in `Manifest.toml`.
- The UUID is the package's identity, not its name. It is why two packages with the same name can coexist.
- `[compat]` is not written by `Pkg.add`. Add it with `pkg> compat X 1` (or `Pkg.compat`) or by hand. It is required for registration and good practice otherwise.[13]
- `Pkg.develop(path = ".")` or `Pkg.develop("X")` tracks a local checkout instead of a registered version. Since 1.11, a `[sources]` table can also express this portably in `Project.toml` (see the Pkg docs).
- Registries are Git repositories mapping names to UUIDs and versions. `General` is the default, and private registries are added with `Pkg.Registry.add(url)`.
- Commit `Manifest.toml` for this repository: it is an application-style research repo, like committing `uv.lock` and `renv.lock`. Versioned `Manifest-v1.13.toml` files take precedence when present.

Python (uv)	R	Julia
`pyproject.toml`	`DESCRIPTION`	`Project.toml`
`uv.lock`	`renv.lock`	`Manifest.toml`
`uv add x`	`renv::install("x")`	`Pkg.add("x")`
`uv sync`	`renv::restore()`	`Pkg.instantiate()`
`uv run cmd`	`Rscript`	`julia --project=. script.jl`
`pip install -e .`	`devtools::load_all()`	`Pkg.develop(path = ".")`

[13] Pkg.jl manual, "Project.toml and Manifest.toml" (including Workspaces and `[sources]`), https://pkgdocs.julialang.org/v1/toml-files/

Python (uv)	R	Julia
dependency groups, extras	`Suggests`	test environment or workspace member
`[tool.uv.workspace]`	(none)	`[workspace]`
`pytest` / `testthat`	`devtools::test()`	`Pkg.test()`

Dev, test and LSP dependencies

Option	Purpose	Main
`[extras]` + `[targets]`	Test-only deps inside the main `Project.toml`	Lega
`test/Project.toml` alone	Separate test environment	Stabl
`[workspace]` with `test/Project.toml`	Same, but one shared manifest and `--project=test` works	New
Named global environments (`@lsp`, `@v1.13`)	Editor tooling, IJulia, Revise	Stabl

Recommendation: a workspace with `test/`, `scripts/` and `examples/` as members, because the target is 1.13 with no legacy constraint. LSP, IJulia and similar tooling live in global environments.

What is configured and what is convention

Item	Mechanism
Entry file `src/Dve.jl`	Convention enforced by Pkg: `name` plus `src/<name>.jl` (an `entryfile` key may exist in
`test/runtests.jl`	Convention hard-coded in `Pkg.test`
`test/Project.toml`	Convention, auto-detected by Pkg
`scripts/`, `examples/`	Free naming. They are known to Pkg only if listed in `[workspace] projects`
`[workspace] projects`	The only configured folder list in `Project.toml`

Workspaces

There are three unrelated meanings of ”workspace”.

- Pkg workspace (Julia 1.12+): a set of projects sharing one `Manifest.toml`, declared by `[workspace] projects = [...]` in the root `Project.toml`. Each folder needs its own `Project.toml`.[15]
- `JuliaWorkspaces.jl`: the language-server engine, an in-memory model of folders, projects and files for editors. The name is a collision only. It does not read `[workspace]` as an instruction to you.
- Editor workspace: a VS Code folder or multi-root workspace, or an Emacs `project.el` root.

Use cases: tests, docs, benchmarks and scripts that need extra dependencies without polluting the main project.

[14] Julia Discourse, ”How to use workspaces to test?”, https://discourse.julialang.org/t/how-to-use-workspaces-to-test/133670

[15] Pkg.jl manual, ”Project.toml and Manifest.toml” (including Workspaces and `[sources]`), https://pkgdocs.julialang.org/v1/toml-files/

Minimal example, already used above:

```
Project.toml      [workspace] projects = ["test", "scripts", "examples"]
test/Project.toml
scripts/Project.toml
examples/Project.toml
Manifest.toml     (single, at the root)
```

Shell, working directory <project-root>:

```
julia --project=scripts -e 'using Dve, ArgParse; println(hello_world("Hi", "ws"))'
```

Expected output: `Hi, ws!`. It works because the shared manifest knows `Dve` and `ArgParse`, even though they were declared in different `Project.toml` files. Pkg discourse threads state that a member needs the parent package in its own `[deps]` but no `[sources]` entry (`[unverified]` against the official docs).

3. Language integration examples

Choice of integration packages

Direction	Choice	Purpose
R to Julia	`JuliaConnectoR` (CRAN)	Runs Julia as a separate process, talks over TCP[16]
R to Julia	`JuliaCall` (CRAN/GitHub)	Embeds Julia in the R process, uses `RCall.jl` for
Julia to Python	`PythonCall.jl`	Calls Python from Julia[18]
Python to Julia	`juliacall` (PyPI, part of the same project)	Calls Julia from Python, managed by `juliapkg`[19][20]

Choice for R: `JuliaConnectoR`, since your R package must stay CRAN-compliant. Process isolation avoids the signal-handler and `libjulia` conflicts of embedding, and the package declares no system requirement.

Shared environment rules

- Python (uv): Julia must use the `uv` virtual environment and must not create a second Python.
  - `uv run` sets `VIRTUAL_ENV` and puts `.venv/bin` on `PATH`.
  - For `PythonCall`, disable its Conda management and pass the interpreter: `JULIA_CONDAPKG_BACKEND=Null` and `JULIA_PYTHONCALL_EXE=<path to .venv python>`.[21]
- R (renv): `JuliaConnectoR` is installed with `renv::install()`, and Julia is a system tool (`juliaup`), not duplicated per language.
- Julia from Python: `juliacall` looks for a Julia on `PATH` or installs its own via `juliaup`, into the venv.

[16] JuliaConnectoR on CRAN, https://cran.r-project.org/package=JuliaConnectoR
[17] JuliaCall (R package), https://github.com/JuliaInterop/JuliaCall
[18] PythonCall.jl documentation, https://juliapy.github.io/PythonCall.jl/stable/
[19] juliacall (Python side of PythonCall), https://juliapy.github.io/PythonCall.jl/stable/juliacall/
[20] pyjuliapkg (`juliapkg`), https://github.com/JuliaPy/pyjuliapkg
[21] CondaPkg.jl, https://github.com/JuliaPy/CondaPkg.jl

- To reuse your Julia, set `PYTHON_JULIAPKG_EXE` to it.
- To reuse your Julia project instead of a private one, set `PYTHON_JULIAPKG_PROJECT=examples`.
- Caveat: `juliapkg` may add `PythonCall` to that project, and its behaviour with workspace members is `[unverified]`. Hence `examples/Project.toml` already lists `PythonCall`.

First-run bootstrap, shell, working directory `<project-root>`:

```
juliaup add 1.13                                          # once per machine
uv sync                                                   # Python env, includes juliacall, pand
julia --project=. -e 'using Pkg; Pkg.instantiate()'       # Julia env, all workspace members
Rscript -e 'renv::restore()'                              # R env, includes JuliaConnectoR
```

Project package `dvepy`

File: `inst/python/dve/__init__.py`

```python
def greet(who: str) -> str:
    return f"Hello, {who}!"


def add_double(df):
    out = df.copy()
    out["double_x"] = out["x"] * 2
    return out
```

This assumes `pyproject.toml` already exposes `inst/python/dve` as the importable package `dve`. I did not see your `pyproject.toml`, so check that `uv run python -c "import dve"` works.

3.1 R to Julia

File: `examples/Project.toml`

```
[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
Dve = "5b3e2a8c-91d4-4f06-8a7e-2c1d9e6f4b30"
PythonCall = "6099a3de-0909-46bc-b1f4-468b9a2dfc0d"

[compat]
DataFrames = "1"
PythonCall = "0.9"
```

File: `examples/r_to_julia.R`

```r
library(JuliaConnectoR)

# Select the Julia environment from plain Julia code (works without extra options).
juliaEval('import Pkg; Pkg.activate("examples"; io = devnull)')

Dve <- juliaImport("Dve")
```

```r
# Primitive types
print(Dve$hello_world("Hello", "R"))        # "Hello, R!"
print(Dve$scaled_mean(c(1, 2, 3), 2))       # 4

# Complex type: data.frame in, data.frame out
df  <- data.frame(x = c(1, 2, 3), y = c(4, 5, 6))
res <- Dve$add_total(df)
print(as.data.frame(res))
```

Shell, working directory `<project-root>`:

```
Rscript examples/r_to_julia.R
```

Expected output:

```
[1] "Hello, R!"
[1] 4
  x y total
1 1 4     5
2 2 5     7
3 3 6     9
```

How it works:

- R objects are serialized and sent over a local TCP connection to a Julia server process.
- An R `data.frame` becomes an object implementing the Tables.jl interface. `DataFrame(tbl)` inside `add_total` builds a `DataFrame` from it.[22]
- The returned `DataFrame` stays in Julia as a proxy. `as.data.frame` pulls it back, which is a copy.
- Everything is copied: the serialization boundary means there is no zero-copy path.
- Details of scalar-versus-vector translation (length-1 R vectors become scalars) are `[unverified]`. See the package's translation table.
- Which Julia binary and project are used is decided by `PATH` (`juliaup`) and the `Pkg.activate` call above. The `JULIA_PROJECT` environment variable may also be honoured, `[unverified]`.

3.2 Julia to Python

File: `examples/julia_to_python.jl`

```julia
using PythonCall
using DataFrames
using Dve

# 1. Primitive types, third-party package
np = pyimport("numpy")
s = pyconvert(Float64, np.sqrt(2.0))                # Py → Float64 (copy of a scalar)
println(s)
```

[22] Tables.jl documentation, https://tables.juliadata.org/stable/

```
# Array: explicit copy vs zero-copy wrapper
x = np.linspace(0, 1, 5)
x_copy = pyconvert(Vector{Float64}, x)                # copy
x_view = PyArray(x)                                   # zero-copy view, shares memory
println(x_copy)

# 2. Project package dvepy (module "dve")
dve = pyimport("dve")
println(pyconvert(String, dve.greet("Julia")))

# 3. DataFrame → pandas → dvepy → back to DataFrame
df = DataFrame(x = [1.0, 2.0, 3.0], y = [4.0, 5.0, 6.0])
pdf = pytable(df)                                     # Julia table → pandas
res = dve.add_double(pdf)
println(DataFrame(PyPandasDataFrame(res)))            # pandas → Julia table
```

Shell, working directory `<project-root>`:

```
export JULIA_CONDAPKG_BACKEND=Null
export JULIA_PYTHONCALL_EXE="$(uv run python -c 'import sys; print(sys.executable)')"
uv run julia --project=examples examples/julia_to_python.jl
```

Expected output (abridged):

```
1.4142135623730951
[0.0, 0.25, 0.5, 0.75, 1.0]
Hello, Julia!
3×3 DataFrame
 Row │ x        y        double_x
     │ Float64  Float64  Float64
─────┼──────────────────────────
   1 │     1.0      4.0       2.0
   2 │     2.0      5.0       4.0
   3 │     3.0      6.0       6.0
```

Conversion and copy semantics:

- `pyconvert(T, x)` copies into a Julia value.
- `PyArray(x)` wraps a numpy array without copying, so both sides see the same memory.
- `pytable` and `PyPandasDataFrame` are the Tables.jl bridges to and from pandas. I treat their exact names and copy behaviour as `[unverified]`; check the PythonCall docs. Assume `pytable` copies into numpy, and that `DataFrame(PyPandasDataFrame(...))` copies columns unless you pass `copycols = false`.
- Fallback if those names differ: build the frame from a dict of columns. `pyimport("pandas").DataFrame(pydic ⇒ df.x, "y" ⇒ df.y)))` goes to pandas, and `pyconvert(Vector{Float64}, pdf["x"].to_numpy())` per column comes back.

3.3 Python to Julia

File: `examples/python_to_julia.py`

```python
import numpy as np
import pandas as pd
from juliacall import Main as jl

# Load Julia packages once; Dve comes from the project environment.
jl.seval("using Statistics, DataFrames, PythonCall, Dve")

# 1. Primitive types and the project package Dve
print(jl.hello_world("Hello", "Python"))              # Julia String → Python str

# 2. Third-party Julia package (Statistics)
arr = np.array([1.0, 2.0, 3.0, 4.0])
print(jl.Statistics.mean(arr))                        # 2.5

# 3. pandas DataFrame → Julia DataFrame → Dve → pandas
pdf = pd.DataFrame({"x": [1.0, 2.0, 3.0], "y": [4.0, 5.0, 6.0]})
jdf = jl.DataFrame(jl.PyPandasDataFrame(pdf))         # pandas → Julia table
res = jl.add_total(jdf)                               # Dve.add_total
print(jl.pytable(res))                                # Julia table → pandas
```

Shell, working directory `<project-root>`:

```
export PYTHON_JULIAPKG_EXE="$(which julia)"
export PYTHON_JULIAPKG_PROJECT="$PWD/examples"
uv run python examples/python_to_julia.py
```

Expected output (abridged):

```
Hello, Python!
2.5
     x    y  total
0  1.0  4.0    5.0
1  2.0  5.0    7.0
2  3.0  6.0    9.0
```

How it works:

- `from juliacall import Main as jl` starts Julia inside the Python process (embedded).
- `juliapkg` resolves which Julia and which project to use. The two environment variables above reuse your Julia binary and your project, so nothing is installed twice. The variable names are `[unverified]`; see the `juliapkg` README.
- Python arguments are converted before the call. A numpy array should reach Julia as a wrapped array without copying (`[unverified]`; check PythonCall's conversion rules page).
- Julia results converted for Python come back as `str`/`float` for primitives. Julia arrays are

exposed as wrapper objects, and `np.asarray(...)` on a dense `Array` can share memory.
- DataFrame conversion copies columns in both directions, as in section 3.2.

Python/R to Julia equivalents

Task	Python	R	Julia
String formatting	`f"{a}, {b}!"`	`sprintf("%s, %s!", a, b)`	`string(a, ", ", b, "!")`
Mean of a vector	`np.mean(v)`	`mean(v)`	`mean(v)` (from `Statistics`)
Data frame	`pandas.DataFrame`	`data.frame`	`DataFrames.DataFrame`
Add a column	`df["t"] = df.x + df.y`	`df$t ← df$x + df$y`	`df.t = df.x .+ df.y`
Elementwise op	implicit (numpy)	implicit	explicit broadcast with `.`
Run tests	`pytest`	`devtools::test()`	`Pkg.test()`

Acceptance checklist

Run from `<project-root>` on a clean checkout. Each item lists the expected result.

- Julia installed: `julia --version` prints `julia version 1.13.x`.
- Julia environment resolves: `julia --project=. -e 'using Pkg; Pkg.instantiate()'` finishes without errors and creates `Manifest.toml`.
- Package loads: `julia --project=. -e 'using Dve; println(hello_world("Hello", "Julia"))'` prints `Hello, Julia!`.
- Unit test: `julia --project=. -e 'using Pkg; Pkg.test()'` ends with `Dve tests passed`.
- Script: `./scripts/hello.jl -s Ciao mondo` prints `Ciao, mondo!`, and `--help` prints usage.
- Kernel: `uv run jupyter kernelspec list` shows `julia-dve…`, and a notebook with that kernel runs `using Dve` successfully.
- R to Julia: `Rscript examples/r_to_julia.R` prints the three-row data frame with `total` 5, 7, 9.
- Julia to Python: the `uv run julia` command in 3.2 prints `Hello, Julia!` and the 3×3 `DataFrame`.
- Python to Julia: `uv run python examples/python_to_julia.py` prints `Hello, Python!`, `2.5` and the pandas frame.
- R package stays clean: `R CMD build . && R CMD check --as-cran dvesimpler_*.tar.gz` reports no new NOTEs about `Project.toml`, `Manifest.toml`, `src/*.jl`, `test`, `scripts` or `examples`.
- Python tests still pass: `uv run pytest tests/pytest`.

Common pitfalls

- `Pkg.test()` cannot see `tests/`. Use `test/runtests.jl`.
- Forgetting the `.Rbuildignore` lines produces R CMD check NOTEs about non-standard files.
- Julia keeps compiling on first use: the first `using Dve` or `using PythonCall` in a fresh depot is slow. Run `Pkg.instantiate()` once and let precompilation finish. Julia 1.13 is reported to

precompile about 30% faster than 1.12.[23]

- `JuliaConnectoR` and `PythonCall` both need `Dve` loadable in the active environment. If `juliaImport("Dve")` fails, you probably did not activate `examples/`.
- PythonCall silently creating its own Conda environment: set `JULIA_CONDAPKG_BACKEND=Null` and `JULIA_PYTHONCALL_EXE` before the first `using PythonCall`. Setting them afterwards has no effect.
- Embedding Julia in Python (`juliacall`) and Python in Julia (`PythonCall`) in the same process is possible but easy to get wrong. Keep the direction one-way per script.
- 1-based indexing, column-major arrays and `.` broadcasting trip up Python and R programmers. Index and slice conversions deserve explicit tests.
- LSP breakage after upgrades: `LanguageServer.jl` 6.0 changed `runserver()`, so pin or update editor clients together with the server.[24]
- Never commit editor tooling (`LanguageServer`, `IJulia`, `Revise`) into `Project.toml`.
- R CMD check on CRAN machines: Julia is absent there, so R code that calls Julia must be guarded (`requireNamespace`, `skip_on_cran()` in testthat).
- Julia workspaces need Julia 1.12+. Older versions silently ignore or misread `[workspace]`.
- Version-specific manifests (`Manifest-v1.13.toml`) shadow `Manifest.toml` without warning. Decide once which one you commit.

Unverified items and documentation to check

- Julia 1.13.1 is tagged and installable via `juliaup add 1.13`. Check the Julia announcements and `juliaup` docs.
- `-S julia --project=@script` shebang support. Check the Julia command-line manual.
- `entryfile` key in `Project.toml`. Check the Pkg TOML-files documentation.
- Workspace member needing no `[sources]` entry for the parent package. Check the Pkg workspaces section.
- Whether the VS Code Julia extension handles workspace members automatically, and the exact setting `julia.environmentPath`. Check the extension documentation.
- Maintenance status of `eglot-jl` and `lsp-julia`. Check each repository's recent commits.
- `LanguageServer.jl` 6.0 details (changed `runserver` signature, Julia 1.13 SymbolServer patch for 5.x). Check the release notes and the downstream issue.
- `JUPYTER` environment variable honoured by `IJulia.installkernel`. Check the IJulia README.
- `JuliaConnectoR` translation rules (length-1 vectors, data frames to Tables objects, `JULIA_PROJECT` handling). Check the package reference manual.
- `pytable`, `PyPandasDataFrame` and `PyArray` names, and their copy semantics. Check the PythonCall documentation.
- numpy to Julia default conversion being a zero-copy `PyArray`. Check the PythonCall conversion-rules page.
- `PYTHON_JULIAPKG_EXE` and `PYTHON_JULIAPKG_PROJECT` names, and `juliapkg` behaviour with workspace members. Check the `juliapkg` README.
- `inst/python/dve` exposed as importable `dve` through your `pyproject.toml`. Check your own

[23] Julia 1.13 Highlights, 10 Sep 2026, https://julialang.org/blog/2026/09/julia-1.13-highlights/

[24] positron-julia pull request 46, pinning LanguageServer.jl to 5.x after the 6.0 release, https://github.com/TidierOrg/positron-julia/pull/46

packaging configuration.

Additional Notes

- Shipping Julia code inside the R package: for `dvesimpler` to call `Dve` after `R CMD INSTALL`, the Julia sources must live under `inst/julia/` in the tarball, because `src/*.jl` is ignored. Keep the repository root as the development source of truth and generate or copy the `inst/julia` tree in a build step. Declare `JuliaConnectoR` in `Suggests`, never `Imports`.
- Package extensions: keep `Dve` free of `PythonCall`. If you later want built-in pandas conversion helpers, use `[weakdeps]` and `[extensions]` so the dependency loads only when `PythonCall` is present.
- Jupytext and GLFM: Jupytext supports Julia notebooks, so the `julia-dve` kernel fits your Markdown-paired workflow.
- CI (GitLab): cache `~/.julia` and the `uv` cache, and run `Pkg.test()` and `pytest` in the same Podman image. Pin the Julia version through `juliaup` or a base image.
- Julia 1.13 also adds REPL syntax highlighting and `@__FUNCTION__`, per the release highlights.[25]

Revision: in-process R to Julia, large data, dual runtime

> Reconsider R-Julia integration with in-process model provided by "JuliaCall", discussing zero-copy scenario for big memory dataframes, or even lazy models, as those implemented in Rust based "Polars" dataframes.
>
> In addition, consider the dual project runtime environment, sharing the same codebase:
>
> - native "out-of-container" with python and julia
> - in-container "podman" rootless container with R in addition to julia and python
>
> externally, home is ordinary user home, in container home is mapped to a project subdirectory uv virtual environments, R renv environments and ~/.julia directory are different
>
> the point is find a way to introduce an optional julia dependency: "RCall" where R is available only in container runtime
>
> what to include in Project.toml, or define a julia workspace, that can be operated only in-container runtime

This supersedes the R-to-Julia choice and the workspace advice in the earlier sections where they conflict. The rest (skeleton, tests, LSP, Python bridges) is unchanged.

1. R to Julia reconsidered: JuliaCall

What changes with the in-process model

`JuliaConnectoR` (earlier choice) runs Julia in a separate process and serializes everything over TCP. It can never be zero-copy. `JuliaCall` embeds Julia in the R process, so shared memory becomes

[25] Julia 1.13 Highlights, 10 Sep 2026, https://julialang.org/blog/2026/09/julia-1.13-highlights/

possible. That is the right trade once large data frames matter.

Criterion	`JuliaCall` (in-process)	`Jul`
Purpose	Julia embedded in the R process, conversions through `RCall.jl`[26]	Juli
Maintenance	`[unverified]`, check recent CRAN and GitHub activity	Mai
Setup effort	Higher: needs a Julia matching R's build, plus `RCall.jl` precompiled against the current R	Low
Prefer when	Large numeric data, low latency, shared memory	Isol

Two consequences for your dual-runtime design, both verified against the `julia_setup()` documentation:[27]

- `JuliaCall` depends on `RCall.jl` on the Julia side. It is not optional in the container; it is infrastructure of the bridge.
- `julia_setup()` has `install = TRUE` (installs the dependent Julia packages), `useRCall = TRUE` and `rebuild = FALSE`. The package's own setup code notes that `RCall` must be precompiled with the current R. So the Julia depot is bound to the R installation. This is a strong argument for separate depots per runtime (section 4).

In-process example

File: `examples/r_juliacall.R`

```r
library(JuliaCall)

# Environment pre-provisioned (see section 4): do not let JuliaCall install anything.
julia_setup(install = FALSE)
julia_command("using Dve")

df <- data.frame(x = c(1, 2, 3), y = c(4, 5, 6))

julia_assign("df", df)                          # R → Julia: copy by default
h <- julia_eval("Dve.add_total(df)", "Julia")   # result stays in Julia (a handle)
julia_eval("size(Dve.add_total(df))")           # only a small result crosses over
```

- The second argument of `julia_eval` selects where the value ends up. `"Julia"` returns a reference instead of converting to R. This is the main tool for keeping big objects out of R: R holds a handle and pulls only small results. I verified the `"Julia"` option from forum examples.
- Whether `julia_assign` turns an R `data.frame` into a Julia `DataFrame` automatically is `[unverified]`. `Dve.add_total` accepts any Tables.jl source, so it works either way.

2. What zero-copy means between R and Julia

Memory layouts

[26] JuliaCall README, https://github.com/JuliaInterop/JuliaCall
[27] JuliaCall reference, `julia_setup`, https://search.r-project.org/CRAN/refmans/JuliaCall/html/julia_setup.html

R object	R layout	Julia coun
`numeric`	contiguous float64, `NA` is a NaN payload	`Vector{Fl`
`integer`	int32, `NA` is `INT_MIN`	`Vector{In`
`logical`	int32	`Vector{Bo`
`character`	pointers into a global string cache	`Vector{St`
`factor`	int32 codes plus a levels attribute	`Categoric`
`data.frame`	list of column vectors plus attributes	`DataFrame`
ALTREP objects (compact sequences, deferred strings)	Lazy; data pointer materializes on demand	n/a

Direction by direction

- Julia to R: verified zero-copy path. The CRAN package `jlview` returns ALTREP vectors whose data pointer points into Julia memory, integrates with `JuliaCall`, and pins the Julia array until R's garbage collector finalizes it.[28] Its vignette states that plain `JuliaCall` transfers copy, and reports a 9.3 GB matrix case dropping from seconds to milliseconds. Writes to such a vector materialize an ordinary R copy, so R semantics are preserved.
- R to Julia: `JuliaCall` converts with `RCall.rcopy`, which copies. A forum thread by a maintainer says wrapping R's raw bits is possible in principle and mentions `RCall.protect` and `unsafe_array`.[29] I could not confirm a supported, documented zero-copy R-to-Julia API, so treat it as `[unverified]` and do not build on it.

Hazards of shared memory

- R is copy-on-modify with reference counting. A Julia function mutating a vector borrowed from R changes an object R believes is immutable. Treat borrowed data as read-only (convention only: Base Julia has no read-only array type).
- The two garbage collectors do not know each other. The owner must outlive every borrower. `jlview` handles this for Julia-owned memory. For R-owned memory you must keep the R object protected yourself.
- `saveRDS`, serialization and many base-R operations materialize ALTREP vectors. Verify with `.Internal(inspect(x))` or by monitoring RSS that no copy happened.
- `NA` semantics differ (the table above). Never hand a borrowed integer column to Julia code that expects `missing`.

3. Columnar and lazy models (Polars-style)

Why borrowing numeric vectors does not scale to data frames

Strings, factors, logical columns and `NA` handling all defeat direct sharing. Laziness (query plans, predicate and projection pushdown) lives in the engine, not in the memory layout. Polars is a lazy engine over an Arrow-style columnar layout. The portable common denominator across R, Julia and Python is therefore Apache Arrow, not R's or Julia's native vectors.

[28] jlview, zero-copy Julia arrays in R via ALTREP, https://github.com/tanaylab/jlview
[29] Julia Discourse, "Fast 4D argmax" (discussion of JuliaCall copying), https://discourse.julialang.org/t/fast-4d-argmax/58566

Three levels

Level	Mechanism	Zero-copy?
A	Borrow numeric columns in-process (`jlview` for Julia to R)	Julia to R yes, R
B	Shared uncompressed Arrow IPC (Feather v2) files, memory-mapped by every reader	Reads yes, one m
C	A lazy engine per runtime over the same files (Polars, DuckDB)	Engine-internal

Level B details:

- `Arrow.jl` reads files through `Mmap` and exposes columns as Arrow-backed vectors. `DataFrame(Arrow.Table(file))` uses those vectors directly, and `copy(...)` materializes ordinary vectors.[30] The DataFrames documentation warns that Arrow-backed columns are immutable, so pass `copycols = true` when you need mutable ones.[31]
- Compression defeats mmap. Write files uncompressed.
- Strings come back as Arrow list views, and converting to Julia `String` copies per element.
- Writers: R `arrow::write_feather(df, path, compression = "uncompressed")`, Python Polars `LazyFrame.sink_ipc(path)`, Julia `Arrow.write(path, tbl)`. Readers: R `arrow::read_feather(path, mmap = TRUE)`, Python `pl.scan_ipc(path)`, Julia `Arrow.Table(path)`. All reader and writer argument names here are `[unverified]`; check each package's documentation.

File: `examples/arrow_exchange.jl`

```
using Arrow, DataFrames

tbl = Arrow.Table("data/cache/big.arrow")        # mmap, no copy
df  = DataFrame(tbl)                             # columns are the Arrow vectors (read-only)
out = combine(groupby(df, :g), :x ⇒ sum ⇒ :sx) # computed result is a normal in-memory frame
Arrow.write("data/cache/result.arrow", out)      # one materialization, readable by R and Python
```

The path layout assumes `data/cache/` is on the project bind mount, visible to both runtimes. On a Linux host, native and container processes mapping the same file share the page cache. On macOS the Podman VM breaks this sharing (`[unverified]`).

Findings on lazy engines in Julia:

- Julia has no Polars binding that I could verify. Treat that as `[unverified]` and check the Julia package registry. Options are Python Polars through `PythonCall` (already in your toolchain), or a DuckDB client on the same files.
- `Arrow.jl` is a pure-Julia implementation. I found no Arrow C Data Interface support, so in-process pointer exchange with R `arrow` or `pyarrow` is `[unverified]` and should be assumed unavailable.

Recommendation: make level B the contract between runtimes and use level A only for dense numeric arrays where measured copies hurt.

[30] Arrow.jl user manual, https://arrow.apache.org/julia/stable/manual

[31] DataFrames.jl constructor documentation (`copycols` and immutable Arrow columns), https://dataframes.juliadata.org/stable/

4. Dual runtime and the optional RCall dependency

Differences between the runtimes

Resource	Native (Python and Julia)	Container (R, Julia and Python)
`HOME`	ordinary user home	project subdirectory (for example `<project>/.container-ho`
uv environment	`.venv`	must not reuse the native one
Julia depot	`~/.julia`	`$HOME/.julia` (inside the project mount)
`renv`	not present (no R)	`renv` library under the container home
Julia manifests	root `Manifest.toml`, no `RCall`	separate `RCall` environment, own manifest

Why not put RCall in Project.toml or a workspace member

- Root `[deps]`: it would be installed and precompiled in the native runtime where there is no R.
- A `[workspace]` member: all members share one root `Manifest.toml`, so `Pkg.instantiate()` natively would still resolve and install `RCall` and its dependencies. The expected failure is a precompile error without R (`[unverified]`, check the `RCall.jl` installation notes). The `[workspace] projects` list is also static, so it cannot be conditional on the runtime.
- Conclusion: a workspace is the wrong tool for dependencies that exist in only one runtime. Use a separate, non-workspace environment, plus an optional extension if code in `Dve` itself must talk to R.

Layout additions

```
<project-root>
  Project.toml              # Dve, workspace: test, scripts, examples (no RCall)
  Manifest.toml             # native manifest
  ext/DveRCallExt.jl        # optional extension, loads only if RCall is present
  envs/rcall/Project.toml   # container-only environment: Dve (path) + RCall
  envs/rcall/Manifest.toml  # generated and committed from the container
  .container-home/          # container HOME, never committed, never in the R tarball
```

Name the directory `envs/rcall`, not `r/`. On case-insensitive filesystems `r/` collides with `R/`.

File: `.Rbuildignore` (append these lines)

```
^envs$
^ext$
^\.container-home$
```

Container-only environment

Dve is brought in by path, so the root manifest is untouched.

File: `envs/rcall/Project.toml`

```
[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
Dve = "5b3e2a8c-91d4-4f06-8a7e-2c1d9e6f4b30"
RCall = "6f49c342-dc21-5d91-9882-a32aef131414"

[sources]
Dve = {path = "../.."}

[compat]
DataFrames = "1"
```

- Copy the `RCall` UUID from the result of `Pkg.add("RCall")` in this environment rather than trusting my transcription. Add `RCall` to `[compat]` with the version that command resolves.
- `[sources]` with `path` requires Julia 1.11 or later (fine for 1.13). Whether Pkg ignores the `[workspace]` table of a dependency loaded by path is `[unverified]`.
- Create it once inside the container, working directory `<project-root>`:

```
julia --project=envs/rcall -e 'using Pkg; Pkg.add("RCall"); Pkg.instantiate()'
```

- `JuliaCall` also needs its own helper Julia packages. Run `julia_setup(install = TRUE)` once with `JULIA_PROJECT=envs/rcall`, let it populate the environment, commit the resulting `envs/rcall/Manifest.toml`, and use `install = FALSE` afterwards. Exactly which packages it adds is `[unverified]`.

Optional code-level integration (only if Dve must call R)

Two different cases are easy to confuse.

- R hosts Julia (`JuliaCall`): `RCall` is part of the bridge. `Dve` never imports it. The `envs/rcall` environment is all you need.
- Julia hosts R (`RCall.jl`, for example a Julia script calling an R package): `Dve` needs optional code. Use a weak dependency plus an extension, so `RCall` is neither installed nor loaded unless the active environment contains it.

File: `Project.toml` (additions to the existing file)

```
[weakdeps]
RCall = "6f49c342-dc21-5d91-9882-a32aef131414"

[extensions]
DveRCallExt = "RCall"
```

File: `src/Dve.jl` (add inside the module, and extend `export`)

```
"Convert an R object to a DataFrame. Methods are added by the RCall extension."
function from_r end

"Convert a DataFrame to an R object. Methods are added by the RCall extension."
function to_r end
```

File: `ext/DveRCallExt.jl`

```
module DveRCallExt

using Dve
using RCall
using DataFrames: DataFrame

Dve.from_r(x::RObject) = rcopy(DataFrame, x)   # copies
Dve.to_r(df::DataFrame) = robject(df)          # copies

end # module
```

- Native runtime: `RCall` is only a weak dependency, so the resolver does not install it and `Dve.from_r` is just a stub without methods. Calling it raises `MethodError`.
- `[compat]` for weak dependencies follows the same rules as for normal ones.
- That `rcopy(DataFrame, ...)` and `robject(::DataFrame)` work without extra imports is `[unverified]`. Check the `RCall.jl` conversion documentation.

Runtime configuration

Runtime selection happens through environment variables, not through `Project.toml`.

Variable	Native	Container
`DVE_RUNTIME`	`native`	`container` (set in the image)
`HOME`	user home	`<project>/.container-home`
`UV_PROJECT_ENVIRONMENT`	unset (`.venv`)	`$HOME/venv`
`JULIA_DEPOT_PATH`	unset	unset (resolves to `$HOME/.julia`)
`JULIA_PROJECT`	`.`	`.` for Julia work, `envs/rcall` for R sessions
`JULIA_CONDAPKG_BACKEND`	`Null`	`Null`
`JULIA_PYTHONCALL_EXE`	`.venv/bin/python`	`$HOME/venv/bin/python`
`RENV_PATHS_ROOT`	not applicable	`$HOME/renv`

Shell, working directory `<project-root>` (host side):

```
podman run --rm -it --userns=keep-id \
  -v "$PWD":/work:Z -w /work \
  -e HOME=/work/.container-home \
  -e DVE_RUNTIME=container \
  -e UV_PROJECT_ENVIRONMENT=/work/.container-home/venv \
  -e RENV_PATHS_ROOT=/work/.container-home/renv \
  -e JULIA_CONDAPKG_BACKEND=Null \
  -e JULIA_PYTHONCALL_EXE=/work/.container-home/venv/bin/python \
  dve-dev:latest bash
```

- `--userns=keep-id` maps your host user into the rootless container so files on the bind mount

keep sane ownership, and `:Z` relabels the mount for SELinux.

- The `dve-dev` image name is a placeholder for your own image. It must contain R built with a shared library (`libR`, needed for embedding), Julia, uv and Python. Rocker-style R images normally qualify; `[unverified]` for your base image.

File: `R/julia.R`

```
dve_julia_setup <- function(project = file.path("envs", "rcall")) {
  if (Sys.getenv("DVE_RUNTIME") ≠ "container") {
    stop("Julia-from-R is only supported in the container runtime.", call. = FALSE)
  }
  if (!requireNamespace("JuliaCall", quietly = TRUE)) {
    stop("Package 'JuliaCall' is required.", call. = FALSE)
  }
  # Must be set before the first julia_setup(): the embedded Julia reads it at startup.
  Sys.setenv(JULIA_PROJECT = normalizePath(project, mustWork = TRUE))
  JuliaCall::julia_setup(install = FALSE)
  JuliaCall::julia_command("using Dve")
  invisible(TRUE)
}
```

- `JuliaCall` goes in `Suggests`, never `Imports`, and R tests that use it start with `skip_on_cran()` and `skip_if(Sys.getenv("DVE_RUNTIME") ≠ "container")`.
- That the embedded Julia honors `JULIA_PROJECT` at startup is `[unverified]`. Fallback: run `julia_command('import Pkg; Pkg.activate("envs/rcall")')` right after `julia_setup()`, understanding that `RCall` must then be found on the stacked global environment.
- After the first `julia_setup()` the project cannot be changed cleanly. Restart the R session instead.

Acceptance checklist (additions)

Run inside the container unless stated otherwise.

- Native, working directory `<project-root>`: `julia --project=. -e 'using Pkg; Pkg.instantiate()'` succeeds with no R installed, and `grep -c RCall Manifest.toml` prints `0`.
- Native: `julia --project=. -e 'using Pkg; Pkg.test()'` passes (no `RCall` involved).
- Container: `julia --project=envs/rcall -e 'using RCall; using Dve; println(Dve.hello_world("He`
  prints `Hello, R!` and `Dve.from_r` has a method (`methods(Dve.from_r)` is non-empty).
- Container: `Rscript -e 'source("R/julia.R"); dve_julia_setup(); JuliaCall::julia_eval("1+1")'`
  prints `[1] 2`.
- Container: `Rscript examples/r_juliacall.R` prints the size of the result without converting the whole frame.
- Native and container: both read `data/cache/big.arrow` with `examples/arrow_exchange.jl` and agree on the result. Check no per-process memory growth proportional to the file size.
- `R CMD build .` produces a tarball without `.container-home`, `envs` or `ext`.

Additional pitfalls

- A bind-mounted `.container-home` is large (depot, venv, renv). Without the `.Rbuildignore`, `.gitignore` and container-ignore entries it gets bundled or committed.
- A venv created natively and used in the container (or the reverse) fails with broken interpreter links. Keep `UV_PROJECT_ENVIRONMENT` different per runtime.
- A Julia depot copied between runtimes can carry precompile caches built against a different R or libc. Rebuild with `JuliaCall::julia_setup(rebuild = TRUE)` when R changes.
- Forking R workers (`parallel::mclapply`) with an embedded Julia running is unsafe. Use socket clusters, or separate processes with the Arrow files as the interface.
- Borrowed memory plus `Arrow.Table` columns are read-only. Mutating them is either an error or undefined behaviour.

Additional unverified items

- Supported zero-copy R to Julia path (`unsafe_array`, `RCall.protect`). Check the `RCall.jl` documentation.
- `julia_assign` converting `data.frame` to `DataFrame` automatically. Check the `JuliaCall` reference.
- Which Julia packages `julia_setup(install = TRUE)` installs, and embedded `JULIA_PROJECT` handling. Check `JuliaCall`'s `julia_setup` documentation and issue tracker.
- Reader and writer argument names for R `arrow` and Python Polars. Check each package's documentation.
- Page-cache sharing of mmapped files between host and container on macOS. Check Podman machine documentation.
- Julia Polars bindings and Arrow C Data Interface support in `Arrow.jl`. Check the Julia registry and the `Arrow.jl` issue tracker.
- `[sources]` path dependency whose own `Project.toml` has a `[workspace]` table. Check the Pkg documentation.
- `RCall` and `JuliaCall` maintenance status and compatibility with Julia 1.13. Check their repositories before pinning.

A:1 (Gemini)

^ *Julia Repository Layout*

- see: gemini

1. Julia Project Skeleton

Project Configuration and Main Module

Project dependencies and metadata are managed via `Project.toml`. Unlike Python's `pyproject.toml` or R's `DESCRIPTION`, Julia tracks exact dependency trees in `Manifest.toml`.

Project.toml

```
name = "Dve"
uuid = "11111111-2222-3333-4444-555555555555"
authors = ["Research Team <dev@example.com>"]
version = "0.1.0"

[deps]
PythonCall = "6099a3de-0909-46ab-b9f4-48469b6d42fb"

[compat]
julia = "1.13.1"
PythonCall = "0.9"
```

src/Dve.jl

```
module Dve

export hello_world

"""
    hello_world(salutation::String, who::String) → String

Returns a formatted greeting string. Contrast with Python's f-strings or R's sprintf.
"""
function hello_world(salutation::String, who::String)::String
    return "$(salutation), $(who)!"
end
```

```
end # module Dve
```

Jupyter Kernel Support

To integrate Julia with Jupyter while binding strictly to the project environment, use the `IJulia` package. Run this once in a Julia REPL activated at the project root:

```
using Pkg
Pkg.add("IJulia")
using IJulia
# Bind kernel explicitly to the current project environment path
installkernel("Julia (Dve)", project=pwd())
```

Executable Script

Scripts located outside the package source directory can load the local package environment using absolute or relative path activation.

bin/run_dve.jl

```
#!/usr/bin/env julia
#=
# Activate the project environment located in the parent directory
=#
if abspath(PROGRAM_FILE) == @__FILE__
    using Pkg
    Pkg.activate(joinpath(@__DIR__, ".."))
end

using ArgParse
using Dve

function parse_command_line()
    s = ArgParseSettings(description = "Run Dve greeting utility.")
    @add_arg_table! s begin
        "--salutation", "-s"
            help = "Greeting salutation word"
            default = "Hello"
        "--who", "-w"
            help = "Target recipient"
            default = "World"
    end
    return parse_args(s)
end
```

```
function main()
    args = parse_command_line()
    result = hello_world(args["salutation"], args["who"])
    println(result)
end

main()
```

Unit Testing

Tests reside in a separate test directory and use Julia's built-in `Test` standard library.

tests/test_dve.jl

```
using Test
using Dve

@testset "Dve Unit Tests" begin
    @test hello_world("Hi", "Julia") == "Hi, Julia!"
    @test_throws MethodError hello_world(123, "Julia")
end
```

Run tests from the project root via shell:

```
julia --project -e 'using Test; include("tests/test_dve.jl")'
```

LSP Editor Support

Julia language support in editors relies on three core components:

- `LanguageServer.jl`: Implements the Language Server Protocol (LSP) protocol handling.
- `SymbolServer.jl`: Indexes package source code and metadata in the background to provide fast autocomplete without executing user code.
- `JuliaWorkspaces`: [unverified] Check official `julia-vscode` or `LanguageServer.jl` documentation regarding workspace partitioning [*ag-lsp*].

For VS Code, the official `julia-vscode` extension automatically manages `LanguageServer.jl` and `SymbolServer.jl`.

For Emacs, compare the two main integration options:

- `lsp-mode` + `lsp-julia`: Feature-rich, heavy client ecosystem. Requires manual server path configuration.
- `eglot` + `eglot-jl`: Lightweight, built-in Emacs client (`eglot` is built-in since Emacs 29), with `eglot-jl` bootstrapping the Julia language server automatically. This is the recommended

modern approach.

Emacs configuration snippet (`use-package` with `eglot`):

.emacs.d/init.el

```
(use-package eglot
  :ensure t
  :hook (julia-mode . eglot-ensure))

(use-package eglot-jl
  :ensure t
  :config
  (eglot-jl-init))
```

---

2. Julia Project Structure

Dependency Management and UUIDs

Julia manages dependencies via `Project.toml` (declarative dependencies and compatibility bounds) and `Manifest.toml` (fully resolved dependency graph with exact commit hashes/versions, similar to `uv.lock` or `renv.lock`).

- `Pkg.add("PkgName")` updates both files.
- Local packages are integrated using `Pkg.dev("path/to/package")` or `Pkg.develop(path=...)`.
- Registries (such as the General registry) serve as package lookup sources, analogous to PyPI or CRAN.

Dev, Test, and LSP Dependencies

Dependencies required exclusively for testing or development should be specified using modern project targets or separate project environments.

- `[extras]` and `[targets]` tables in `Project.toml` allow defining test-only dependencies without bloating production manifests [*ag-pkg*].
- Alternatively, maintaining a separate `test/Project.toml` isolates test dependencies entirely.

Recommendation: Use `test/Project.toml` for large research codebases to maintain strict separation between production runtimes and testing tools.

Source, Test, and Script Folders

Julia package conventions dictate:

- `src/`: Contains the primary module files (e.g., `src/Dve.jl`). The folder structure is a convention, but the module name must match the package name declared in `Project.toml`.
- `test/` or `tests/`: Contains test suites.
- Scripts and notebooks reside outside `src/` to prevent package loading issues.

Workspaces

Pkg workspaces (`[workspace]` table in `Project.toml`) allow multiple interdependent local packages to be developed simultaneously within a single active session without continuous `Pkg.develop` calls [*ag-workspaces*].

Project.toml (Workspace Example)

```
[workspace]
packages = ["."]
```

---

3. Language Integration Examples

1. R to Julia (`RCall.jl`)

To call Julia from R, use the `RCall` R package [*ag-rcall*]. It embeds a Julia interpreter directly inside the R session.

Primitive and Complex Types

R/julia_bridge.R

```r
library(RCall)

# Primitive argument passing
res <- juliaEval("sqrt(16.0)")
print(res)

# Complex types: data.frame to DataFrames.DataFrame
juliaEval("using DataFrames")
df_r <- data.frame(a = c(1, 2, 3), b = c("x", "y", "z"))

# Pass R data.frame to Julia
julia_df <- juliaDataset(df_r)
print(juliaEval("typeof(julia_df)"))

# Return back to R
back_to_r <- juliaEval("julia_df")
print(back_to_r)
```

Conversion Mechanics and Bootstrapping

- `RCall` maps R vectors to Julia vectors and R `data.frame` objects to Julia `DataFrames.DataFrame` instances using shallow wrapping or copying depending on mutability constraints. Zero-copy semantics apply to numeric vectors where memory is shared via pointers.
- Bootstrapping: R loads `RCall`, which launches libjulia. The first run automatically builds or links against the local Julia installation.

---

2. Julia to Python (`PythonCall.jl`)

To call Python from Julia, `PythonCall.jl` is the modern, high-performance successor to `PyCall.jl` [*ag-pythoncall*]. It handles automatic conversion and environment mapping.

Primitive Types, DataFrames, NumPy, and `dvepy`

src/julia_to_python.jl

```julia
using PythonCall

# Ensure PythonCall binds to the uv environment
# PythonCall reads UV_PROJECT_ENVIRONMENT or VIRTUAL_ENV automatically.
const pd = pyimport("pandas")
const np = pyimport("numpy")
const dvepy = pyimport("dvepy")

# Primitive types
x = pyint(10)
y = pyconvert(Int, x + 5)
println(y)

# Complex types: Julia DataFrame to pandas DataFrame and back
using DataFrames
df_julia = DataFrame(A = [1, 2], B = [3.5, 4.5])

# Convert Julia DataFrame to pandas DataFrame via dictionaries/numpy arrays
df_py = pd.DataFrame(pydict(Dict("A" ⇒ pylist(df_julia.A), "B" ⇒ pylist(df_julia.B))))
println(df_py)

# Call numpy and project python package 'dvepy'
arr = np.array(pylist([1.0, 2.0, 3.0]))
mean_val = np.mean(arr)
println("NumPy mean: $(pyconvert(Float64, mean_val))")
```

Conversion Mechanics and Bootstrapping

- Data conversion between Julia DataFrames and pandas uses explicit dictionary/list wrapping (`pydict`, `pylist`) to avoid iterator ambiguities.
- Environment Discovery: `PythonCall` automatically detects the active virtual environment managed by `uv` via standard environment variables (`VIRTUAL_ENV` or `.venv` detection), avoiding duplicate installations.

---

3. Python to Julia (`juliacall`)

To call Julia from Python, use the `juliacall` library (part of the PythonCall ecosystem) [*ag-juliacall*].

Primitive Types, DataFrames, Statistics, and `Dve`

inst/python/demo_bridge.py

```python
from juliacall import Main as jl

# Import Julia standard modules and local package
jl.seval("using Statistics")
jl.seval("using Dve")

# Primitive types
res = jl.sqrt(25.0)
print(res)

# Call project package function
greeting = jl.Dve.hello_world("Bonjour", "Python")
print(greeting)

# Complex types: pandas DataFrame to Julia DataFrame and back
import pandas as pd
import numpy as np

pdf = pd.DataFrame({"col1": [1, 2, 3], "col2": [4.0, 5.0, 6.0]})

# Convert pandas DataFrame to Julia DataFrame
jl.seval("using DataFrames")
jl.pdf = pdf
julia_df = jl.seval("DataFrame(pdf)")
print(julia_df)

# Compute statistic in Julia
mean_col1 = jl.mean(julia_df.col1)
print(f"Julia mean of col1: {mean_col1}")
```

Conversion Mechanics and Bootstrapping

- `juliacall` bridges Python objects into Julia via dynamic proxy wrappers. Zero-copy is used for NumPy arrays sharing underlying memory buffers with Julia arrays.
- Environment Discovery: `juliacall` activates the local Julia environment specified in `Project.toml` when initialized within the repository root.

Acceptance Tests Checklist

- ☐ **Julia Package Loading**: Run `julia --project -e 'using Dve; println(hello_world("Hi", "Test"))'` and expect `Hi, Test!`.
- ☐ **Unit Tests**: Run `julia --project -e 'using Test; include("tests/test_dve.jl")'` and expect all tests to pass.
- ☐ **Jupyter Kernel**: Open Jupyter Lab, select kernel `Julia (Dve)`, and run `using Dve`.
- ☐ **R to Julia Integration**: Run `Rscript -e 'library(RCall); print(juliaEval("2 + 2"))'` and expect `4`.
- ☐ **Python to Julia Integration**: Run `uv run python inst/python/demo_bridge.py` and expect successful execution of Julia functions.

Common Pitfalls

- **Mixing PyCall and PythonCall**: Do not mix `PyCall` and `PythonCall` in the same Julia session, as they initialize different Python runtimes. Stick exclusively to `PythonCall`.
- **Environment Mismatch**: Failing to run Julia with `--project` or `Pkg.activate()` causes package imports (`Dve`) to fail because Julia defaults to the global user environment.
- **Dataframe Iteration Mismatch**: Passing raw Julia dictionaries directly to pandas without `pydict()` results in pandas iterating over keys instead of values.

List of Unverified Items

- [unverified] `JuliaWorkspaces` role and syntax configuration details: Check official LanguageServer.jl and VS Code extension documentation [*ag-lsp*].
- [unverified] Precise target syntax for `[targets]` in Julia 1.13.1 Project.toml files: Check official Pkg documentation [*ag-pkg*].

Additional Notes

- This architecture ensures zero code duplication across R, Python, and Julia layers while respecting the constraints of `renv` and `uv`.
- For optimal performance in research pipelines, make sure type stability is maintained in Julia core functions before calling them across language boundaries.

References

- [ag-lsp] LanguageServer.jl Documentation: https://juliaserver.github.io/LanguageServer.jl/
- [ag-pkg] Julia Pkg Documentation: https://pkgdocs.julialang.org/
- [ag-workspaces] Julia Pkg Workspaces: https://pkgdocs.julialang.org/v1/workspaces/
- [ag-rcall] RCall.jl Documentation: https://juliainterop.github.io/RCall.jl/
- [ag-pythoncall] PythonCall.jl Documentation: https://cjdoris.github.io/PythonCall.jl/
- [ag-juliacall] juliacall Documentation: https://cjdoris.github.io/PythonCall.jl/stable/juliacall/

A:1 (ChatGPT)

^ ***Julia integration tutorial for a hybrid research repository***

- see: ChatGPT

Target runtime: Julia 1.13.1 · Python: uv · R: renv · Package: Dve

1. Julia project skeleton

1.1 Proposed repository tree

Julia 1.13.1 is a real stable release, published in September 2026. The examples below target that version and do not require compatibility with older Julia releases.

The key architectural decision is to make the repository root both the existing R package root and the Julia package root. This is workable, provided that the R build exclusions are tested explicitly.

File path: repository root

```
.
├── Project.toml                # Julia package and environment
├── Manifest.toml               # Julia dependency resolution
├── pyproject.toml              # Python project, managed by uv
├── uv.lock
├── DESCRIPTION                 # R package metadata
├── renv.lock
├── .Rbuildignore
├── .gitignore
├── R/
│   └── ...
├── inst/
│   ├── include/
│   └── python/
│       └── dve/
│           ├── __init__.py
│           └── ...
├── src/
│   ├── Dve.jl                  # Julia package entry point
│   ├── greetings.jl            # Julia implementation
│   ├── Makevars
│   └── ...                     # Existing Rcpp C++ sources
```

```
├── test/
│   ├── Project.toml            # Julia test environment
│   └── runtests.jl
├── scripts/
│   └── hello.jl                # Standalone Julia CLI
├── tests/
│   ├── pytest/
│   ├── testthat/
│   └── ...                     # Existing test suites
└── docs/
    └── ...
```

Three important details:

- Julia's conventional package test directory is `test/`, singular. The existing Python and R tests can remain where they are.
- Julia's conventional package entry point is `src/Dve.jl`. This coexists with the R package's native code in the same directory, but the `.jl` files must not leak into the built R source package.
- Julia's `Project.toml` and `Manifest.toml` have the same filenames used by other ecosystems. They are not R package metadata files. Explicitly exclude them from the R source package if necessary, and validate the resulting tarball with `R CMD build` and `R CMD check`.

There is one architectural constraint: the R package's `src/` directory is not an independent Julia source directory. It is simply the directory Julia uses for its package source files because the repository root is the Julia project root.

1.2 Minimal Julia package

Use a stable UUID for `Dve` and retain it permanently. The UUID below is an example; once selected for the repository, it must be used consistently in every environment that references this package.

File path: `Project.toml`

```
name = "Dve"
uuid = "bf600719-03fc-4660-bed4-7402f139157a"
authors = ["DEMS/datalab"]
version = "0.1.0"

[compat]
julia = "1.13"
```

This is a valid minimal package project without external runtime dependencies.

File path: `src/Dve.jl`

```
module Dve

include("greetings.jl")
```

```
export hello_world

end
```

File path: `src/greetings.jl`

```
"""
    hello_world(salutation, who)

Return a formatted greeting.
"""
function hello_world(salutation, who)
    return "$(salutation), $(who)!"
end
```

Activate the environment, instantiate its dependencies, and load the package.

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.instantiate()

using Dve

println(hello_world("Hello", "Julia"))
```

Expected output:

```
Hello, Julia!
```

`using Dve` resolves the package through the active environment and the package's UUID. You do not need `include("src/Dve.jl")` in normal package use.

For development, `Pkg.instantiate()` installs the versions recorded in the manifest. It does not add missing direct dependencies to `Project.toml`.

1.3 CLI script with a shebang and `--help`

A Julia script outside `src/` is an application entry point, not a package source file. The script must activate the repository environment explicitly; otherwise, Julia may use the default environment or a project inferred from the working directory.

File path: `scripts/hello.jl`

```
#!/usr/bin/env julia
```

```
using Pkg

const ROOT = normpath(joinpath(@__DIR__, ".."))
Pkg.activate(ROOT)

using Dve

function main(args)
    if "--help" in args || "-h" in args
        println("""
        Usage: hello.jl [--salutation TEXT] [--who NAME]

        Options:
          --salutation TEXT  Greeting prefix (default: Hello)
          --who NAME         Recipient (default: Julia)
          -h, --help         Show this help
        """)
        return 0
    end

    salutation = "Hello"
    who = "Julia"

    i = 1
    while i ⩽ length(args)
        if args[i] == "--salutation" && i < length(args)
            salutation = args[i + 1]
            i += 2
        elseif args[i] == "--who" && i < length(args)
            who = args[i + 1]
            i += 2
        else
            println(stderr, "Unknown or incomplete argument: $(args[i])")
            return 2
        end
    end

    println(hello_world(salutation, who))
    return 0
end

exit(main(ARGS))
```

The parser is deliberately small and has no third-party dependency. For a larger CLI, consider `ArgParse.jl`, which provides conventional argument definitions and generated help.

Working directory: repository root.

File path: shell commands

```
chmod +x scripts/hello.jl

julia --project=. scripts/hello.jl --who Researcher
julia --project=. scripts/hello.jl --salutation Welcome --who Julia
julia --project=. scripts/hello.jl --help
```

Expected output for the first two commands:

```
Hello, Researcher!
Welcome, Julia!
```

The help command prints the usage documentation.

The shebang `#!/usr/bin/env julia` locates Julia through `PATH`. It does not activate the project. `@__DIR__` anchors the project path to the script's own location, so the script also works when invoked from another working directory:

```
/path/to/repository/scripts/hello.jl --who Researcher
```

On Unix-like systems, this requires an executable file and a Julia executable on `PATH`. On Windows, invoke the script using `julia`.

1.4 Unit test

Julia's `Test` standard library provides assertions and test sets.

File path: `test/runtests.jl`

```
using Test
using Dve

@testset "Dve greetings" begin
    @test hello_world("Hello", "Julia") == "Hello, Julia!"
    @test hello_world("Welcome", "Researcher") ==
          "Welcome, Researcher!"
end
```

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.test()
```

Expected output: a passing `Dve greetings` test set with two successful assertions.

For a reproducible research repository, the test environment is best defined explicitly. The next section explains the recommended Julia 1.13 approach using a workspace and `test/Project.toml`.

1.5 Jupyter kernel support

The standard Julia Jupyter kernel is provided by `IJulia.jl`. The important distinction is between installing a kernel and activating the environment in which notebook code executes.

Install IJulia into the project environment.

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.add("IJulia")
using IJulia
IJulia.installkernel("Julia — Dve", "--project=@.")
```

The kernel specification passes `--project=@.` to Julia. This uses the project in the notebook's working directory. Consequently, start Jupyter from the repository root for this configuration.

Working directory: repository root, with the Python Jupyter environment activated by `uv`.

File path: shell commands

```
uv sync --group jupyter
uv run jupyter lab
```

Expected result: JupyterLab starts, and the `Julia — Dve` kernel is available after its installation.

If the kernel must work regardless of the notebook server's working directory, register it with the repository's absolute path instead. The drawback is that moving the checkout requires updating the kernel specification.

Also distinguish the two environments:

- `uv` manages the Python interpreter and JupyterLab installation.
- Julia's project and manifest manage Julia packages loaded by the kernel.

The Jupyter server and kernel are separate processes. They do not automatically share package environments merely because they are started from the same directory.

Documentation:

- IJulia.
- Pkg.

1.6 LSP support: VS Code and Emacs

The recommended starting point is the official Julia extension for VS Code, and an LSP client in Emacs using `LanguageServer.jl`.

The current architecture matters because several similarly named packages are not interchangeable.

Editor / LSP client

VS Code · Emacs · Neovim

- `LanguageServer.jl`: Implements the Language Server Protocol
- `JuliaWorkspaces.jl`: Code analysis, project indexing, diagnostics and completion

`JuliaWorkspaces.jl` is the analysis engine underlying `LanguageServer.jl`. `SymbolServer.jl` historically supplied symbol information to the language server, but its functionality has moved into `JuliaWorkspaces.jl`; its repository is no longer maintained. Do not install `SymbolServer` as a new, independent solution.

For this project:

- VS Code: install the official Julia extension and open the repository root. The extension manages the editor integration and uses the Julia project environment for project-aware tooling.
- Emacs: use either `lsp-mode` with the Julia LSP client or `eglot` with `eglot-jl`. These are alternative editor clients, not alternative Julia analysis engines.
- Julia environment: the language server needs access to the project and its dependencies. It should not be confused with the environment used by the Python LSP server or R language tools.

Emacs alternative A: `lsp-mode`

The Julia integration is commonly distributed as `lsp-julia`, alongside `lsp-mode`. This option fits an Emacs configuration already using `lsp-mode` for Python, R, C++, and other languages.

File path: Emacs configuration, for example `~/.emacs.d/init.el`

```
(use-package lsp-mode
  :commands (lsp lsp-deferred)
  :init
  (setq lsp-completion-provider :capf))

(use-package julia-mode
  :mode "\\.jl\\'"
  :hook (julia-mode . lsp-deferred))

(use-package lsp-julia
  :after lsp-mode)
```

The Julia executable must be available through `PATH`. Open a `.jl` file within the repository and check the LSP status. If your Emacs distribution obtains `lsp-julia` from a separate package archive, ensure

that the archive provides the package before relying on the snippet.

Emacs alternative B: `eglot`

`eglot` is built into modern Emacs releases. The `eglot-jl` package provides Julia-specific integration for Eglot.

File path: Emacs configuration, for example `~/.emacs.d/init.el`

```
(use-package julia-mode
  :mode "\\.jl\\'")

(use-package eglot
  :ensure nil
  :hook (julia-mode . eglot-ensure))
```

This is the minimal Eglot configuration. It assumes the Julia LSP server can be started using the executable or server command configured by the installed Julia integration. If `eglot-jl` is needed to configure that command in your package version, install and configure it according to its own documentation rather than assuming the two packages have identical setup requirements.

Which Emacs client should you choose?

Criterion	`lsp-mode` + `lsp-julia`	`eglot` + `eglot-jl`
Purpose	Full-featured LSP client	Lightweight, Emacs-integrat
Maintenance	Established ecosystem; verify current Julia adapter compatibility	Eglot is built in; verify the ex
Setup effort	More packages and configuration	Usually less configuration
Prefer it when	Your existing workflow depends on `lsp-mode` features	You prefer Emacs-native LSP

The exact current package requirements and server startup conventions for `eglot-jl` are not fully established by the official Julia LSP sources consulted here. Treat that detail as `[unverified]` and check the package's own documentation before copying a production configuration.

Documentation:

- Julia VS Code extension
- LanguageServer.jl
- JuliaWorkspaces.jl.

2. Julia project structure

2.1 Dependencies, UUIDs, and reproducibility

Julia package resolution is based on package names and UUIDs, not names alone. The UUID distinguishes packages that might share a name.

Julia concept	Python equivalent	R equivalent
`Project.toml`	`pyproject.toml`	`DESCRIPTION` plus dependency declarations
`Manifest.toml`	`uv.lock`	`renv.lock`
`[deps]`	Project dependencies	`Imports` / `LinkingTo`
`[compat]`	Version constraints	`Depends` / version constraints
`Pkg.add("DataFrames")`	`uv add pandas`	`install.packages("...")`
`Pkg.develop(path="...")`	Local editable install	Local package development with `renv` and development too

The analogy is useful but not exact: Julia's manifest is generated by Julia's package manager and records a resolved dependency graph, including package UUIDs, versions, and sources.

`Pkg.add()` is sufficient to add a dependency to the currently active project, but it is not a complete reproducibility policy. Use the project to record direct dependencies and the manifest to lock the resolved graph.

File path: `Project.toml`

```
name = "Dve"
uuid = "bf600719-03fc-4660-bed4-7402f139157a"
authors = ["DEMS/datalab"]
version = "0.1.0"

[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"

[compat]
DataFrames = "1"
julia = "1.13"
```

The `DataFrames` UUID shown here is its registered UUID. For other packages, obtain UUIDs from their official `Project.toml` or the Julia package registry rather than guessing.

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.add("DataFrames")
Pkg.status()
```

Expected output: the active project and its direct dependencies are displayed, including `DataFrames`.

For a clean checkout:

```
using Pkg
```

```
Pkg.activate(".")
Pkg.instantiate()
Pkg.status()
```

`Pkg.instantiate()` installs the dependencies recorded in the manifest, or resolves them if no manifest exists.

Version constraints

The compatibility declaration:

```
[compat]
DataFrames = "1"
julia = "1.13"
```

means that the package accepts compatible DataFrames 1.x releases and Julia 1.13 releases under Julia's compatibility rules. It does not mean that all users install the same version. That is the manifest's role.

For a research project, commit `Manifest.toml` when reproducible project execution is important. Review manifest changes alongside dependency updates.

Registries

Julia's default General registry contains metadata for registered packages. Packages can also be installed from Git repositories, local paths, or other registries.

Use a registry when a package is intended to be published and versioned for reuse. Use `Pkg.develop` when working on a local package that is not yet released or when you need changes in the checkout to be immediately visible.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.develop(path="../DveUtilities")
```

Expected result: the local package is added to the active environment in development mode, with its path recorded in the manifest.

The path must point to an actual Julia package containing its own `Project.toml`.

Documentation:

- Pkg project files

2.2 Runtime, test, development, and LSP dependencies

Julia 1.13 supports Pkg workspaces, making a separate test environment a good default for this repository.

There are three different concerns:

- Runtime dependencies: packages imported by `Dve` itself belong in the root `[deps]`.
- Test dependencies: packages needed only to run tests belong in `test/Project.toml`.
- Editor tooling: `LanguageServer` and related tooling should generally live in a separate tooling environment, rather than being runtime dependencies of `Dve`.

Do not add editor-only packages to `[deps]` simply because the editor needs them.

Recommended approach: Pkg workspace

File path: `Project.toml`

```
name = "Dve"
uuid = "bf600719-03fc-4660-bed4-7402f139157a"
authors = ["DEMS/datalab"]
version = "0.1.0"

[compat]
julia = "1.13"

[workspace]
projects = ["test"]
```

File path: `test/Project.toml`

```
[deps]
Dve = "bf600719-03fc-4660-bed4-7402f139157a"
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"

[sources]
Dve = {path = ".."}
```

This declares `Dve` and `Test` in the test project. The workspace allows the package manager to resolve the root and test projects together and maintain a shared root manifest. Dependencies of the root project are not automatically available as direct dependencies of the test project; declare every package used directly by tests.

This is the recommended modern approach. The `[extras]` and `[targets]` mechanism remains available for older projects, but Julia's current Pkg documentation describes it as the legacy approach for test-specific dependencies.

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.instantiate()
```

```
Pkg.test()
```

Expected result: Pkg resolves the workspace, installs the test environment's dependencies, and executes the package tests.

Separate test environments versus legacy targets

Criterion	Workspace with `test/Project.toml`	`[extras]` / `[targets]`
Purpose	Isolated test dependencies with workspace resolution	Test-only dependencies declared in
Maintenance status	Recommended for Julia 1.13	Legacy mechanism, retained for co
Setup effort	More explicit files and setup	Simpler for small existing packages
Prefer it when	Starting a new project or using a monorepo	Maintaining a package that already

A standalone `test/Project.toml` can also be used outside a workspace. That produces a separate environment and manifest, however, rather than the workspace's shared dependency resolution. Use it when independent test dependency resolution is intentional.

LSP and development tooling

A separate tooling environment is useful when you want reproducible editor dependencies without coupling them to the research package.

File path: `tools/Project.toml`

```
[deps]
LanguageServer = "bcc55880-"
```

The UUID above is intentionally incomplete and must not be used as a runnable configuration. Obtain the complete UUID from the official package metadata and generate the tooling environment with Pkg instead of hand-writing incomplete dependency records.

For a runnable setup, let Pkg generate the project:

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate("tools")
Pkg.add("LanguageServer")
Pkg.instantiate()
```

This creates or updates `tools/Project.toml` and `tools/Manifest.toml`. The tooling environment is independent unless you explicitly include it in the root workspace.

For most users, the simplest setup is to use the official VS Code extension and avoid maintaining a separate tooling environment until the team needs to pin LSP versions.

2.3 Source, test, and script directories

Julia's package layout is conventional, with a few package-manager conventions that are recognized automatically.

Path	Meaning	Automatically selected?
`src/Dve.jl`	Package entry point	Yes, for a package named `Dve`
`test/runtests.jl`	Conventional package test entry point	Yes, by `Pkg.test()`
`scripts/hello.jl`	Standalone executable script	No
`docs/`	Documentation sources	No, not inherently
`Project.toml`	Package identity and direct dependencies	Yes
`Manifest.toml`	Resolved dependency versions	Yes, if present

`Project.toml` does not define an arbitrary source-directory key or script-directory key. Julia's package loader expects the conventional package source layout. Scripts can live anywhere; their paths are chosen by the repository author.

The existing `tests/pytest` and `tests/testthat` directories are independent Python and R conventions. Julia does not automatically discover tests inside `tests/*.jl` just because they have a `.jl` extension.

Keep the Julia tests under `test/`, and keep cross-language integration tests in the appropriate language's test framework.

2.4 What does “workspace” mean?

The word workspace has three different meanings in this setup.

- Pkg workspace: a package-manager feature that resolves several Julia projects together.
- JuliaWorkspaces.jl: the static-analysis engine used by Julia's language server.
- Editor workspace: a folder or collection of folders opened in VS Code or another editor.

These are related only in the broad sense of working with a group of files or projects. They are not interchangeable.

For the hybrid repository, a Pkg workspace is useful when the root `Dve` package and its test project need compatible dependencies but should declare different direct dependencies.

For example, the root package may depend on `DataFrames`, while the test environment adds `Test` and additional test libraries. The workspace lets Pkg resolve those projects together.

The editor workspace, by contrast, should simply open the repository root so that Julia, Python, and R tooling can find their respective project metadata.

Documentation:

- Pkg workspaces and test environments
- JuliaWorkspaces.jl

3. Language integration examples

3.1 Integration architecture

Use one primary bridge for each direction. Avoid installing multiple competing bridges unless a specific feature requires them.

1. R
    - `JuliaCall` (R package): Embeds Julia in R
2. Julia
    - `PythonCall.jl` (Julia Package): Calls Python from Julia
3. Python
    - `juliacall` (Python Package): Calls Julia from Python; built on `PythonCall.jl`

The bridge packages are different:

- `JuliaCall` (R package) embeds Julia in R. It is not `RCall.jl`.
- `PythonCall.jl` calls Python from Julia and supports Python-object conversion.
- `juliacall` is the Python package that calls Julia, using the PythonCall implementation.

Sources:

- JuliaCall for R
- PythonCall.jl

3.2 Environment ownership and the no-duplicate-installation rule

For a hybrid project, the preferred model is to keep each language's package manager authoritative for its own dependencies.

Environment	Owner	Responsibilities
Python	`uv`	Python interpreter, `dvepy`, pandas, NumPy, JupyterLab
R	`renv`	R packages, including `JuliaCall`
Julia	`Pkg`	`Dve`, DataFrames, PythonCall, IJulia
Python-to-Julia bridge	`juliacall`	Connects the existing Python process to Julia
R-to-Julia bridge	`JuliaCall`	Connects the existing R process to Julia

Do not assume that activating `renv` or a `uv` virtual environment automatically configures the other runtime.

For Julia calling Python, explicitly configure PythonCall to use the existing `.venv` Python interpreter. This prevents it from creating its own managed Python environment.

For Python calling Julia, configure `juliacall` to use the repository's Julia executable and Julia project. This avoids an independent Julia installation or unrelated global Julia environment.

For R calling Julia, use a Julia installation shared with the repository and configure JuliaCall to use that installation. R's `renv` does not manage Julia dependencies; Julia's Pkg does.

These are separate runtime environments, but they need not be separate installations of Python or Julia.

3.3 R to Julia

Integration choice

Use the R package `JuliaCall`, rather than `RCall.jl`.

`JuliaCall` is designed to embed Julia in R and provides initialization, evaluation, calling, and conversion facilities. `RCall.jl` solves the opposite problem: calling R from Julia.

The first-run setup requires a working Julia installation and an R package installation. JuliaCall can install Julia automatically, but that is not the best choice for this repository because you want a controlled Julia version shared across the project.

Working directory: repository root, in an R session with the project's `renv` activated.

File path: R console commands

```
renv::install("JuliaCall")
renv::snapshot()
```

Expected result: `JuliaCall` is recorded in the R environment's lockfile.

Initialize the bridge and use the repository Julia project.

File path: R console commands

```
library(JuliaCall)

julia_setup()

julia_eval("using Pkg")
julia_eval("Pkg.activate(\".\")")
julia_eval("using Dve")

julia_eval('Dve.hello_world("Hello", "Julia")')
```

Run this from the repository root, with Julia available to JuliaCall. The exact way JuliaCall locates an existing Julia installation depends on its installed version and setup; check its official documentation if it selects a different Julia version. Do not assume `renv` configures the Julia executable.

Expected result: the Julia expression returns the string `Hello, Julia!`.

Primitive arguments and return values (R to Julia)

The following example uses `julia_call` to call the Julia function with R scalar values.

File path: R console commands

```
library(JuliaCall)

julia_setup()
julia_eval("using Pkg")
julia_eval("Pkg.activate(\".\")")
julia_eval("using Dve")

greeting <- julia_call(
  "Dve.hello_world",
  "Welcome",
  "Researcher"
)

stopifnot(identical(greeting, "Welcome, Researcher!"))
print(greeting)
```

Expected output:

```
[1] "Welcome, Researcher!"
```

The bridge converts R character scalars to Julia strings and converts the Julia string result back to an R character scalar.

The example assumes the installed JuliaCall version exposes the documented `julia_call` interface and automatic string conversion. Verify against the JuliaCall documentation if your installed release differs.

Complex types: R `data.frame` to Julia `DataFrame`

The safe conceptual conversion is to construct a Julia `DataFrame` from the R columns. Do not assume an arbitrary R `data.frame` becomes a Julia `DataFrame` automatically in every JuliaCall version.

First, ensure that `DataFrames` is a direct dependency of the repository's Julia project.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.add("DataFrames")
```

Then pass individual columns through JuliaCall and construct the table on the Julia side.

File path: R console commands

```
library(JuliaCall)

julia_setup()
```

```
julia_eval("using Pkg")
julia_eval("Pkg.activate(\".\")")
julia_eval("using DataFrames")

df_r <- data.frame(
  x = c(1.0, 2.0, 3.0),
  y = c(10.0, 20.0, 30.0)
)

julia_assign("x_r", df_r$x)
julia_assign("y_r", df_r$y)

julia_eval("df_jl = DataFrame(x = x_r, y = y_r)")

julia_eval("df_jl")
```

Expected result: a Julia `DataFrame` with columns `x` and `y` and three rows.

The use of `julia_assign` for ordinary vectors is documented by JuliaCall; the precise conversion of each R vector type should be tested with the installed version.

To return a Julia table, extract its columns into R-compatible vectors and construct an R `data.frame`. This is more explicit than assuming that JuliaCall has a universal automatic `DataFrames.DataFrame` converter.

File path: R console commands

```
julia_assign("x_out", julia_eval("df_jl.x"))
julia_assign("y_out", julia_eval("df_jl.y"))

df_out <- data.frame(
  x = x_out,
  y = y_out
)

stopifnot(identical(df_out, df_r))
print(df_out)
```

Expected output:

```
  x  y
1 1 10
2 2 20
3 3 30
```

This example deliberately uses a numeric-only table to keep the conversion simple. Factors, categorical columns, missing values, dates, and timezone-aware timestamps need explicit policies.

Copy versus zero-copy

For this column-wise bridge, treat the conversion as a copy across runtimes unless you have verified a specific zero-copy conversion path for the exact types and bridge version.

The R and Julia runtimes have separate object models and garbage collectors. Passing vectors through an embedding API does not establish shared memory or zero-copy semantics.

For larger data, transfer only required columns and avoid repeatedly converting the same table. If performance becomes important, benchmark the full call including conversion, not just the Julia function.

3.4 Julia to Python

Integration choice: `PythonCall.jl`

Use `PythonCall.jl` rather than `PyCall.jl` for new work.

Both support calling Python and importing third-party modules. PythonCall provides an extensible conversion model and is the natural companion to `juliacall`. PyCall remains a viable alternative for established projects, but its interpreter configuration and build-time linkage make it less convenient for a project that needs to select a `uv`-managed interpreter.

Bootstrap PythonCall against the `uv` environment

First, ensure that the Python project has its dependencies declared.

File path: `pyproject.toml`

```toml
[project]
name = "dvepy"
version = "0.1.0"
requires-python = "⩾3.11"
dependencies = [
    "numpy",
    "pandas",
]

[tool.uv]
package = true
```

This is a minimal illustration of the Python package metadata, not a replacement for the repository's existing configuration. Preserve any existing build backend, dependency groups, and package-discovery settings.

Install or synchronize the Python environment.

Working directory: repository root.

File path: shell commands

```
uv sync
uv run python -c "import numpy, pandas; print(numpy.__version__, pandas.__version__)"
```

Expected result: both library versions are printed.

Install Julia dependencies.

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.add("PythonCall")
Pkg.add("DataFrames")
Pkg.instantiate()
```

Configure PythonCall to use the existing `.venv` interpreter, with CondaPkg disabled as a Python installer. Set these variables before PythonCall is first loaded in the Julia process.

Working directory: repository root.

File path: shell commands

```
export JULIA_PYTHONCALL_EXE="@.venv"
export JULIA_CONDAPKG_BACKEND="Null"

julia --project=. -e 'using PythonCall; println(pyimport("sys").executable)'
```

Expected result: the printed Python executable points to the repository's `uv` environment.

The `@.venv` selector and `Null` backend are PythonCall/CondaPkg configuration features. If the installed package version does not recognize them, consult PythonCall configuration rather than silently falling back to a second Python installation.

Primitive arguments and return values (Julia to Python)

File path: `scripts/julia_to_python.jl`

```
using PythonCall

math = pyimport("math")

x = 9.0
result = math.sqrt(x)

println(pyconvert(Float64, result))
```

Working directory: repository root.

File path: shell command

```
JULIA_PYTHONCALL_EXE="@.venv" \
JULIA_CONDAPKG_BACKEND="Null" \
julia --project=. scripts/julia_to_python.jl
```

Expected output:

```
3.0
```

The Julia `Float64` is passed to Python as a numeric value. PythonCall returns a Python object, and `pyconvert(Float64, result)` explicitly converts the result back to a Julia scalar.

This distinction is useful: PythonCall often leaves Python values as Python objects until you request conversion. That makes it possible to continue using Python APIs without prematurely converting every result.

Calling NumPy

File path: `scripts/julia_numpy.jl`

```
using PythonCall

np = pyimport("numpy")

values = np.array([1.0, 2.0, 3.0])
result = np.mean(values)

println(pyconvert(Float64, result))
```

Working directory: repository root.

File path: shell command

```
JULIA_PYTHONCALL_EXE="@.venv" \
JULIA_CONDAPKG_BACKEND="Null" \
julia --project=. scripts/julia_numpy.jl
```

Expected output:

```
2.0
```

NumPy is imported from the interpreter configured for PythonCall. It should therefore be the NumPy installed by `uv` in `.venv`.

Calling the project package `dvepy`

Your repository uses a non-standard Python source location under `inst/python`. Python needs to be told where that package lives. The Python package's import name and the distribution name are distinct: the distribution is `dvepy`, while the import shown here is `dve`.

File path: `inst/python/dve/__init__.py`

```
def hello_world(salutation: str, who: str) → str:    return f"{salutation}, {who}!"
```

Expose the source directory to the embedded Python process.

Working directory: repository root.

File path: shell commands

```
PYTHONPATH="$PWD/inst/python${PYTHONPATH:+:$PYTHONPATH}" \
JULIA_PYTHONCALL_EXE="@.venv" \
JULIA_CONDAPKG_BACKEND="Null" \
julia --project=. -e '
using PythonCall
dve = pyimport("dve")
println(pyconvert(String, dve.hello_world("Hello", "Python")))
'
```

Expected output:

```
Hello, Python!
```

For the final repository, prefer configuring Python package discovery correctly in `pyproject.toml`, or installing the project into `.venv` through `uv`, rather than requiring developers to export `PYTHONPATH` manually. If you retain `inst/python` as the source root, verify that your build backend supports that package layout.

Complex types: Julia `DataFrame` to pandas and back

PythonCall supports converting tables through `pytable` and `PyTable`. A Julia table can be exported as a pandas `DataFrame`; a pandas table can be wrapped as a PythonCall table and converted into a Julia `DataFrame`

File path: `scripts/julia_pandas.jl`

```
using DataFrames
using PythonCall

pd = pyimport("pandas")

df_jl = DataFrame(
    x = [1.0, 2.0, 3.0],
    y = [10.0, 20.0, 30.0],
)

df_py = pytable(df_jl; format = :pandas)

println(df_py)
```

```
df_back = DataFrame(PyTable(df_py))

@assert df_back == df_jl
println(df_back)
```

Working directory: repository root.

File path: shell command

```
JULIA_PYTHONCALL_EXE="@.venv" \
JULIA_CONDAPKG_BACKEND="Null" \
julia --project=. scripts/julia_pandas.jl
```

Expected result: the first and second tables have the same three rows and columns.

The conversion is not guaranteed to be zero-copy. It depends on the Julia table type, the pandas representation, column types, and the conversion path. Treat the example as a correctness-oriented conversion and benchmark any claimed zero-copy optimization separately.

3.5 Python to Julia

Integration choice: `juliacall`

Use the Python package `juliacall`, built on PythonCall.jl, to invoke Julia from Python.

This avoids manually embedding Julia through the C API and provides access to Julia objects through a Python interface. By default, `juliacall` can download Julia and create a Julia package environment. For this repository, configure it to use the shared Julia installation and project instead.

Bootstrap the bridge

Add the Python bridge to the existing `uv` environment.

Working directory: repository root.

File path: shell commands

```
uv add juliacall
```

Ensure that the Julia project includes the dependencies needed by Python-called code.

Working directory: repository root.

File path: Julia REPL commands

```
using Pkg

Pkg.activate(".")
Pkg.add("PythonCall")
Pkg.add("DataFrames")
Pkg.instantiate()
```

`PythonCall` must be installed in the Julia environment used by `juliacall`. This is important because the Python package and Julia package cooperate to establish the bridge.

Configure `juliacall` to use the existing Julia installation and root project.

Working directory: repository root.

File path: shell commands

```
export PYTHON_JULIACALL_PROJECT="$PWD"
export PYTHON_JULIACALL_EXE="$(command -v julia)"

uv run python -c 'from juliacall import Main as jl; print(jl.seval("VERSION"))'
```

Expected result: the Julia version is printed.

Both configuration variables must be set before importing `juliacall`. `PYTHON_JULIACALL_PROJECT` points to the Julia project, while `PYTHON_JULIACALL_EXE` selects the Julia executable. The selected project must contain a compatible installation of `PythonCall`

This setup should not require a second Julia download. If `juliacall` reports that it is provisioning another Julia environment, check the environment variables and whether the root Julia project has been instantiated.

Primitive arguments and return values (Python to Julia)

File path: `scripts/python_to_julia.py`

```
from juliacall import Main as jl

jl.seval("using Dve")

greeting = jl.Dve.hello_world("Welcome", "Julia")

print(greeting)
```

Working directory: repository root.

File path: shell command

```
PYTHON_JULIACALL_PROJECT="$PWD" \
PYTHON_JULIACALL_EXE="$(command -v julia)" \
uv run python scripts/python_to_julia.py
```

Expected output:

```
Welcome, Julia!
```

Python strings are converted to Julia strings at the bridge boundary. The Julia string result is converted to a Python string by the bridge's return conversion.

For numeric calculations, explicitly verify the Python and Julia numeric types if exact integer width

or floating-point precision matters.

Calling `Statistics`

`Statistics` is a Julia standard library, so it does not need a separately installed third-party package. It must still be imported in the Julia session.

File path: `scripts/python_statistics.py`

```
from juliacall import Main as jl

jl.seval("using Statistics")

values = [1.0, 2.0, 3.0, 4.0]
result = jl.Statistics.mean(values)

print(result)
```

Working directory: repository root.

File path: shell command

```
PYTHON_JULIACALL_PROJECT="$PWD" \
PYTHON_JULIACALL_EXE="$(command -v julia)" \
uv run python scripts/python_statistics.py
```

Expected output:

```
2.5
```

This illustrates an important distinction: the standard library is supplied by the Julia installation, but the code must still import the module into the Julia session.

Calling the project package `Dve`

File path: `scripts/python_dve.py`

```
from juliacall import Main as jl

jl.seval("using Dve")
print(jl.Dve.hello_world("Hello", "Researcher"))
```

Working directory: repository root.

File path: shell command

```
PYTHON_JULIACALL_PROJECT="$PWD" \
PYTHON_JULIACALL_EXE="$(command -v julia)" \
uv run python scripts/python_dve.py
```

Expected output:

```
Hello, Researcher!
```

The package is loaded through the Julia project, not by importing `src/Dve.jl` as a Python module. This ensures that Julia's package loading, dependencies, and UUID resolution remain authoritative.

Complex types: pandas to Julia `DataFrame` and back

Use PythonCall's table conversion layer rather than assuming that pandas objects become native Julia tables automatically.

File path: `scripts/python_pandas.py`

```python
import pandas as pd
from juliacall import Main as jl

jl.seval("using DataFrames, PythonCall")

df_py = pd.DataFrame({
        "x": [1.0, 2.0, 3.0],
        "y": [10.0, 20.0, 30.0],
        })
print(df_py)

df_jl = jl.DataFrames.DataFrame(jl.PythonCall.PyTable(df_py))
print(df_jl)

df_back = jl.PythonCall.pytable(df_jl, format="pandas")
print(df_back)
```

Working directory: repository root.

File path: shell command

```
PYTHON_JULIACALL_PROJECT="$PWD" \
PYTHON_JULIACALL_EXE="$(command -v julia)" \
uv run python scripts/python_pandas.py
```

Expected result: the first table is represented as a Julia `DataFrame`, and the returned pandas table has the same column values.

The conversion uses PythonCall's `PyTable` wrapper to expose the pandas table to Julia and `pytable` to export a Julia table. For production code, verify the exact method dispatch and PythonCall version against its table conversion documentation.

This path should be treated as a conversion, not a promise of zero-copy semantics. Copying may occur while adapting column storage, handling missing values, or normalizing types.

4. Acceptance tests and operational validation

4.1 Acceptance checklist

Run these checks from a clean checkout after installing the specified Julia, Python, and R runtimes.

Check	Command or action
Julia version	`julia --version`
Julia dependencies	`julia --project=. -e 'using Pkg; Pkg.instantiate(); Pkg.status()'`
Package import	`julia --project=. -e 'using Dve; println(Dve.hello_world("Hello", "Julia"))'`
CLI defaults	`julia --project=. scripts/hello.jl`
CLI arguments	`julia --project=. scripts/hello.jl --who Researcher`
Julia tests	`julia --project=. -e 'using Pkg; Pkg.test()'`
Jupyter	Start JupyterLab and select `Julia — Dve`
Python dependencies	`uv sync --locked`
Python tests	`uv run pytest tests/pytest`
R tests	`R -q -e 'renv::restore(); devtools::test()'`
Julia from Python	Run `scripts/python_to_julia.py` with bridge variables set
Python from Julia	Run `scripts/julia_to_python.jl` with PythonCall configured
R to Julia	Run the JuliaCall primitive example
Table round-trip	Run each table integration example
R package build	`R CMD build .`

The Python and R test commands assume the existing repository's test configuration and dependencies are already declared. If the R tests require an installed package rather than the checkout being directly loadable, run them using the repository's normal development workflow.

4.2 R package compliance

Because this repository has both R and Julia package structures, test the actual R source tarball rather than relying solely

A:1 (DeepSeek)

^ *Julia integration tutorial*

- see: DeepSeek

Proposed Repository Layout and Julia Version

The Julia version targeted is **1.13.1** (stable release, September 25, 2026).

The repository follows the existing R project layout with Julia sources placed under `src/`:

```
<project-root>
  pyproject.toml
  DESCRIPTION
  uv.lock
  renv.lock
  Project.toml                  # Julia root project (Dve)
  Manifest.toml                 # Julia lockfile
  R/                            # R package "dvesimpler"
  inst/include                  # Rcpp C++ interface
  inst/python/dve/__init__.py   # Python "dvepy.dve" module
  src/Makevars                  # Rcpp build specification
  src/*.cpp                     # Rcpp C++ sources
  src/Dve.jl                    # Julia Dve module
  src/*.jl                      # Other Julia sources
  tests/pytest                  # Python test base
  tests/testthat                # R test base
  tests/*.jl                    # Julia test sources
```

The Julia package root is the repository root, with `Project.toml` and `Manifest.toml` at the top level. Julia sources live in `src/`, tests in `tests/`, and the executable script outside the package (e.g., `bin/hello.jl`). This requires adding `src/*.jl`, `tests/*.jl`, and `bin/` to `.Rbuildignore` to maintain CRAN compliance.

Objective 1: Julia Project Skeleton

Minimal `Dve` Module

File: `src/Dve.jl`

```
module Dve

export hello_world

"""
    hello_world(salutation, who) → String

Return a formatted greeting.
"""
function hello_world(salutation::AbstractString, who::AbstractString)
    return "$salutation, $who!"
end

end # module
```

Jupyter Kernel Support

Install `IJulia` and register a kernel bound to this project:

```
# From the project root
julia --project=. -e 'using Pkg; Pkg.add("IJulia")'
julia --project=. -e 'using IJulia; installkernel("Julia Dve", "--project=$(Base.active_project(
```

The `installkernel` call passes `--project` pointing to this project's `Project.toml`, so the notebook kernel activates the correct environment. The default IJulia kernel uses `--project=@.`, which searches parent directories for `Project.toml`; an explicit kernel avoids ambiguity.

Executable Script

File: `bin/hello.jl`

```
#!/usr/bin/env julia
# The shebang locates Julia via `env`, which resolves the first `julia` on PATH.
# To bind to the project environment, run: julia --project=@. bin/hello.jl

using ArgParse

function parse_commandline()
    s = ArgParseSettings(description = "Print a greeting using Dve.hello_world.")
    @add_arg_table! s begin
        "salutation"
            help = "Greeting word (e.g., Hello)"
            default = "Hello"
        "who"
            help = "Name of the person to greet"
            default = "World"
    end
```

```
    return parse_args(s)
end

function main()
    args = parse_commandline()
    # Activate the project if not already active
    if Base.active_project() == joinpath(DEPOT_PATH[1], "environments", "v$(VERSION.major).$(VER
        using Pkg; Pkg.activate(joinpath(@__DIR__, ".."))
    end
    using Dve
    println(hello_world(args["salutation"], args["who"]))
end

main()
```

Run from the project root:

```
julia --project=@. bin/hello.jl --help
julia --project=@. bin/hello.jl "Hi" "Julia"
```

Expected output:

```
usage: hello.jl [--salutation SALUTATION] [--who WHO] [-h]

Print a greeting using Dve.hello_world.

optional arguments:
  --salutation SALUTATION
                        Greeting word (e.g., Hello) (default: "Hello")
  --who WHO             Name of the person to greet (default: "World")
  -h, --help            show this help message and exit

Hi, Julia!
```

Unit Test

File: `test/runtests.jl`

```
using Test
using Dve

@testset "hello_world" begin
    @test hello_world("Hello", "World") == "Hello, World!"
    @test hello_world("Hi", "Julia") == "Hi, Julia!"
end
```

Run tests:

```
julia --project=. -e 'using Pkg; Pkg.test()'
```

LSP Editor Support

VS Code — Install the Julia extension (`julia-vscode.julia-vscode`), which bundles `LanguageServer.jl`.

Emacs — Two client options exist:

Client	Package	Status	Notes
`eglot`	`eglot-jl`	Active, maintained	Simpler, built-in since Emacs 29. Recommended.
`lsp-mode`	`lsp-julia`	Maintained but heavier	More features, more configuration.

Recommended `use-package` configuration for `eglot-jl`:

```
(use-package eglot-jl
  :ensure t
  :init
  (eglot-jl-init)
  :hook
  (julia-mode . eglot-ensure))
```

Relationship between `LanguageServer`, `JuliaWorkspaces`, and `SymbolServer`:

- `LanguageServer.jl` is the LSP server implementation that editors communicate with.
- `SymbolServer.jl` was a helper package providing symbol information without loading packages. **Its functionality has been moved to `JuliaWorkspaces.jl` and the standalone package is no longer maintained**.
- `JuliaWorkspaces.jl` is the analysis engine powering `LanguageServer.jl`, incorporating code from `StaticLint`, `CSTParser`, and `SymbolServer`.

They are **components**, not alternatives. `JuliaWorkspaces` is the modern analysis backend; `SymbolServer` is deprecated as a standalone package.

Objective 2: Julia Project Structure

Dependency Management and UUIDs

`Pkg.add` is sufficient for adding registered packages to an environment. It updates `Project.toml` (declaring the dependency with its UUID) and resolves `Manifest.toml` (recording exact versions and transitive dependencies).

- **`Project.toml`** — Human-edited; lists direct dependencies, `[compat]` bounds, and project metadata.
- **`Manifest.toml`** — Machine-generated; pins exact versions, hashes, and full dependency graph. Commit it for reproducible environments.
- **`[compat]`** — Semver bounds for dependencies. Example:

```
[compat]
DataFrames = "1.7"
julia = "1.13"
```

- **`Pkg.develop`** — Adds a local path dependency: `Pkg.develop(path="/path/to/LocalPkg")`. The dependency appears in `Project.toml` with a path source rather than a registry UUID.
- **Registries** — Default is the General registry. Custom registries can be added with `Pkg.Registry.add`.

Test and Dev Dependencies

Two patterns exist:

Pattern A — `[extras]` + `[targets]` (legacy):

```
[extras]
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"

[targets]
test = ["Test"]
```

Pattern B — `test/Project.toml` (recommended):

Create `test/Project.toml` with `[deps]` containing test-only packages. This isolates test dependencies and is the recommended approach in modern Julia packages. It avoids bloating the root `Project.toml` and supports independent resolution.

Source, Test, and Script Folders

- `src/` — **Convention**, not enforced by `Project.toml`. `using Dve` expects `src/Dve.jl`.
- `test/` — **Convention**. `Pkg.test()` runs `test/runtests.jl`.
- Scripts — **No convention**. Place them wherever appropriate (e.g., `bin/`).

Workspaces

Pkg workspaces (Julia 1.12+) allow multiple related projects to share a single `Manifest.toml`, solving monorepo dependency conflicts.

Minimal example:

Root `Project.toml`:

```
[workspace]
projects = ["SubPkgA", "SubPkgB"]
```

Sub-project `SubPkgA/Project.toml`:

```
name = "SubPkgA"
uuid = "..."
```

Run `Pkg.instantiate()` from the root to resolve all subprojects against a unified manifest.

Objective 3: Language Integration Examples

R to Julia

Package: `RCall.jl` — the standard, actively maintained bridge.

Primitive types:

```
using RCall

# R → Julia
rcopy(R"1 + 1")          # ⇒ 2
rcopy(R"c(1.0, 2.0, 3.0)") # ⇒ [1.0, 2.0, 3.0]

# Julia → R
robject(42)              # RObject{IntSxp} [1] 42
```

`data.frame` ↔ `DataFrame`:

```
using RCall, DataFrames

# R → Julia
r_df = R"data.frame(x = 1:3, y = c('a','b','c'))"
jl_df = rcopy(r_df)       # DataFrame with 3 rows, 2 columns

# Julia → R
df = DataFrame(x = 1:3, y = ["a", "b", "c"])
r_df2 = robject(df)       # R data.frame
```

Conversion semantics: `rcopy` creates a **deep copy** in Julia. `robject` wraps the Julia object in an R-compatible representation; for mutable types, changes in R may affect the original Julia object.

Interpreter discovery: `RCall` finds the R installation via `R_HOME` or PATH. It does not duplicate R installations.

Julia to Python

Package: `PythonCall.jl` — preferred over `PyCall` for its broader conversion support and non-copying semantics.

Primitive types:

```
using PythonCall

# Julia → Python
x = pyconvert(Py, 42)          # Py(42)
y = pyconvert(Py, [1.0, 2.0])  # Py([1.0, 2.0])

# Python → Julia
```

```
pyconvert(Int, py"1 + 1")      # 2
```

DataFrame □ pandas.DataFrame:

```
using PythonCall, DataFrames

df = DataFrame(a = 1:3, b = ["x", "y", "z"])
pydf = pytable(df)             # Python pandas DataFrame

# Back to Julia
df2 = DataFrame(PyTable(pydf))
```

Conversion semantics: `pytable` performs a **zero-copy** conversion for numeric columns where possible; string columns may be copied. `PyTable` wraps the Python object without copying.

Interpreter discovery: `PythonCall` uses `CondaPkg` to create an isolated Conda environment per Julia project, avoiding conflicts with `uv`-managed Python. For integration with an existing `uv` environment, set `ENV["JULIA_CONDAPKG_BACKEND"] = "Null"` and configure `PythonCall` to use the system Python.

Python to Julia

Package: `JuliaCall` (PyPI: `juliacall`) — the Python-side module of the PythonCall ecosystem.

Primitive types:

```
from juliacall import Main as jl

# Python → Julia
x = jl.eval("1 + 1")          # 2

# Julia → Python
jl.println("Hello from Julia!")
y = jl.rand(3, 5)             # Julia matrix exposed as Python object
```

pandas.DataFrame □ DataFrame:

```
from juliacall import Main as jl
import pandas as pd

jl.seval("using DataFrames")
pdf = pd.DataFrame({"a": [1, 2, 3], "b": ["x", "y", "z"]})
jdf = jl.DataFrame(pdf)        # Julia DataFrame

# Back to pandas
pdf2 = pd.DataFrame(jdf)
```

Conversion semantics: `JuliaCall` wraps Julia objects in `AnyValue` containers. Conversion to Python happens on demand. Numeric arrays are **zero-copy** where possible.

Interpreter discovery: `JuliaCall` manages its own Julia installation via `JuliaPkg` unless config-

ured otherwise. To use the project's Julia, set `JULIA_PROJECT` to the project root before importing `juliacall`.

Calls to Project Packages

- **R □ Julia `Dve`**: Use `RCall` to `@rput` arguments and `reval` to call `Dve.hello_world`, then `@rget` the result.
- **Julia □ Python `dvepy`**: `pyimport("dvepy.dve")` then call functions.
- **Python □ Julia `Dve`**: `jl.seval("using Dve")` then `jl.Dve.hello_world("Hi", "Julia")`.

Acceptance Tests Checklist

#	Command	Expected
1	`julia --project=. -e 'using Pkg; Pkg.instantiate()'`	No errors; `Manifest`
2	`julia --project=. -e 'using Dve; println(Dve.hello_world("Hi","Julia"))'`	`Hi, Julia!`
3	`julia --project=@. bin/hello.jl --help`	Usage message with
4	`julia --project=@. bin/hello.jl "Hello" "World"`	`Hello, World!`
5	`julia --project=. -e 'using Pkg; Pkg.test()'`	All tests pass
6	`julia --project=. -e 'using RCall; println(rcopy(R"1+1"))'`	`2`
7	`julia --project=. -e 'using PythonCall; println(pyconvert(Int, py"1+1"))'`	`2`
8	`python -c "from juliacall import Main as jl; print(jl.eval('1+1'))"`	`2`

Common Pitfalls

- **Forgetting `--project`** — Julia defaults to the global environment. Always pass `--project=.` or `--project=@.`.
- **Mixing `test/Project.toml` and `[extras]`** — Choose one pattern. Using both causes resolution conflicts.
- **`SymbolServer` deprecation** — Do not install `SymbolServer.jl` directly; use `JuliaWorkspaces.jl` via the editor extension.
- **`PythonCall` vs `uv`** — `PythonCall` creates its own Conda environment. Set `JULIA_CONDAPKG_BACKEND=Null` to use `uv`-managed Python.
- **RCall and `renv`** — `RCall` uses the system R, not the `renv`-project R. Set `R_HOME` to the `renv` library path if needed.
- **Shebang portability** — `#!/usr/bin/env julia` requires `julia` on PATH. On Windows, invoke via `julia bin/hello.jl`.

Unverified Items

- `[unverified]` The exact behavior of `JuliaCall` when `JULIA_PROJECT` points to a project with a custom `Manifest.toml`. Check the PythonCall.jl documentation.
- `[unverified]` Whether `eglot-jl` fully supports Julia 1.13.1's LSP features. Check the eglot-jl repository.

- `[unverified]` The precise zero-copy semantics for string columns in `pytable`. Check the PythonCall.jl conversions page.

Additional Notes

- **Julia version pinning** — The shebang does not select a Julia version. Use `juliaup` to install 1.13.1 and set it as default.
- **CRAN compliance** — Adding `Project.toml` and `Manifest.toml` to the repository root may trigger CRAN checks. Add them to `.Rbuildignore` and document the Julia integration as a non-R component.
- **Workspace vs. subdirectory** — For this project, a Pkg workspace is appropriate if you plan to add multiple Julia sub-packages. Otherwise, a single `Project.toml` at the root is sufficient.

References

A:1 (Kimi)

^ ***Adding Julia to a hybrid R + Python research project***

- see: Kimi

A hands-on tutorial for expert Python and R programmers learning Julia. All examples target Julia 1.13.1, the latest stable release (1.13.0 shipped 2026-09-10, 1.13.1 on 2026-09-25), with no legacy-compatibility constraints.

The running example is a research monorepo that already contains an R package (`dvesimpler`, built with Rcpp, managed by `renv`) and a Python package (`dvepy`, managed by `uv`). We add a Julia package `Dve` and wire all three languages together.

0. Repository tree and first-time bootstrap

0.1 Proposed tree

Julia's package manager expects a `Project.toml` at the project root, the package entry point at `src/Dve.jl`, tests under `test/`, and arbitrary folders for sub-environments. The layout below keeps the existing R package structure untouched and adds only CRAN-ignorable entries. Every deviation from the original tree is listed in §0.3.

```
<project-root>
├── .Rbuildignore                 # <- extend: ignore all Julia files (§0.3)
├── .gitignore                    # <- add: /.CondaPkg, /test/Manifest.toml etc.
├── Project.toml                  # <- Julia "Dve" package + workspace root
├── Manifest.toml
├── juliapkg.json                 # <- juliacall-side Julia dependencies (§3.3)
├── pyproject.toml                # Python "dvepy" (uv)
├── uv.lock
├── DESCRIPTION                   # R "dvesimpler"
├── renv.lock
├── R/
├── inst/
│   ├── include/                  # Rcpp headers
│   └── python/dve/__init__.py    # the `dve` module of Python package `dvepy`
├── src/
│   ├── Makevars                  # Rcpp build
│   ├── *.cpp                     # Rcpp sources
│   └── Dve.jl                    # <- Julia package entry point
```

```
├── test/
│   ├── Project.toml              # <- Julia test environment (workspace member)
│   └── runtests.jl               # <- shim that includes ../tests/*.jl
├── tests/
│   ├── testthat/                 # R tests (devtools::test())
│   └── test_hello.jl             # <- Julia test file (lives next to R tests)
├── bin/
│   ├── Project.toml              # <- script environment (workspace member)
│   └── hello.jl                  # <- executable entry point with CLI
└── (notebooks may live anywhere under the root; see §1.2)
```

0.2 Concept map: Python/R to Julia equivalents

Python / R concept	Julia equivalent
`venv` / `renv` project library	Pkg environment: `Project.toml` + `Manifest.toml`
`pip install pkg` / `install.packages()`	`] add PkgName` (Pkg REPL) or `Pkg.add("PkgName")`
`requirements.txt` / `renv.lock`	`Project.toml` `[deps]` (intent) / `Manifest.toml` (exact pins)
`pyproject.toml` / `DESCRIPTION`	`Project.toml` (`name`, `uuid`, `version`, `[deps]`, `[compat]`)
`import numpy as np` / `library(dplyr)`	`using DataFrames`
`from os import path` / `dplyr::select`	`import DataFrames: select, transform`
`None` / `NULL`	`nothing`
`NaN` / `NA`	`missing`
`list` / `c()`	`Vector{T}`
`dict` / named `list()`	`Dict{K,V}`
`def f(x)` / `f ← function(x)`	`f(x) = ...` or `function f(x) ... end`
class methods / S3 dispatch	multiple dispatch
broadcast (`np`-style vectorization) / `Vectorize()`	dot syntax: `sin.(x)`, `x .+ 1`
pytest / testthat	`Test` stdlib + `] test`
`python script.py` / `Rscript script.R`	`julia --project=. script.jl`
editable install `pip install -e .`	`] dev path/to/Package`

0.3 Deviations from the assumed layout (CRAN compliance)

- `test/` (Julia) vs `tests/` (R): Julia's `Pkg.test()` hardcodes `test/runtests.jl`; R uses `tests/`. We honor both: real Julia test files stay in `tests/*.jl` next to `testthat/`, and `test/runtests.jl` is a two-line shim. This is the only clean way to satisfy both toolchains.
- New top-level entries (`Project.toml`, `Manifest.toml`, `juliapkg.json`, `bin/`, `test/`, `src/*.jl`, `tests/*.jl`, `.venv`, `.CondaPkg/`) are non-standard for an R package and must be excluded from the CRAN tarball.
- Add to `.Rbuildignore`:

```
^Project\.toml$
^Manifest.*\.toml$
^juliapkg\.json$
^bin$
```

```
^test$
^tests/.*\.jl$
^src/.*\.jl$
^\.venv$
^\.CondaPkg$
```

0.4 Clean-checkout bootstrap

Working directory: <project-root>. Expected output: a working three-language environment.

```
# 1. Julia 1.13.1 via juliaup (also installs the `julia` launcher on PATH)
curl -fsSL https://install.julialang.org | sh
juliaup add 1.13.1 && juliaup default 1.13.1

# 2. Python side (creates .venv, installs dvepy editable + juliacall)
uv sync                                    # or: uv venv && uv pip install -e inst/python juliaca

# 3. R side
R -e 'renv::restore()'

# 4. Julia side: instantiate the committed Manifest.toml
julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

The first `Pkg.instantiate()` downloads and precompiles all pinned dependencies; this can take several minutes once.

1. Julia project skeleton

1.1 Project files

Working directory: <project-root>.

Generate the UUID once (it identifies the package forever; it goes into `Project.toml` and later into `juliapkg.json`):

```
julia -e 'using UUIDs; println(uuid4())'
```

`Project.toml` (fill in the printed UUID):

```
# Project.toml
name = "Dve"
uuid = "8d1e5f2a-3c4b-4a6e-9b7d-2f1a0c9e5d77"   # <- replace with your uuid4() output
version = "0.1.0"

[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
PythonCall = "6099a3de-0756-4e0e-815e-e66c5fce93e4"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"
```

```
[compat]
DataFrames = "1"
PythonCall = "0.9"
julia = "1.13"

[workspace]
projects = ["test", "bin"]
```

Only `name`, `uuid`, `version` and `[deps]` are required for a loadable package; `[compat]` and `[workspace]` are added now because §2 and §3 build on them. `src/Dve.jl` is the package:

```
# src/Dve.jl
module Dve

export hello_world

"""
    hello_world(salutation, who) → String

Return the greeting string `"\$salutation, \$who!"`.
Python equivalent: f"{salutation}, {who}!"  —  R equivalent: sprintf("%s, %s!", salutation, who)
"""
hello_world(salutation::AbstractString, who::AbstractString) = string(salutation, ", ", who, "!"

end # module Dve
```

Working directory: `<project-root>`. Expected output: the greeting string.

```
julia --project=. -e 'using Dve; println(Dve.hello_world("Hello", "World"))'
```

```
Hello, World!
```

Contrast with Python/R: `using Dve` (like `library(dvesimpler)`) loads the package defined by the active `Project.toml` from `src/Dve.jl` — a convention the loader hardcodes when the project name matches the module name (details in §2.3). The type annotations `::AbstractString` are not needed for correctness; they restrict this method to string-like arguments and participate in multiple dispatch.

1.2 Jupyter kernel

Install IJulia in the *base* environment (not the project), then register a project-bound kernel:

```
# Run in the base Julia environment (no --project flag):
julia> using Pkg; Pkg.add("IJulia")
julia> using IJulia
julia> IJulia.installkernel("Julia 1.13 (Dve)", "--project=@.")
```

Working directory: any. Expected output: a kernelspec directory such as `~/.local/share/jupyter/kernels/julia-1.13-(dve)-.../kernel.json`.

```
jupyter kernelspec list
```

How the kernel binds to the project: `installkernel` writes a `kernel.json` whose `argv` launches `julia -i --color=yes --project=@. -e "import IJulia; IJulia.run_kernel()" {connection_file}`. The `--project=@.` flag tells the Julia process, at kernel startup, to search for a `Project.toml` starting from the kernel's current directory (the notebook's directory) and walking upward — so any notebook located anywhere under the repository automatically uses the `Dve` environment. The IJulia package itself is provided by the base environment; `Dve`, `DataFrames`, etc. come from the project. The default kernel that IJulia installs also passes `--project=@.` , but for custom kernels you must pass it explicitly, otherwise the kernel starts in the base environment.

Sanity check, first cell of a notebook saved under `<project-root>`:

```julia
using Pkg; Pkg.status()   # shows Dve, DataFrames, PythonCall — confirms the right env
using Dve
Dve.hello_world("Hello", "notebook")
```

If the notebook lives outside the repo, replace `"--project=@."` with an absolute `--project=/path/to/<project-root>` at install time (or set `env=Dict("JULIA_PROJECT" ⇒ "/path/to/<project-root>")`).

1.3 Executable script with command-line interface

The script lives outside the package in `bin/`. It activates the `bin` project (a workspace member, §2.4) relative to its own location, so it works from any working directory; the shebang makes it directly executable.

```
# bin/Project.toml
[deps]
ArgParse = "c7e460c6-2fb9-53a9-8c5b-16f535841c63"

[sources]
Dve = {path = ".."}
```

```julia
# bin/hello.jl
#!/usr/bin/env julia
# Greet somebody using the Dve package.
#
# The shebang delegates to `julia` on PATH; the project environment is found
# *relative to this file* (not to the caller's working directory) via @__DIR__.
# One-liner alternative for GNU/BSD env only:
#     #!/usr/bin/env -S julia --project=@.
# (portability of `env -S` on macOS is unverified — see the Unverified list.)

using Pkg
Pkg.activate(@__DIR__)   # resolves bin/Project.toml; the root [workspace] unifies the Manifest

using ArgParse
using Dve
```

```
function parse_cli()
    s = ArgParseSettings(
        description = "Print a greeting produced by the Dve Julia package.",
        add_help = true,          # provides -h / --help with full documentation
    )
    @add_arg_table! s begin
        "--salutation", "-s"
            help = "salutation to use"
            arg_type = String
            default = "Hello"
        "who"
            help = "whom to greet"
            arg_type = String
            required = true
    end
    return parse_args(s)          # parses ARGS; exits with the help text on -h/--help
end

function main()
    args = parse_cli()
    println(Dve.hello_world(args["salutation"], args["who"]))
end

main()
```

Working directory: any. Expected output: usage text, then the greeting.

```
chmod +x bin/hello.jl
./bin/hello.jl --help
```

```
usage: hello.jl [--salutation SALUTATION] [-s SALUTATION] who

Print a greeting produced by the Dve Julia package.

positional arguments:
  who              whom to greet

optional arguments:
  -s, --salutation  salutation to use (default: "Hello")
  -h, --help        show this help message and exit
```

```
./bin/hello.jl -s Hi Julia
```

```
Hi, Julia!
```

Contrast with Python/R: `ArgParse.jl` plays the role of `argparse`/`optparse`; Julia has no built-in

equivalent. `ARGS` is the analogue of `sys.argv[1:]` / `commandArgs(trailingOnly = TRUE)`.

1.4 Unit test

Julia's `Pkg.test()` hardcodes the entry point `test/runtests.jl`. Since this repository keeps test files in `tests/` (R package layout), the shim includes them:

```
# test/Project.toml
[deps]
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"

[sources]
Dve = {path = ".."}
```

```
# test/runtests.jl
using Dve
using Test

# Julia convention requires this entry point; the actual test files live in
# ../tests/ next to the R testthat suite, per the repository layout.
@testset "Dve" begin
    include(joinpath(@__DIR__, "..", "tests", "test_hello.jl"))
end
```

```
# tests/test_hello.jl
@testset "hello_world" begin
    @test Dve.hello_world("Hello", "World") == "Hello, World!"
    @test Dve.hello_world("Hi", "Julia") == "Hi, Julia!"
    @test Dve.hello_world("Bonjour", "monde") isa String
end
```

Working directory: `<project-root>`. Expected output: a green test summary.

```
julia --project=. -e 'using Pkg; Pkg.test()'
```

```
     Testing Running tests...
Test Summary: | Pass  Total  Time
Dve           |    3      3  0.0s
     Testing Dve tests passed
```

Contrast with Python/R: `@test` behaves like `assert`/`test_that` + `expect_equal`; `@testset` groups and labels tests. The `Test` package is part of the standard library, like R's `tools`/`testthat` hybrid but always available.

1.5 LSP editor support

1.5.1 What the three packages are

`LanguageServer.jl`, `JuliaWorkspaces.jl` and `SymbolServer.jl` are *components of one system*, not alternatives:

- `LanguageServer.jl` implements the Microsoft Language Server Protocol (LSP) for Julia — hover, completion, go-to-definition, linting . Every editor integration (VS Code, Emacs, Vim, Neovim) ultimately talks to an instance of it.
- `SymbolServer.jl` is a helper package of `LanguageServer.jl` that provides information about the internal and exported variables of installed packages *without loading them*, building a disk cache for fast startup . It answers ”what does `DataFrames.transform` look like?” without paying the JIT cost of importing DataFrames.
- `JuliaWorkspaces.jl` is the workspace-model library that `LanguageServer.jl` depends on for discovering projects, environments, test items and includes in the folders you have open. Its notion of ”workspace” (editor folders) is unrelated to Pkg's `[workspace]` table (§2.4) — same word, different layer.

The wiring is therefore: editor client (VS Code extension or eglot/lsp-mode) launches `LanguageServer.jl`, which uses `JuliaWorkspaces.jl` for your code and `SymbolServer.jl` for dependency packages. Deprecation check: older guides told you to `Pkg.add("LanguageServer")` into every project — that is obsolete advice. The server and its helpers belong in a dedicated environment managed by the editor integration (or one you create yourself, §2.2), never in your package's `[deps]`.

1.5.2 VS Code

Install the official Julia extension (`julialang.language-julia`). It downloads and launches its own pinned `LanguageServer.jl` — no manual setup. Only configure `julia.executablePath` if `julia` is not on your `PATH`. The project environment is picked up automatically from the folder you open (it respects `Project.toml` discovery, including the workspace layout of §2.4).

1.5.3 Emacs: eglot vs lsp-mode

Criterion	eglot + eglot-jl
Purpose	Minimal LSP client using built-in Emacs facilities (completion-at-point, flymake, project.el)
Maintenance status	eglot is built into Emacs 29+; `eglot-jl` (MELPA, by non-Jedi) is actively maintained and l
Setup effort	Lowest for vanilla Emacs
Prefer when	You want zero extra UI and fast setup

Recommended configuration (eglot path):

```
;; init.el — Julia editing + LSP via eglot + eglot-jl
(use-package julia-mode
  :ensure t
```

```
  :mode "\\.jl\\'")

;; eglot itself is built into Emacs 29+. On older Emacsen: (use-package eglot :ensure t)
(use-package eglot-jl
  :ensure t
  :after (julia-mode eglot)
  :config
  (eglot-jl-init)             ; registers the Julia language server with eglot/project.el
  :hook (julia-mode . eglot-ensure))
```

The first `M-x eglot` in a `julia-mode` buffer downloads and installs `LanguageServer.jl` into a private environment under the Julia depot, which can take minutes — increase `eglot-connect-timeout` for that first run. There is no per-project server configuration: the server discovers `Project.toml`/`Manifest.toml` the same way Pkg does.

lsp-mode alternative (verify the exact customization names on MELPA — see Unverified list):

```
(use-package lsp-julia
  :ensure t
  :after lsp-mode
  :config
  (setq lsp-julia-default-environment "~/.julia/environments/v1.13")
  :hook (julia-mode . lsp-deferred))
```

2. Julia project structure

2.1 Dependency management and UUIDs

Is `Pkg.add` enough? For anything in a registry you have installed (by default: the General registry, a git clone under `~/.julia/registries/General`), yes:

- `] add DataFrames` resolves a version compatible with your `[compat]` bounds, records `DataFrames = "<uuid>"` in `Project.toml [deps]`, and pins the exact versions of DataFrames *and everything it needs* in `Manifest.toml`. This mirrors `pip install` + `uv.lock`, or `install.packages()` + `renv.lock`, in one step.

Where `Pkg.add` is not enough:

- *UUIDs.* A UUID is a package's permanent identity; it is generated once by `Pkg.generate` (or `uuid4()`) and must never change. Two packages may share a name but never a UUID, which is why `[deps]` entries are `name = "uuid"` pairs. Your own `Dve` needs a UUID before other packages (or `juliapkg.json`, §3.3) can depend on it.
- *Local, unregistered packages* (like `Dve` before you publish it): use an editable reference —
  - `] dev path/to/Dve` (like `pip install -e .`): Manifest records the path; changes to the source are picked up on restart. Preferred while developing.
  - or, on Julia 1.12+, a `[sources]` entry in the consuming `Project.toml`: `Dve = {path = ".."}`. This is what `test/Project.toml` and `bin/Project.toml` above use; it works both

inside and outside `Pkg.test`, which `[extras]` does not (§2.2).

- *Packages not in any registry*: `] add https://github.com/Org/SomePkg.jl.git` (optionally `#branch` or `#commit`). For a team, run a private registry with `LocalRegistry.jl` instead of copy-pasting URLs — then `Pkg.add` is enough again.
- *[compat] bounds.* `DataFrames = "1"` means `⩾1.0, <2.0`; `"1.2"` means `⩾1.2, <1.3` (caret semantics by default). Add bounds for every direct dependency; CompatHelper (GitHub Action) opens PRs when new versions land.

2.2 Dev, test and LSP dependencies

Three mechanisms exist for test-only dependencies:

Mechanism	Visible to
`[extras]` + `[targets]` in the main `Project.toml`	only `Pkg.test()` — it synthesizes a temporary enviro
`test/Project.toml` (+ `[sources] Dve = {path=".."}`)	`Pkg.test()` and manual activation
`test/` as a `[workspace]` member	same, plus a single unified `Manifest.toml` for the wh

Recommendation: the workspace approach (`[workspace] projects = ["test", "bin"]` + `test/Project.toml` with `[sources]`), because the test environment is a normal project — it resolves against the root manifest, precompiles in parallel with the main package, and you can open it interactively (`julia --project=test`) to debug tests without `Pkg.test`'s separate process.

LSP and dev tooling (LanguageServer, SymbolServer, Revise, JuliaFormatter, TestEnv) must never enter `[deps]` — users of your package would inherit them. Dedicate an environment, e.g. for a manual/editor setup:

Working directory: any. Expected output: an isolated env with the server packages.

```
julia --project="$HOME/.julia/environments/lsp-1.13" -e 'using Pkg; Pkg.add(["LanguageServer", "
```

VS Code and eglot-jl manage equivalent environments automatically (§1.5), so you only need this for editors without built-in bootstrap.

2.3 How source, test and script folders are determined

- *Source.* Nothing in `Project.toml` configures source locations. When the active project's `name` matches the module you `using`, the loader falls back to the hardcoded convention `src/<name>.jl`; further files are pulled in with `include("file.jl")` from there. That is why `src/Dve.jl` must keep its exact name and location. (`[sources]` maps *dependency names to paths*; it does not move package entry points.)
- *Tests.* `Pkg.test()` hardcodes `test/runtests.jl`. There is no configuration key to relocate it — hence the shim in §1.4.
- *Scripts.* Any folder is fine (`bin/`, `scripts/`, `tools/`); pure convention. A script that needs extra packages gets its own `Project.toml` (as `bin/` does), ideally as a workspace member so it shares the root manifest.

Summary: only `[workspace] projects` and `[sources]` in `Project.toml` point at folders; everything else is convention enforced by the loader or by `Pkg.test`.

2.4 Workspaces

A Pkg *workspace* is a group of projects declared in a root `Project.toml` that resolve together into a single root `Manifest.toml`:

```
# Project.toml (root) — already present in our skeleton
[workspace]
projects = ["test", "bin"]
```

Each listed folder must contain its own `Project.toml` (its own deps and compat), and the root manifest then pins one coherent set of versions for all of them. The Pkg prompt shows membership: `(Dve/test) pkg>`. Workspaces were introduced in Julia 1.12 and are not backported to older versions, which silently ignore the `[workspace]` table .

Minimal extension example — adding a benchmark environment:

Working directory: `<project-root>`. Expected output: a new workspace member prompt.

```
mkdir -p bench && printf '[deps]\nBenchmarkTools = "6e4b80f9-dd63-53aa-95a3-0cdb28fa8baf"\n\n[so
# add "bench" to the [workspace] projects list in Project.toml, then:
julia --project=bench -e 'using Pkg; Pkg.instantiate()'
```

Use cases:

- Monorepo of several *unregistered, interdependent* packages (exactly this repository's situation): no `LocalRegistry.jl` needed; the root manifest records each member by path.
- Test, docs and benchmark environments with one shared resolve — no version drift between main code and tests, and faster parallel precompilation.
- CI: one `Pkg.instantiate()` at the root materializes everything.

Two gotchas: (1) a workspace member is *not* automatically importable — to `using` another member from the root project you still add it to `[deps]` and `] dev` it (or use `[sources]`); (2) keep member-local `Manifest.toml` files out of the way — a stray manifest in `test/` can block workspace detection, so delete them (or keep versioned ones such as `Manifest-v1.11.toml` for cross-version work).

The word ”workspace” appears in three unrelated places: Pkg's `[workspace]` table (this section), `JuliaWorkspaces.jl` (editor-side model for the language server, §1.5.1), and your editor's generic workspace folders. They do not interact.

3. Language integration examples

Three bridges are used, one per direction, each chosen for active maintenance and copy semantics:

Direction	Package	Alternatives considered	Wh
R → Julia	`JuliaCall` (R package, CRAN)	`XRJulia`	CRA
Julia → Python	`PythonCall.jl`	`PyCall.jl`	No

Direction	Package	Alternatives considered	Wh
Python → Julia	`juliacall` (PyPI; the Python-side twin of PythonCall.jl)	`PyJulia` (`pip install julia`)	Sam

3.1 R calls Julia (`JuliaCall`)

Bootstrapping and environment discovery

JuliaCall is an ordinary R package: put it in `renv.lock` (`renv::install("JuliaCall")`), then `renv::restore()` restores it like any other dependency — nothing Python-like has to be configured on the R side. It locates the Julia *binary* (a single user-wide installation, never duplicated) by, in order: the `JULIA_HOME` argument of `julia_setup()`, the R option `JULIA_HOME`, the environment variable `JULIA_HOME`, then `julia` on `PATH`; you can also specify the location of the Julia binary with the `JULIA_HOME` keyword argument, and `julia_setup(installJulia = TRUE)` can even install Julia for you . The minimum supported versions are Julia 1.10 and R 4.4 .

The Julia *project environment* is selected by the standard `JULIA_PROJECT` variable, which the Julia subprocess started by R inherits. Set it before `julia_setup()`:

```
# tests/testthat or an R session; working directory: <project-root>
library(JuliaCall)

Sys.setenv(JULIA_PROJECT = normalizePath("."))   # the repo root Project.toml = the Dve env
# If julia is not on PATH, one of:
#   Sys.setenv(JULIA_HOME = "~/.juliaup/bin")
#   julia <- julia_setup(JULIA_HOME = "~/.juliaup/bin")
julia <- julia_setup()          # first run: installs a small setup script + precompiles (minute

# Make the project package visible inside Julia:
julia_command("using Dve")
```

Expected output on first run: `Julia version 1.13.1 at location ... will be used. Loading setup script for JuliaCall... Finish loading setup script.` Subsequent runs are much faster; use `julia_setup(rebuild = TRUE)` if you upgrade R.

renv is never ”discovered” by Julia — the R side just supplies the JuliaCall package itself. The Julia depot (`~/.julia`) and the per-project environment are the analogues of renv's per-project library and are already handled by `JULIA_PROJECT`.

Primitives: arguments and return values

```
julia_call("sqrt", 2)                    # 1.414214  — Int/Float/Str/Bool convert automatically
julia_call("Dve.hello_world", "Hello", "World")   # "Hello, World!" — our own package
julia_call("sin.", c(0, pi/4, pi/2))     # c(0, 0.7071068, 1) — trailing dot = broadcast, like V
```

Conversion is by value: an R vector is copied into a Julia `Vector{Float64}`, and results are copied back. `julia_call("name", ...)` resolves `name` in Julia's `Main` module, which is why `using Dve` must run first. `julia_eval("sqrt(2)")` evaluates arbitrary expressions; `julia_assign("x", r_obj)` /

`julia_eval("x")` move data by name.

Complex types: R `data.frame` → Julia `DataFrame` and back

Load `DataFrames` on the Julia side once; afterwards `data.frames` convert automatically in both directions — the conversion is "very nicely working both ways: data.frame and DataFrame" :

```
julia_command("using DataFrames")        # or julia_library("DataFrames")

julia_assign("df", mtcars)               # R data.frame becomes a DataFrames.DataFrame in Julia
julia_command("df.log_mpg = log.(df.mpg)")
julia_command("using Statistics")
julia_command("df.hp_mean = mean(df.hp)")  # scalar: 146.6875

back <- julia_eval("df")                 # Julia DataFrame becomes an R data.frame again
class(back)                              # "data.frame"
```

Working directory: `<project-root>` (R session). Expected output: `back` is a `data.frame` with 32 rows and the new `log_mpg` column; `back$hp_mean[1]` is 146.6875.

How the conversion works: JuliaCall marshals the R object across the C boundary and, with DataFrames loaded on the Julia side, maps `data.frame` → `DataFrames.DataFrame` (each column becomes a typed Julia vector) and back. This is a *copy* in both directions — there is no zero-copy path between R and Julia. For large data, cross the boundary in Arrow format instead (write with Julia's `Arrow.jl`, read with R's `arrow` package, or vice versa) so the payload is serialized once.

3.2 Julia calls Python (`PythonCall.jl`)

Bootstrapping and environment discovery

By default PythonCall manages a private Conda environment per Julia project via `CondaPkg.jl` — convenient, but it would duplicate the packages that `uv` already manages. Since this repository has a `uv` project, point PythonCall at the uv-managed `.venv` and disable Conda (the standard pattern is the "CondaPkg backend set to Null, and set the exe to the python venv"):

```
# Working directory: <project-root>; run BEFORE `Pkg.add("PythonCall")` (or persist via preferen
using Pkg
Pkg.activate(".")

ENV["JULIA_CONDAPKG_BACKEND"] = "Null"   # never touch Conda
ENV["JULIA_PYTHONCALL_EXE"] = "@venv"    # use the .venv of the active project (uv's venv)

Pkg.add("PythonCall")    # no build step: nothing is compiled against libpython
```

The two settings can also be persisted as Julia preferences (recommended for a shared repo):

```
using PreferenceTools   # ] preference add CondaPkg backend=Null
                        # ] preference add PythonCall exe=@venv
```

(If your uv venv lives elsewhere, use an absolute path for `exe`. `exe` also accepts `@CondaPkg` (default), `@PyCall`, and `@venv`.) Now install the Python-side dependencies into the same environment that uv manages:

Working directory: `<project-root>`. Expected output: `dve` importable from `.venv`.

```
uv pip install -e inst/python numpy pandas
```

Primitives: arguments and return values

```
# Working directory: <project-root>; julia --project=.
using PythonCall

np = pyimport("numpy")
x  = np.linspace(0.0, 1.0, 5)
y  = np.sin(x)                              # a Python object (type Py), not yet converted
pyconvert(Vector{Float64}, y)               # explicit conversion: copies into Vector{Float64}
```

PythonCall deliberately keeps results as Python objects (`Py`) and converts only on request with `pyconvert(T, x)` — the analogue of `py.call` + explicit casting, and more predictable than PyCall's eager automatic conversion. The general rule: mutable containers (lists, dicts, numpy arrays) are wrapped, while immutable objects such as numbers and strings are converted (copied) to the native Julia equivalent .

Complex types: Julia `DataFrame` ↔ `pandas.DataFrame`

```
using DataFrames

df  = DataFrame(a = 1:3, b = ["x", "y", "z"])
pdf = pytable(df)                    # Tables.jl table → NEW pandas.DataFrame (data is copied)
pdf.describe()                       # call pandas methods directly on the result

# pandas → Julia, zero-copy wrapper first:
back = DataFrame(PyTable(pdf))       # PyTable is a lazy Tables.jl view of the pandas object
```

Working directory: `<project-root>`. Expected output: `pdf` prints a 3-row pandas DataFrame; `back` is a 3×2 `DataFrame` equal to `df`.

How the conversion works: `pytable(src, format=:pandas)` builds a fresh pandas DataFrame from any Tables.jl-compatible table (a copy; `pytable` accepts any Tables.jl-compatible source). In the other direction, `PyTable(x)` (shorthand for `pyconvert(PyTable, x)`) wraps a pandas DataFrame as `PyPandasDataFrame`, a zero-copy Tables.jl-compatible view; `DataFrame(::PyTable)` materializes it, with numpy-backed columns sharing memory where dtypes allow. Numeric arrays also cross cheaply: a strided Julia array satisfies Python's buffer protocol (numpy can wrap it), and `PyArray(np_array)` gives a zero-copy `AbstractArray` view of a numpy array.

Calling the project package `dvepy`

The Python module `dve` (inside the `dvepy` distribution) is installed in the same `.venv`:

```python
# inst/python/dve/__init__.py
def double(x):
    """Double the input."""
    return 2 * x
```

```julia
using PythonCall   # Working directory: <project-root>; julia --project=.
dve = pyimport("dve")
dve.double(21)            # 42 (a Python int; wrap with pyconvert(Int, ...) if needed)
```

Because PythonCall is bound to the uv venv, `import dve` resolves to the editable install of `inst/python/dve/` — a single source tree, no duplicated installation.

3.3 Python calls Julia (`juliacall`)

Bootstrapping and environment discovery

Install the Python package from PyPI into the uv project:

Working directory: `<project-root>`. Expected output: juliacall added to the lockfile.

```
uv add juliacall numpy pandas        # or: uv pip install juliacall numpy pandas
```

On the *first* `import juliacall`, its dependency manager (`pyjuliapkg`) finds Julia (or downloads one) and automatically installs Julia and the required Julia packages if they are not already there , creating a private Julia environment and installing the matching `PythonCall.jl` into it — one-time, several minutes of precompilation. To use the repository's own Julia project instead of a private one, declare the Julia dependencies in `juliapkg.json` at the repo root (next to `pyproject.toml`); pyjuliapkg then dev-installs `Dve` from its path, reusing the workspace resolution:

```jsonc
// juliapkg.json
{
    "packages": {
        "PythonCall": {},
        "DataFrames": {},
        "Dve": { "uuid": "<same UUID as in Project.toml>", "path": ".", "dev": true }
    }
}
```

The `"dev": true` + `"path"` combination is the `pip install -e .` equivalent for Julia packages (required because `Dve` is not registered) . Alternatively, for one-off scripts on shared machines, disable pyjuliapkg entirely and point at an existing environment — both variables must be set together (`PYTHON_JULIACALL_EXE`, `PYTHON_JULIACALL_PROJECT`, and optionally `PYTHON_JULIACALL_SYSIMAGE`), and the target project must already contain a version-matched PythonCall :

```
import os
```

```
os.environ["PYTHON_JULIACALL_EXE"] = "/path/to/julia"     # optional
os.environ["PYTHON_JULIACALL_PROJECT"] = "/path/to/<project-root>"
```

Under `uv run python ...`, juliacall runs inside the uv-managed interpreter automatically — `uv` and `renv` never need to be "found": each bridge simply inherits the interpreter of the process that started it.

Primitives: arguments and return values

```
# Working directory: <project-root>; uv run python
from juliacall import Main as jl   # `jl` is Julia's Main module
import numpy as np

jl.seval("using Statistics")       # evaluate Julia code (like R's julia_eval)
jl.mean(np.array([1.0, 2.0, 3.0])) # 2.0 — numpy arrays convert without copying
```

For library code, avoid polluting `Main`:

```
import juliacall
jl = juliacall.newmodule("MyProject")
jl.seval("using Statistics")
```

Complex types: `pandas.DataFrame` ↔ Julia `DataFrame`

```
# Working directory: <project-root>; uv run python
from juliacall import Main as jl
import pandas as pd

jl.seval("using DataFrames")

df  = pd.DataFrame({"a": [1, 2, 3], "b": [1.5, 2.5, 3.5]})
jdf = jl.DataFrame(df)               # pandas → Julia DataFrame (via the PyTable rule)
jdf.a                                # works like a Python attribute
back = jl.PythonCall.pytable(jdf)    # Julia table → NEW pandas.DataFrame
list(back.columns)                   # ['a', 'b']
```

Working directory: `<project-root>`. Expected output: `jdf` prints as a 3×2 DataFrame; `back` equals the original `df`.

Conversion semantics are exactly PythonCall's, in mirror image: pandas DataFrames convert through the `PyTable` wrapper (lazy, zero-copy view over the numpy columns), while `pytable(...)` constructs a new pandas object (a copy). Numbers, strings, booleans and tuples convert by value; mutable containers are wrapped, not copied.

Calling `Statistics` and the project package `Dve`

```
jl.seval("using Dve")                        # works because juliapkg.json dev-installs Dve
jl.Dve.hello_world("Hello", "World")         # 'Hello, World!'
```

```
jl.seval("using Statistics")
jl.mean([1.0, 2.0, 4.0])                    # 2.3333333333333335
```

Functions reached through a module object (`jl.Dve.hello_world`) need no `export`; unexported names are also accessible that way, e.g. `jl.Dve.hello_world` would work even without the `export` in §1.1.

3.4 Conversion semantics at a glance

Payload	JuliaCall (R→Julia)	PythonCall (Julia→Python)
Int, Float, Bool, String	copied by value	copied by value
Vectors / numpy arrays	copied both ways	numpy → Julia: zero-copy view
`data.frame` / `DataFrame` / pandas	copied both ways (use Arrow for big data)	`pytable` copies; `PyTable` zero-c
Functions	wrapped (callable from the other side)	wrapped as `Py` / `AnyValue`

3.5 Interpreter and environment discovery, summarized

Question	R → Julia (JuliaCall)
Where is the foreign binary?	`julia` on `PATH`, else `JULIA_HOME` (arg > R option > env var)
Which environment is used?	`JULIA_PROJECT` inherited by the Julia subprocess — set before `julia_setup()`
How are duplicates avoided?	One Julia install per machine (depot is user-wide); per-repo env via `JULIA_PROJE`
First-run cost	`julia_setup()` installs setup deps + precompiles

First-run bootstrapping, end to end, on a clean checkout: `juliaup add 1.13.1` → `uv sync` → `renv::restore()` → `julia --project=. -e 'using Pkg; Pkg.instantiate()'` → one `julia_setup()` call from R → one `import juliacall` from Python. Every step after that is incremental.

4. Acceptance checklist

#	Command (working directory)
1	`julia --project=. -e 'using Dve; println(Dve.hello_world("Hello", "World"))'` (<project-root>
2	`julia --project=. -e 'using Pkg; Pkg.instantiate()'` (<project-root>, clean clone)
3	`julia --project=. -e 'using Pkg; Pkg.test()'` (<project-root>)
4	`./bin/hello.jl --help` (any cwd)
5	`./bin/hello.jl -s Hi Julia` (any cwd)
6	`jupyter kernelspec list` (after §1.2 install)
7	VS Code: open root, hover over `hello_world` in `src/Dve.jl`
8	`R -e 'library(JuliaCall); Sys.setenv(JULIA_PROJECT="."); julia_setup(); julia_command("using`
9	R: `julia_assign("df", mtcars); julia_eval("df")` after `julia_command("using DataFrames")`
10	`julia --project=. -e 'using PythonCall, DataFrames; dve = pyimport("dve"); println(pyconvert`
11	`julia --project=. -e 'using PythonCall; np = pyimport("numpy"); println(pyconvert(Vector{Flo`

#	Command (working directory)
12	`uv run python -c "from juliacall import Main as jl; jl.seval('using Dve'); print(jl.Dve.hell`
13	Python: pandas round trip of §3.3
14	`R CMD build .` (`<project-root>`)

5. Common pitfalls

- *Naming collision.* The R package is `JuliaCall` and the Python module is also `juliacall` (exposing a `JuliaCall`-branded API in the PythonCall docs). Same word, two unrelated ecosystems — check which side a guide is talking about.
- *PythonCall configured too late.* `JULIA_CONDAPKG_BACKEND` / `JULIA_PYTHONCALL_EXE` must be set before `Pkg.add("PythonCall")` or persisted as preferences; setting them after installation is ignored on some versions, and PythonCall will happily create its own Conda env — duplicating exactly what `uv` already manages.
- *`[extras]`/`[targets]` only works via `Pkg.test`.* Running `julia --project=. test/runtests.jl` fails with "Package X not found" for extras. Use `test/Project.toml` (+ workspace) as in §2.2.
- *`--project=@.` resolves from the kernel's working directory, not the script's location.* For `bin/hello.jl` that is why we `Pkg.activate(@__DIR__)` instead of trusting the shebang; for notebooks keep them under the repo or use an absolute `--project`.
- *IJulia installed in the project env.* The kernel then starts but `IJulia` is missing from the base env and the kernel dies silently. Keep IJulia in the base environment; bind kernels to projects via `--project`.
- *Workspace detection blocked by stray manifests.* A `Manifest.toml` inside `test/` or `bin/` can prevent workspace unification on 1.12+; keep only the root manifest (`Manifest-v1.11.toml`-style versioned files are the exception).
- *Forgetting that Julia is 1-based and inclusive.* `df[2:3, :]` in DataFrames.jl corresponds to pandas `df.iloc[1:3]`. Off-by-one bugs live at every language boundary.
- *`missing` vs `NaN` vs `NA`.* Julia's `missing` propagates like R's `NA`; pandas uses `NaN` for both missing data and not-a-number. Expect dtype surprises (`Union{Missing, Float64}` columns) after round trips.
- *`using` vs `import`.* Like `library()` vs `pkg::fun`: `using Dve` brings exports in; unexported access needs `Dve.hello_world` qualification — which is also how `jl.Dve.hello_world` works from Python.
- *World-age error when calling just-defined functions from R/Python.* If you `julia_eval("f(x) = ...")` and immediately call `f` through the bridge, Julia may refuse with "method too new". Define functions in files (`include/julia_source`) rather than at the REPL boundary.
- *First-run latency mistaken for a hang.* `julia_setup()`, first `import juliacall`, and first `Pkg.test()` each pay precompilation costs; subsequent runs are fast. CI should cache `~/.julia`.
- *CRAN check failures.* Without the `.Rbuildignore` entries of §0.3, `R CMD check` chokes on `Project.toml`, `bin/`, and `*.jl` files; Julia files under `tests/` would even be executed as R scripts.

6. Additional notes

- *Interactive development.* Add `Revise.jl` to a personal environment (`julia -e 'using Pkg; Pkg.add("Revise")'` in a scratch env, or via `~/.julia/config/startup.jl`): edits to `src/Dve.jl` are picked up without restarting the REPL — the Julia analogue of `devtools::load_all()` + autoreload.
- *Running tests inside a live session.* `TestEnv.jl` activates the test environment (`test/Project.toml`) in your current REPL, replicating what `Pkg.test` does but interactively.
- *Large data between R and Julia.* JuliaCall always copies; for serious volume use Apache Arrow (`Arrow.jl` □ R's `arrow` package) or an on-disk Parquet/CSV exchange rather than growing bridge calls.
- *Startup latency in production.* If `bin/hello.jl`-style scripts start too slowly, compile a system image with `PackageCompiler.jl` (or pass `sysimage_path` to `julia_setup` on the R side, `PYTHON_JULIACALL_SYSIMAGE` on the Python side).
- *Formatting.* Add a `.JuliaFormatter.toml` (e.g. `style = "blue"`) at the root; the VS Code extension and `JuliaFormatter.jl` both honor it — the analogue of `styler` + `.lintr`.
- *DESCRIPTION metadata.* Put `JuliaCall` in `Suggests:` of `dvesimpler` (the R integration is optional), and note `SystemRequirements: Julia (⩾ 1.10)` in `DESCRIPTION` so CRAN checks and users see the external dependency.
- *Version management.* Pin `julia = "1.13"` in `[compat]` (done in §1.1) and keep the team's Julia updated with `juliaup`; IJulia needs a rebuild (`Pkg.build("IJulia")`) after every minor Julia upgrade so kernels point at the new binary.
- *Notebooks.* Keep `*.ipynb` in a top-level `notebooks/` folder (add it to `.Rbuildignore`); the `--project=@.` kernel then always finds the root environment.

7. Unverified items (documentation to check)

- `JuliaWorkspaces.jl` exact responsibilities and whether it is a hard dependency of current `LanguageServer.jl` — check the JuliaWorkspaces.jl README and LanguageServer.jl Project.toml on GitHub (julia-vscode org).
- Whether `[extras]`/`[targets]` prints a deprecation warning on Julia 1.13 — check the Pkg.jl changelog / `Pkg.test` source for 1.13.
- `#!/usr/bin/env -S julia --project=@.` shebang portability on macOS (and non-GNU `env`) — check your platform's `env(1)` man page; the `Pkg.activate(@__DIR__)` variant in §1.3 is the portable fallback.
- Exact copy semantics of JuliaCall's `data.frame` □ `DataFrame` conversion (deep copy of columns vs shared) — check JuliaCall's vignette "Automatic type conversion" / README section on conversion.
- Whether `jl.DataFrame(pdf)` performs the implicit pandas → `PyTable` conversion in the current juliacall release, or whether the explicit `jl.DataFrame(jl.PythonCall.PyTable(df))` form is required — check the PythonCall.jl "Conversion: Python to Julia" table.
- Whether the IJulia kernel resolves `--project=@.` from the notebook directory or the Jupyter server's working directory in all frontends — check IJulia.jl's troubleshooting/installation docs.
- Whether `JuliaCall` honors `JULIA_PROJECT` in all startup paths or only when inherited from the parent process — check the JuliaCall README ("Julia is not found" / environment sections).

- Current maintenance status and exact customization variable names of `lsp-julia` (e.g. `lsp-julia-default-environment`) — check the emacs-lsp/lsp-julia README and MELPA page.
- Default of `PYTHON_JULIACALL_HANDLE_SIGNALS` with multiple threads on the current juliacall release — check the juliacall configuration table in the PythonCall docs.
- Whether `uv sync` picks up `juliacall` from `pyproject.toml` dependencies cleanly on Windows with a non-default `UV_PROJECT_ENVIRONMENT` — check the uv docs on virtual environments.

References

A:1 (Perplexity)

^ *Julia in a Multi-Language Research Repository*

- see: Perplexity

Assumed Julia version: **1.13.1**, the current stable release. julialang

This tutorial uses:

- **`PythonCall.jl`** for Julia → Python, because it is actively maintained, supports Julia ≥1.10 and Python ≥3.10, and has a symmetric companion package, `juliacall`, for Python → Julia. github
- **`juliacall`** for Python → Julia, for the same reason. pypi
- **`JuliaCall`** for R → Julia, because it is the established CRAN package for embedding Julia in R and provides automatic primitive-type conversion. juliainterop.github

Proposed Repository Tree

```
<project-root>/
├── .Rbuildignore
├── DESCRIPTION
├── Manifest.toml
├── Project.toml
├── pyproject.toml
├── renv.lock
├── uv.lock
├── R/
├── inst/
│   ├── include/
│   └── python/
│       ├── dve/
│       │   └── __init__.py
│       └── dvepy/            # if dvepy is a separate importable package
├── scripts/
│   └── hello.jl
├── src/
│   ├── Dve.jl
│   ├── interop.jl
│   └── Makevars
├── tests/
│   ├── pytest/
```

```
│   ├── testthat/
│   ├── runtests.jl
│   └── test_hello_world.jl
└── vignettes/                  # optional
```

The principal deviation from a conventional Julia package is that tests live in `tests/`, not `test/`. Julia's `Pkg.test()` convention is `test/runtests.jl`; to retain the R-style layout, run the suite explicitly with `julia --project=. tests/runtests.jl`. If you later want `Pkg.test()`, add a `test/runtests.jl` that includes `../tests/runtests.jl`, or move the Julia tests to `test/` and list that directory in `.Rbuildignore`.

1. Julia Project Skeleton

Package Entry Point

`src/Dve.jl` is the module file implied by `name = "Dve"` in `Project.toml`. Julia finds it by convention: the file name must match the package name.

```julia
# src/Dve.jl
module Dve

export hello_world

"""
    hello_world(salutation, who)

Return `"salutation, who!"`.
"""
function hello_world(salutation::AbstractString, who::AbstractString)
    return "$(strip(salutation)), $(strip(who))!"
end

end # module
```

Project File

Create this at the repository root.

```toml
# Project.toml
name = "Dve"
uuid = "11111111-2222-3333-4444-555555555555"
authors = ["Your Name <you@example.com>"]
version = "0.1.0"

[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
PythonCall = "6099a3de-0909-46bc-b1f4-468b9a2dfc0d"
```

```
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"

[compat]
DataFrames = "1"
Julia = "1.13"
PythonCall = "0.9"
Statistics = "1"
julia = "1.13"

[extras]
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"

[targets]
test = ["Test"]
```

Generate a real UUID once with:

```
# Working directory: <project-root>
julia -e 'using UUIDs; println(UUIDs.uuid4())'
```

Expected output: one UUID, for example `3f0d9f2a-...`. Replace the placeholder UUID with it.

`Pkg.add` is enough to add a registered dependency: it writes the package to `[deps]` in `Project.toml` and records the resolved version in `Manifest.toml`. You do not normally hand-write UUIDs; the UUIDs above are the standard registry UUIDs for `DataFrames`, `PythonCall`, `Statistics`, and `Test`. [unverified — confirm UUIDs with `Pkg.add` or the General registry before committing.]

Install Dependencies

```
# Working directory: <project-root>
julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Expected output: package resolution/installation messages, or no output if the manifest is already complete.

Jupyter Kernel

Install `IJulia` in a *separate global environment*, not in the `Dve` project environment:

```
julia -e 'using Pkg; Pkg.activate(); Pkg.add("IJulia")'
```

Then, from the repository root, bind a kernel to the project environment:

```
# Working directory: <project-root>
julia --project=. -e 'using IJulia; installkernel("Julia Dve", "--project=@.")'
```

The `--project=@.` argument makes the kernel activate the project found from the notebook's working directory. Start Jupyter from the repository root so that `Project.toml` is discovered:

```
# Working directory: <project-root>
```

```
julia --project=. -e 'using IJulia; jupyterlab()'
```

In the notebook, select the kernel named **Julia Dve**. The kernel process uses the `Dve` project environment; `IJulia` itself remains in your default global environment.

Executable Script

```
#!/usr/bin/env julia

# scripts/hello.jl
import Pkg

const PROJECT_ROOT = normpath(joinpath(@__DIR__, ".."))

function usage(io::IO = stdout)
    println(io, """
    Usage: hello.jl [--salutation TEXT] [--who TEXT] [--help]

    Print a greeting using the local Dve package.

    Options:
      --salutation TEXT  Salutation (default: "Hello")
      --who TEXT         Addressee (default: "world")
      --help             Show this help
    """)
end

function main(args)
    salutation = "Hello"
    who = "world"

    i = 1
    while i ⩽ length(args)
        arg = args[i]
        if arg == "--help" || arg == "-h"
            usage()
            return 0
        elseif arg == "--salutation" && i < length(args)
            salutation = args[i + 1]
            i += 1
        elseif arg == "--who" && i < length(args)
            who = args[i + 1]
            i += 1
        else
            println(stderr, "Unknown or incomplete argument: $arg")
            usage(stderr)
```

```
            return 2
        end
        i += 1
    end

    Pkg.activate(PROJECT_ROOT; io = devnull)
    using .Dve  # ERROR: see corrected form below
end

exit(main(ARGS))
```

The `using` statement cannot occur after runtime activation in that position. Use `Base.eval` or, more simply, activate through the command line. The recommended version is:

```
#!/usr/bin/env julia

# scripts/hello.jl
import Pkg

const PROJECT_ROOT = normpath(joinpath(@__DIR__, ".."))

function usage(io::IO = stdout)
    println(io, """
    Usage: hello.jl [--salutation TEXT] [--who TEXT] [--help]

    Print a greeting using the local Dve package.

    Options:
      --salutation TEXT  Salutation (default: "Hello")
      --who TEXT         Addressee (default: "world")
      --help             Show this help
    """)
end

function parse_args(args)
    salutation = "Hello"
    who = "world"
    i = 1
    while i ≤ length(args)
        arg = args[i]
        if arg in ("-h", "--help")
            return nothing
        elseif arg == "--salutation" && i < length(args)
            salutation = args[i + 1]
            i += 1
        elseif arg == "--who" && i < length(args)
```

```
            who = args[i + 1]
            i += 1
        else
            error("unknown or incomplete argument: $arg")
        end
        i += 1
    end
    return salutation, who
end

function main(args)
    parsed = parse_args(args)
    if isnothing(parsed)
        usage()
        return 0
    end
    salutation, who = parsed

    @eval Main using Dve
    println(Main.hello_world(salutation, who))
    return 0
end

if abspath(PROGRAM_FILE) == @__FILE__
    Pkg.activate(PROJECT_ROOT; io = devnull)
    try
        exit(main(ARGS))
    catch err
        println(stderr, "error: ", sprint(showerror, err))
        exit(2)
    end
end
```

The shebang finds `julia` through `PATH`. The script finds the project by resolving `PROJECT_ROOT` relative to its own location, then calls `Pkg.activate(PROJECT_ROOT)`. This is analogous to a Python script using `uv run` or setting `PYTHONPATH`, but the project is selected explicitly rather than inferred from an installed package.

Make it executable and run it:

```
# Working directory: <project-root>
chmod +x scripts/hello.jl
./scripts/hello.jl --salutation "Bonjour" --who "Julia"
```

Expected output:

```
Bonjour, Julia!
```

Unit Test

```
# tests/test_hello_world.jl
using Test
using Dve

@testset "hello_world" begin
    @test hello_world("Hello", "world") == "Hello, world!"
    @test hello_world("  Hi  ", "  Julia  ") == "Hi, Julia!"
    @test_throws MethodError hello_world(1, 2)
end
```

```
# tests/runtests.jl
using Test

include("test_hello_world.jl")
```

Run it:

```
# Working directory: <project-root>
julia --project=. tests/runtests.jl
```

Expected output:

```
Test Summary: | Pass  Total  Time
hello_world   |    3      3  ...
```

Editor Support

`LanguageServer.jl`, `JuliaWorkspaces.jl`, and `SymbolServer.jl` are **components**, not alternatives:

Component	Purpose
`LanguageServer.jl`	Implements the Language Server Protocol wire protocol and editor-facing services. julia-
`JuliaWorkspaces.jl`	The analysis engine: parses files, tracks project environments, and supplies diagnostics, c
`SymbolServer.jl`	A helper that indexes exported names and package metadata without loading packages;

The project documentation describes `JuliaWorkspaces.jl` as the underlying engine for `LanguageServer.jl`, with the long-term goal that `LanguageServer.jl` retains mainly the LSP protocol layer. I found **no evidence that `SymbolServer.jl` is deprecated**; its current documentation describes it as a helper package for `LanguageServer.jl`. The VS Code extension has recently reworked its analysis engine around `JuliaWorkspaces.jl`. julia-vscode

VS Code

Install the **Julia** extension from the VS Code Marketplace. It bundles the language-server tooling; you do not need to add `LanguageServer.jl` to `Project.toml`.

Recommended workspace setting:

```
// .vscode/settings.json
{
  "julia.enableTelemetry": false,
  "julia.execution.resultInCommandLine": true
}
```

Open the repository root in VS Code. The extension should detect `Project.toml` and use that environment for the language server and the integrated REPL. If it selects the wrong environment, run **Julia: Select Environment** and choose `<project-root>/Project.toml`.

Emacs

Two viable clients exist:

Setup	Assessment
`lsp-mode` + `lsp-julia`	Mature and feature-rich; uses `LanguageServer.jl`. Prefer it if you already use `lsp-mod`
`eglot` + `eglot-jl`	Lighter, built into modern Emacs; uses the same Julia language server. Prefer it for a m

```
;; Emacs initialization file
(use-package julia-mode
  :ensure t
  :mode "\\.jl\\'")

;; Option A: lsp-mode
(use-package lsp-julia
  :ensure t
  :after (julia-mode lsp-mode)
  :config
  (setq lsp-julia-default-environment
        (expand-file-name "."))  ; repository root containing Project.toml
  :hook (julia-mode . lsp-deferred))

;; Option B: eglot
(use-package eglot-jl
  :ensure t
  :after (julia-mode eglot)
  :config
  (setq eglot-jl-project-root (expand-file-name "."))
  :hook (julia-mode . eglot-ensure))
```

Use **either** Option A or Option B, not both. The exact customization variable names may differ across package versions; check each package's README before relying on them. [unverified — check `lsp-julia` and `eglot-jl` documentation.]

2. Julia Project Structure

Dependencies and UUIDs

Julia packages are identified by UUIDs, not names. A package may have the same name in different registries or local paths, but its UUID disambiguates it.

```
# Project.toml
[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"

[compat]
DataFrames = "1"
julia = "1.13"
```

- `Project.toml` declares direct dependencies and compatibility bounds. It should be committed.
- `Manifest.toml` records the complete resolved dependency graph, including indirect dependencies and exact versions. Commit it for applications and reproducible research projects; for reusable libraries, committing it is optional but common in research repositories.
- `Pkg.add("DataFrames")` adds a direct dependency and resolves it.
- `Pkg.compat("DataFrames", "1")` sets a compatibility bound.
- `Pkg.develop(path="path/to/LocalPkg")` tracks a local package by path rather than a registry release; it is Julia's analogue of `pip install -e` or R's local installation workflow.

Registries are package catalogs. The default is the General registry; custom registries can be added with `Pkg.Registry.add`.

Test and Development Dependencies

For a package, prefer `[extras]` plus `[targets]`:

```
# Project.toml
[extras]
Aqua = "4c88cf16-eb10-579e-8560-4a9242c79595"
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"

[targets]
test = ["Test", "Aqua"]
```

This keeps test-only packages out of the runtime dependency graph. `test/Project.toml` is an alternative for a separate test environment, but it adds another environment to maintain. For this repository, use `[extras]`/`[targets]` for Julia package tests and a separate scratch environment for ad hoc LSP or analysis tools.

For a one-off tool environment:

```
# Working directory: <project-root>
julia -e 'using Pkg; Pkg.activate("environments/dev"); Pkg.add("Revise")'
julia --project=environments/dev
```

This creates `environments/dev/Project.toml` and `Manifest.toml`, analogous to a `uv` dependency group or a separate `renv` profile.

Source, Tests, and Scripts

Location	Status
`src/Dve.jl`	Required convention; module file must match `name` in `Project.toml`.
`src/*.jl`	Convention only; included from `Dve.jl` with `include`.
`test/runtests.jl`	Julia/Pkg convention for `Pkg.test()`.
`tests/*.jl`	Your repository convention; not understood by `Pkg.test()` without adaptation.
`scripts/*.jl`	Pure convention; Julia has no built-in script directory.
`docs/`, `benchmark/`	Common conventions, not configured in `Project.toml`.

`Project.toml` does **not** specify source or test directories. It specifies the package name, UUID, version, dependencies, compatibility, and optional targets/workspaces.

Pkg Workspaces

A Pkg workspace is a multi-project layout in which several projects share one `Manifest.toml`. It is useful for a repository containing a Julia package plus notebooks, applications, or benchmark environments that should resolve together.

```toml
# Project.toml
name = "Dve"
uuid = "11111111-2222-3333-4444-555555555555"
version = "0.1.0"

[workspace]
projects = [
    "environments/dev",
    "environments/benchmark",
]
```

With this table, the root project and the listed projects form one workspace and share resolution state. This is distinct from:

- An **editor workspace**: the folder or folders opened in VS Code or Emacs.
- **`JuliaWorkspaces.jl`**: the language server's internal representation of source files, projects, and analysis results. julia-vscode

For this repository, start without `[workspace]`. Add it only when you have multiple Julia projects that must resolve consistently.

3. Language Integration Examples

Python/R to Julia Equivalents

Python / R	Julia	Note
`list` / `list`	`Vector`	Homogeneous in Julia; use `Any[]` for mixed types.
`dict` / named `list`	`Dict` or named tuple	`Dict` is mutable; named tuples are immutable.
`float` / double	`Float64`	Julia's default floating-point type.
`int` / integer	`Int64`	Julia integers are fixed-width.
`pandas.DataFrame` / `data.frame`	`DataFrames.DataFrame`	Conversion is typically through the Tables.jl interfa
`numpy.ndarray` / matrix	`Matrix` / `Array`	Layout and copying semantics need care.
`module` / package	module / package	A Julia package contains one or more modules.

Julia to Python

Use `PythonCall.jl`. It is preferred over `PyCall.jl` here because it has an actively maintained symmetric Python counterpart and explicit environment controls. github

```
# Working directory: <project-root>
julia --project=. -e 'using Pkg; Pkg.add("PythonCall")'
```

```
# src/interop.jl
using PythonCall
using DataFrames

function python_roundtrip()
    np = pyimport("numpy")
    x = np.arange(5)

    # Primitive conversion
    n = pyconvert(Int, np.size(x))

    # Project package call
    dve = pyimport("dve")
    greeting = pyconvert(String, dve.hello_world("Hello", "Python"))

    # DataFrame conversion
    df = DataFrame(a = 1:3, b = ["x", "y", "z"])
    pdf = pytable(df)                    # Julia table → pandas.DataFrame
    df2 = DataFrame(PyTable(pdf))        # pandas.DataFrame → Julia table

    return n, greeting, df2
end
```

Include it from the module:

```
# src/Dve.jl
module Dve

export hello_world, python_roundtrip
```

```
include("interop.jl")

function hello_world(salutation::AbstractString, who::AbstractString)
    return "$(strip(salutation)), $(strip(who))!"
end

end # module
```

Conversion semantics:

- `pyimport(...)` returns a `Py` wrapper around a live Python object. It is a reference, not a converted copy.
- `pyconvert(T, x)` converts a Python value to a Julia value of type `T`; this generally materializes a new Julia object.
- `pytable(df)` converts a Tables.jl-compatible Julia table to a pandas DataFrame; `PyTable(pdf)` wraps a pandas DataFrame as a Julia table, and `DataFrame(...)` materializes a Julia `DataFrame`. pypi
- Treat table conversions as copying unless profiling demonstrates otherwise. The exact buffer-sharing behavior depends on column types and conversion path. [unverified — check `PythonCall.jl` conversion reference.]

Environment Binding: Julia to Python

By default, `PythonCall` uses `CondaPkg.jl` and creates a project-specific Conda environment. To use the repository's `uv` environment instead, set:

```
# Working directory: <project-root>
export JULIA_PYTHONCALL_EXE="@venv"
julia --project=. -e 'using Dve; println(Dve.python_roundtrip())'
```

`@venv` tells `PythonCall` to use a `.venv` virtual environment in the active project. Create it with: juliapy.github

```
# Working directory: <project-root>
uv venv
uv pip install -e .
```

If your virtual environment has another name or location, set the executable directly:

```
export JULIA_PYTHONCALL_EXE="$PWD/.venv/bin/python"
```

This avoids duplicate Python installations: Julia calls the interpreter managed by `uv`, and `dvepy` is imported from that environment.

Python to Julia

Use `juliacall`, the Python-side counterpart to `PythonCall`. pypi

```
# Working directory: <project-root>
uv pip install juliacall
```

```
# tests/pytest/test_julia.py
import os
from pathlib import Path

ROOT = Path(__file__).resolve().parents

# [github](https://github.com/JuliaLang/julia/releases)

os.environ["PYTHON_JULIACALL_EXE"] = "julia"
os.environ["PYTHON_JULIACALL_PROJECT"] = str(ROOT)

from juliacall import Main as jl  # noqa: E402


def test_primitives():
    jl.seval("using Statistics")
    result = jl.seval("mean([1, 2, 3])")
    assert float(result) == 2.0


def test_dataframe_roundtrip():
    import pandas as pd

    df = pd.DataFrame({"a": [1, 2, 3], "b": ["x", "y", "z"]})

    jl.seval("using DataFrames")
    jl_df = jl.DataFrame(df)  # pandas → Julia DataFrame
    assert jl_df.ncol == 2

    pdf = jl.pytable(jl_df)   # Julia table → pandas DataFrame
    assert isinstance(pdf, pd.DataFrame)
    assert list(pdf.columns) == ["a", "b"]


def test_dve_package():
    jl.seval("using Dve")
    greeting = str(jl.Dve.hello_world("Hello", "Python"))
    assert greeting == "Hello, Python!"
```

Run:

```
# Working directory: <project-root>
uv run pytest tests/pytest/test_julia.py
```

Expected output:

```
3 passed
```

Conversion semantics:

- `jl.DataFrame(df)` converts a pandas DataFrame into a Julia `DataFrame` through the Tables.jl-compatible interface. pypi
- `jl.pytable(jl_df)` converts a Julia table into a pandas DataFrame. pypi
- `jl.seval("...")` evaluates Julia source in the embedded Julia session.
- `jl.Dve` accesses the `Dve` module after `using Dve`.
- Conversions of data frames materialize Julia or Python objects; do not assume zero-copy behavior. [unverified — check `juliacall` conversion documentation.]

Environment Binding: Python to Julia

`juliacall` uses `PYTHON_JULIACALL_EXE` and `PYTHON_JULIACALL_PROJECT`, and the documentation states that these two options must be set together. Set them before importing `juliacall`: github

```
# Working directory: <project-root>
export PYTHON_JULIACALL_EXE="$(command -v julia)"
export PYTHON_JULIACALL_PROJECT="$PWD"
uv run pytest tests/pytest/test_julia.py
```

This points Python at the system Julia binary and at the repository's Julia project, so `Dve`, `DataFrames`, and `Statistics` resolve from the same `Project.toml`/`Manifest.toml` used by R and Julia scripts.

R to Julia

Use `JuliaCall`. It is available on CRAN and provides high-level Julia calls with automatic conversion for primitive types. juliainterop.github

```
# R/interop.R
dve_julia_setup <- function(project_root = system.file(package = "dvesimpler")) {
  stopifnot(requireNamespace("JuliaCall", quietly = TRUE))

  JuliaCall::julia_setup(
    JULIA_HOME = Sys.getenv("JULIA_HOME", unset = ""),
    installJulia = FALSE
  )

  # Activate the repository's Julia project.
  JuliaCall::julia_command(
    sprintf('using Pkg; Pkg.activate(%s)', deparse(project_root))
  )

  invisible(TRUE)
}
```

```r
dve_hello <- function(salutation = "Hello", who = "R") {
  dve_julia_setup()
  JuliaCall::julia_call("hello_world", salutation, who)
}

dve_summarize <- function(df) {
  dve_julia_setup()

  # Pass primitive columns explicitly; construct the Julia DataFrame in Julia.
  JuliaCall::julia_command("using DataFrames")
  JuliaCall::julia_assign("r_a", df$a)
  JuliaCall::julia_assign("r_b", as.character(df$b))

  JuliaCall::julia_eval(
    'DataFrame(a = r_a, b = r_b)'
  )
}
```

```r
# tests/testthat/test-julia.R
test_that("R calls Julia primitives", {
  skip_if_not_installed("JuliaCall")
  expect_equal(dve_hello("Hello", "R"), "Hello, R!")
})

test_that("R data.frame columns become a Julia DataFrame", {
  skip_if_not_installed("JuliaCall")
  df <- data.frame(a = 1:3, b = c("x", "y", "z"))
  out <- dve_summarize(df)
  expect_s3_class(out, "data.frame")
  expect_equal(nrow(out), 3L)
})
```

Run:

```r
# Working directory: <project-root>
devtools::test()
```

Expected output:

```
✔ | 2 tests passed
```

Conversion semantics:

- Scalars, vectors, and character vectors are converted automatically by `JuliaCall`. juliainterop.github
- An R `data.frame` is **not** guaranteed to become a `DataFrames.DataFrame` automatically. The example avoids relying on that behavior: it passes columns as vectors and constructs the

Julia `DataFrame` explicitly. [unverified — check `JuliaCall` conversion documentation for your installed version.]

- The returned Julia `DataFrame` is converted back to an R data frame by `JuliaCall`; verify column types and names in tests.

Environment Binding: R to Julia

`JuliaCall` finds Julia through, in order, the `JULIA_HOME` global option, the `JULIA_HOME` environment variable, and then `julia` on `PATH`. To avoid a second Julia installation, ensure the same `julia` executable used elsewhere is on `PATH`: rdocumentation

```
# Working directory: <project-root>
export JULIA_HOME="$(dirname "$(command -v julia)")"
Rscript -e 'devtools::test()'
```

`renv` manages R packages, while JuliaCall manages the embedded Julia runtime and Julia packages through Julia's own project environment. The R project activates the Julia project explicitly with `Pkg.activate(project_root)`, so both languages share the repository's Julia dependency manifest.

First-Run Bootstrapping

```
# Working directory: <project-root>

# 1. Python environment
uv sync
uv pip install juliacall

# 2. Julia environment
julia --project=. -e 'using Pkg; Pkg.instantiate()'

# 3. R environment
Rscript -e 'renv::restore()'

# 4. Verify all three directions
julia --project=. tests/runtests.jl
uv run pytest tests/pytest/test_julia.py
Rscript -e 'devtools::test()'
```

Acceptance Tests

- `julia --project=. -e 'using Dve; println(Dve.hello_world("Hello", "Julia"))'` prints `Hello, Julia!`.
- `julia --project=. tests/runtests.jl` passes the `hello_world` tests.
- `./scripts/hello.jl --help` prints usage; `./scripts/hello.jl --salutation Hi --who Julia` prints `Hi, Julia!`.

- `julia --project=. -e 'using Dve; Dve.python_roundtrip()'` returns an integer, a greeting string, and a three-row `DataFrame`.
- `uv run pytest tests/pytest/test_julia.py` passes three tests.
- `Rscript -e 'devtools::test()'` passes both JuliaCall tests.
- VS Code reports no unresolved `using Dve` import when the repository root is open.
- A fresh clone passes all commands after `uv sync`, `Pkg.instantiate()`, and `renv::restore()`.

Common Pitfalls

- Editing `Project.toml` UUIDs by hand. Generate them with `UUIDs.uuid4()` and let `Pkg.add` manage dependency UUIDs.
- Installing `IJulia` into the project environment. Keep notebook infrastructure in a global or dedicated environment.
- Calling `using Dve` before `Pkg.activate` has taken effect in a script. Activate first, then import.
- Assuming `Pkg.test()` works with `tests/runtests.jl`. It expects `test/runtests.jl` unless you adapt the layout.
- Assuming R `data.frame` □ Julia `DataFrame` conversion is automatic. Convert columns explicitly or test the installed `JuliaCall` behavior.
- Mixing `PyCall` and `PythonCall` in one Julia session. Choose one Python bridge per project.
- Letting `PythonCall` create a Conda environment when the repository already uses `uv`. Set `JULIA_PYTHONCALL_EXE=@venv` or an explicit interpreter path. juliapy.github
- Setting only one of `PYTHON_JULIACALL_EXE` and `PYTHON_JULIACALL_PROJECT`. The documentation requires both together. github
- Forgetting that Julia `Vectors` are homogeneous. Use concrete element types where possible; `Any[]` sacrifices performance and type stability.

Additional Notes

- For CRAN compliance, add Julia-only files and directories to `.Rbuildignore`, for example `scripts/`, `environments/`, `Manifest.toml`, `Project.toml`, `src/*.jl`, and `tests/*.jl`, while retaining the R package sources under `R/` and `src/`.
- Keep `Manifest.toml` under version control for reproducibility, but ensure CRAN submission excludes it through `.Rbuildignore` if it is not needed in the submitted tarball.
- Prefer `PythonCall/juliacall` for new work; `PyCall` remains widely used but is a separate bridge with its own Python-discovery mechanism. github
- For performance-sensitive numerical exchange, prefer Arrow/Parquet files or explicit arrays over repeated data-frame conversion.

Unverified Items

- Exact UUIDs for `DataFrames`, `PythonCall`, `Statistics`, and `Test`: verify with `Pkg.add` or the General registry before committing `Project.toml`.
- Automatic R `data.frame` → `DataFrames.DataFrame` conversion in `JuliaCall`: verify against the installed version's documentation. juliainterop.github

- Exact zero-copy semantics for array and data-frame conversions in `PythonCall/juliacall`: verify in the conversion reference. juliapy.github
- Exact Emacs variable names for `lsp-julia` and `eglot-jl`: verify in each package's current README.
- `IJulia` kernel name and `--project=@.` behavior across recent IJulia versions: verify in the IJulia documentation. [unverified]

References

- ap-julia-release: Julia v1.13.1 release
- ap-julia-news: Julia v1.13 release notes
- ap-pythoncall: PythonCall & JuliaCall guide
- ap-pythoncall-compat: PythonCall compatibility tools
- ap-juliacall: juliacall on PyPI
- ap-juliacall-env: juliacall environment documentation
- ap-juliacall-r: JuliaCall for R
- ap-juliacall-cran: JuliaCall CRAN reference
- ap-languageserver: JuliaWorkspaces.jl
- ap-symbolserver: SymbolServer.jl
- ap-vscode: Julia extension for VS Code
- ap-dataframes: DataFrames.jl getting started

A:1 (Vibe)

^ *Cross Language Integration Bridge*

- see: Vibe

Proposed repository tree

```
<project-root>
├── DESCRIPTION                      # R "dvesimpler" package metadata
├── pyproject.toml                   # Python project (uv)
├── Project.toml                     # Julia package "Dve" (root project + workspace base)
├── Manifest.toml                    # Julia manifest, shared by the workspace
├── uv.lock / renv.lock
├── .Rbuildignore                    # add Julia entries (see below)
├── R/                               # R package sources
├── inst/include, src/Makevars, src/*.cpp   # Rcpp layer
├── src/Dve.jl                       # Julia Dve main module (convention: src/<name>.jl)
├── src/*.jl                         # other Julia sources
├── test/                            # NEW (Julia convention) — add to .Rbuildignore
│   ├── Project.toml                 # test environment (workspace member)
│   └── runtests.jl                  # shim that includes ../tests/*.jl
├── tests/
│   ├── hello_world_test.jl          # Julia test sources (Pkg.test sees them via the shim)
│   ├── pytest/
│   │   └── test_julia_integration.py
│   └── testthat/
├── scripts/
│   ├── hello.jl                     # executable Julia front end
│   ├── r2julia-demo.R
│   └── j2python-demo.jl
└── inst/python/dve/__init__.py      # Python "dvepy.dve" module
```

Corrections to the assumed layout

Two assumptions in the brief need adjusting; both keep CRAN compliance:

- Julia's `Pkg.test()` runs exactly `test/runtests.jl` (singular, relative to the package root). This is a fixed convention — there is no `Project.toml` key to relocate it. Since the repo standardizes on `tests/` (shared with `testthat/` and `pytest/`), add a two-line shim `test/runtests.jl` that includes the real test files from `tests/`, and add `test/` to `.Rbuildignore`.

- `src/` is shared with Rcpp. That works, but every `.jl` file in it must be excluded from the R build. `Makevars` compiles `*.cpp` only, so the files are inert for R; still keep them out of the tarball.

`.Rbuildignore` additions:

```
# .Rbuildignore (Julia entries)
^Project\.toml$
^Manifest(-v[0-9.]+)?\.toml$
^src/.*\.jl$
^test$
^scripts$
^\.gitignore$  # (already present, kept for illustration — remove if duplicate)
```

Python–R–Julia equivalents

Python (uv)	R (renv)	Julia (Pkg)
`pyproject.toml`	`DESCRIPTION`	`Project.toml`
`uv.lock`	`renv.lock`	`Manifest.toml`
`uv sync`	`renv::restore()`	`Pkg.instantiate()`
`uv add pkg`	`install.packages("pkg")`	`Pkg.add("pkg")`
virtualenv (activated)	renv project library	active environment (`--project=`, `JULIA_PROJECT`)
dev-dependency group	`Suggests:`	`test/Project.toml` (workspace member)
PyPI	CRAN	General registry
`import module`	`library(pkg)`	`using Pkg` / `import Pkg`
pytest	testthat	`@testset` via `Pkg.test()`

Objective 1 — Julia project skeleton

`Project.toml` and the `Dve` module

Generate the UUID once with `import UUIDs; UUIDs.uuid4()` (or `pkg> generate Dve`) — never invent one by hand:

```toml
# Project.toml
name = "Dve"
uuid = "4f81d0a5-2f9d-487d-8fe0-4f9f77b1ac2b"  # replace with your own UUIDs.uuid4()
version = "0.1.0"
authors = ["Giovanni Pelosi <giovanni@example.org>"]

[deps]
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
PythonCall = "6099a3de-0909-5bc5-a295-3f136d948b72"

[compat]
DataFrames = "1"
```

```
PythonCall = "0.9"
julia = "1.13"

[workspace]
projects = ["test"]
```

`DataFrames` and `PythonCall` are declared because `Dve` is the project's cross-language integration layer (they are used in Objective 3). The `[workspace]` table is explained in Objective 2.

```
# src/Dve.jl
module Dve

export hello_world

"""
    hello_world(salutation, who) → String

Return the greeting `"<salutation>, <who>!"`.

Equivalent to the Python `"f"{salutation}, {who}!"` or the R
`paste0(salutation, ", ", who, "!")`.
"""
hello_world(salutation::AbstractString, who::AbstractString) =
    string(salutation, ", ", who, "!")

end # module
```

Notes for the Python/R reader:

- `module Dve` must match the `name` field, and the file must be `src/Dve.jl`. This is enforced convention, not optional[32].
- `string(a, b, c)` is the non-vectorized analogue of `paste0`; unlike R, there is no implicit recycling — arguments are exact.
- `AbstractString` plays the role of R's `character` / Python's `str` duck-typing: it accepts any string type.

Instantiate and smoke-test:

```
$ julia --project=. -e 'using Pkg; Pkg.instantiate(); using Dve; println(Dve.hello_world("Hello"
Hello, Julia!
```

Jupyter kernel

`IJulia` is a dependency of the *editor environment*, not of your package, so install it in the default environment (`@v1.13`), not in `Dve`:

[32] Pkg.jl guide, *Creating packages* — https://pkgdocs.julialang.org/v1/creating-packages/ (test-specific dependencies, workspaces with `test/Project.toml`, `pkg> test` convention)

```
$ julia -e 'using Pkg; Pkg.add("IJulia")'
$ julia --project=. -e 'using IJulia; IJulia.installkernel("Julia-dve", "--project=$(Base.active
Installing Julia kernelspec in ~/.local/share/jupyter/kernels/julia-dve-1.13
```

How the kernel binds to the project environment: `installkernel` writes a Jupyter *kernelspec* whose launch command embeds the extra arguments verbatim[33]. Passing `--project=$(Base.active_project())` freezes the absolute path of the repository's `Project.toml` into the spec, so every notebook started with the `Julia-dve` kernel activates that environment — the Julia analogue of an RStudio session using the renv library, or a Jupyter kernel launched inside an activated venv. The binding happens at install time: re-run `installkernel` if you move or rename the repository. `IJulia` respects `JUPYTER_DATA_DIR` if you want the spec installed elsewhere[34].

Executable script

```
#!/usr/bin/env -S julia --startup-file=no
# scripts/hello.jl — command-line front end for Dve.hello_world.
#
# How the project environment is found: the shebang only launches Julia with
# reasonable flags (env -S is needed to pass more than one argument on Linux;
# plain "#!/usr/bin/env julia" also works). The environment is then activated
# *relative to this file* via Pkg.activate(@__DIR__ "..") so the script runs from
# any CWD — unlike `--project=.` in a shebang, which would depend on where the
# caller stands. (Julia also honours the JULIA_PROJECT environment variable.)

import Pkg
Pkg.activate(joinpath(@__DIR__, ".."); io = devnull)

using Dve

function usage()
    println("""
    usage: hello.jl [-s SALUTATION] [-w WHO] [-h]

    Prints Dve.hello_world(salutation, who).

    options:
      -s, --salutation SALUTATION   salutation (default: "Hello")
      -w, --who WHO                 recipient (default: "world")
      -h, --help                    show this help and exit""")
end

function main(args)
```

[33] IJulia documentation, *Usage* — https://julialang.github.io/IJulia.jl/stable/manual/usage/ (installkernel with `--project`)

[34] IJulia documentation, *Installation* — https://julialang.github.io/IJulia.jl/stable/manual/installation/ (kernelspecs, `JUPYTER_DATA_DIR`)

```
    salutation, who = "Hello", "world"
    i = firstindex(args)
    while i ⩽ lastindex(args)
        a = args[i]
        if a == "-h" || a == "--help"
            usage(); return
        elseif a == "-s" || a == "--salutation"
            salutation = args[i += 1]
        elseif a == "-w" || a == "--who"
            who = args[i += 1]
        else
            error("unknown argument: $a (see --help)")
        end
        i += 1
    end
    println(Dve.hello_world(salutation, who))
end

abspath(PROGRAM_FILE) == @__FILE__ && main(ARGS)
```

The hand-rolled parser keeps the skeleton dependency-free. When requirements grow, compare the alternatives:

Option	Purpose	Maintenance	Setup effort
manual `ARGS` parsing	tiny CLIs, zero deps	you own it	none
`ArgParse.jl`	argparse-style CLIs	mature, stable	one dep
`Comonicon.jl`	full app framework (subcommands, tab completion)	actively developed	medium

```
$ chmod +x scripts/hello.jl
$ ./scripts/hello.jl --help
usage: hello.jl [-s SALUTATION] [-w WHO] [-h]
...
$ ./scripts/hello.jl -s Ciao -w Giovanni
Ciao, Giovanni!
```

Unit test

`Pkg.test()` runs `test/runtests.jl` in a fresh process with the test environment active:

```
# test/runtests.jl — thin shim so the real tests can live in tests/ (repo standard)
include(joinpath(@__DIR__, "..", "tests", "hello_world_test.jl"))

# tests/hello_world_test.jl
using Test
using Dve
```

```
@testset "hello_world" begin
    @test Dve.hello_world("Hello", "Julia") == "Hello, Julia!"
    @test Dve.hello_world("Ciao", "Giovanni") == "Ciao, Giovanni!"
    @test_throws MethodError Dve.hello_world(1.0, 2.0)  # typed API: no implicit coercion
end
```

```
# test/Project.toml — test environment, a member of the workspace (see Objective 2)
[deps]
Dve = "4f81d0a5-2f9d-487d-8fe0-4f9f77b1ac2b"
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"

[sources]
Dve = {path = ".."}
```

```
$ julia --project=. -e 'using Pkg; Pkg.test()'
     Testing Dve
      Status `~/repo/Manifest.toml`
Test Summary: | Pass  Total  Time
hello_world   |    2      2  0.0s
     Testing Dve tests passed
```

The command that builds `test/Project.toml` from scratch:

```
$ julia --project=. -e 'using Pkg; Pkg.activate("test"); Pkg.develop(path=abspath(".")); Pkg.add
```

LSP support

VS Code

Install the extension `julialang.language-julia`. Nothing goes into your `Project.toml`: on first use the extension installs `LanguageServer.jl` itself into an isolated environment in the Julia depot (`~/.julia`), and lets you pick the interpreter (juliaup channels appear automatically). Open the repository as the workspace folder; the server discovers the root `Project.toml` (and considers `JULIA_PROJECT`), giving diagnostics, completion, jump-to-definition and inline results against the *package's own* manifest[35].

`LanguageServer`, `JuliaWorkspaces`, `SymbolServer`: components, not alternatives

They are parts of one system, and their relationship changed in 2026[36]:

- `LanguageServer.jl` is the LSP *server* itself (currently v6.0.x). It speaks the protocol with any editor.
- `SymbolServer.jl` was, up to LanguageServer v4, the component that indexed the symbols of your dependencies from the depot into a cache; `StaticLint.jl` was the linter.

[35] Julia VS Code extension documentation — https://www.julia-vscode.org/docs/stable/

[36] LanguageServer.jl releases (v5.0.0 Salsa/JuliaWorkspaces rewrite; v6.0.0 "Integrate StaticLint and SymbolServer into JuliaWorkspaces") — https://github.com/julia-vscode/LanguageServer.jl/releases ; SymbolServer.jl — https://github.com/julia-vscode/SymbolServer.jl ; JuliaWorkspaces.jl — https://github.com/julia-vscode/JuliaWorkspaces.jl

- `JuliaWorkspaces.jl` is the internal architecture that replaced both: LanguageServer v5 (March 2026) was rewritten on a Salsa-based `JuliaWorkspaces.jl`, which models the "workspace" — every project environment and file visible to the server — and v6.0.0 (September 2026) folded `SymbolServer` and `StaticLint` into `JuliaWorkspaces`. So today `SymbolServer.jl` is *legacy*: still on GitHub, no longer a LanguageServer dependency. This is a verified deprecation, not a rumour[37].

Practical consequence: the editor's *workspace folders* are mapped by LanguageServer onto Julia environments; keep the repo root as the workspace folder so the root `Project.toml` (and the `test` workspace member) are picked up. First start is slow (server downloads + precompiles); subsequent starts are cached.

Emacs

Client	Purpose
`eglot` + `eglot-jl`	LSP client built into Emacs 29+, plus glue that registers LanguageServer.jl and pre-inst
`lsp-mode` + `lsp-julia`	full LSP suite (UI, treemacs, modeline) plus glue that launches LanguageServer.jl

The LanguageServer wiki documents both clients[38]. Current `use-package` configuration for the recommended pair:

```
;; init.el — Julia LSP via eglot + eglot-jl
(use-package julia-mode           ; or julia-ts-mode on Emacs 29+
  :ensure t)

(use-package eglot-jl
  :ensure t
  :hook (julia-mode . (lambda ()
                        (eglot-jl-init)          ; register the server + project.el support
                        (eglot)))               ; start it for this buffer's project
  :config
  ;; First run downloads and precompiles LanguageServer.jl into ~/.julia:
  ;; allow plenty of time, watch *EGLOT ... stderr* for progress.
  (setq eglot-connect-timeout 600))
```

`eglot-jl` starts a server *for the Julia project the buffer belongs to* (it honours the buffer's project root), so editing `src/Dve.jl` and `tests/*.jl` resolves against the right `Project.toml` automatically[39].

[37] LanguageServer.jl releases (v5.0.0 Salsa/JuliaWorkspaces rewrite; v6.0.0 "Integrate StaticLint and SymbolServer into JuliaWorkspaces") — https://github.com/julia-vscode/LanguageServer.jl/releases ; SymbolServer.jl — https://github.com/julia-vscode/SymbolServer.jl ; JuliaWorkspaces.jl — https://github.com/julia-vscode/JuliaWorkspaces.jl

[38] LanguageServer.jl wiki, *Emacs* — https://github.com/julia-vscode/LanguageServer.jl/wiki/Emacs

[39] eglot-jl (README: `eglot-jl-init`, first-run install into the depot, timeout advice) — https://github.com/non-Jedi/eglot-jl ; lsp-julia page — https://emacs-lsp.github.io/lsp-mode/page/lsp-julia/

Objective 2 — Julia project structure

Dependency management and UUIDs: is `Pkg.add` enough?

Yes for registered dependencies: `Pkg.add("DataFrames")` resolves the *name* □ *UUID* □ *version* chain through the registries in the depot (the General registry is cloned automatically on first use; private registries via `Pkg.Registry.add`), and writes both `Project.toml` (`[deps]`) and `Manifest.toml`. A package's identity in Julia is the *(name, UUID)* pair, not its path — the UUID is why two packages with the same name can coexist across registries[40].

`Pkg.add` is *not* enough in three cases:

- *Your own package*: the UUID must be minted by you (`UUIDs.uuid4()` / `pkg> generate`).
- *Unregistered/local dependencies*: use `Pkg.develop(path=...)` (the `dev` of the Julia world), or since Julia 1.11 a `[sources]` entry (`SomeDependency = {path = "deps/SomeDependency.jl"}`), which `dev` and `add <url>` write for you.
- *Version bounds*: `Pkg.add` never writes `[compat]`; add bounds yourself, including `julia = "1.13"`. `[compat]` constrains resolution and is checked by the ecosystem's registration bot if you ever publish.

`Project.toml` (direct deps, compat, identity) is the `pyproject.toml`; `Manifest.toml` (the fully resolved graph, exact versions, machine-managed — do not hand-edit) is the `uv.lock`/`renv.lock`. Commit the manifest in this repository: it is a research *application*, and reproducibility of the analysis outranks library etiquette (libraries published to the General registry traditionally do not commit manifests). Since Julia 1.11 you may also name it `Manifest-v1.13.toml` to pin manifests per minor release[41].

Dev, test and LSP dependencies

Mechanism	Purpose
`[extras]` + `[targets test]` in root `Project.toml`	test-only deps (old style)
`test/Project.toml` alone (with `[sources] path = ".."`)	test-only deps with a separate mani
workspaces (`[workspace] projects = ["test"]` + `test/Project.toml`)	optional deps resolved together wit
default environment `@v1.13` for REPL tools	Revise, BenchmarkTools, IJulia…
depot environment managed by the editor	LanguageServer.jl

The workspace approach is recommended because the resolver sees the package and its test environment *together* and records a single shared `Manifest.toml` next to the root project — one lock file for the whole repo, consistent with how `uv.lock` and `renv.lock` each cover their toolchain. Note that

[40] Pkg.jl guide, *Project.toml and Manifest.toml* — https://pkgdocs.julialang.org/v1/toml-files/ (UUID semantics, `[compat]`, `[sources]`, `[workspace]`, `[extras]` legacy, version-specific manifests)

[41] Pkg.jl guide, *Project.toml and Manifest.toml* — https://pkgdocs.julialang.org/v1/toml-files/ (UUID semantics, `[compat]`, `[sources]`, `[workspace]`, `[extras]` legacy, version-specific manifests)

[42] Pkg.jl guide, *Project.toml and Manifest.toml* — https://pkgdocs.julialang.org/v1/toml-files/ (UUID semantics, `[compat]`, `[sources]`, `[workspace]`, `[extras]` legacy, version-specific manifests)

[43] Pkg.jl guide, *Creating packages* — https://pkgdocs.julialang.org/v1/creating-packages/ (test-specific dependencies, workspaces with `test/Project.toml`, `pkg> test` convention)

workspace members do not inherit the root package's `[deps]`: anything used directly in tests must appear in `test/Project.toml`, including `Dve` itself (via `Pkg.develop/[sources] path = ".."`)[44].

Dev tooling goes in the default environment because Julia *stacks* environments: the active project is layered on `@v#.#` and `@stdlib`, so `using Revise` works on top of any `--project` without polluting the package — this replaces the notion of a "dev extra" group in `pyproject.toml`.

How source, test and script folders are determined

- `src/Dve.jl` — *enforced convention*: for a package named `Dve`, the module must be `Dve` in `src/Dve.jl`. Splitting into `src/*.jl` and `include(...)` lines is your code's business; Pkg never looks at other files.
- `test/runtests.jl` — the single entry point `Pkg.test()` executes. Fixed; not settable in `Project.toml` (hence the shim in Objective 1).
- `scripts/`, `docs/`, `benchmarks/` — pure convention, invisible to Pkg. A script is "outside the package" because nothing imports it; you may give scripts their own environment by adding `"scripts"` to `[workspace] projects` if they need deps the package does not have.

Workspaces

Three distinct meanings share this word; disambiguate them once:

- *Pkg workspaces* — the `[workspace]` table just used: a set of projects (paths relative to the base project) resolved jointly, sharing one manifest at the base. Use cases: test/docs/benchmark environments inside one package, and monorepos with several Julia packages (each keeps its own `Project.toml`, one manifest for all). Minimal example: the `[workspace] projects = ["test"]` + `test/Project.toml` pair above[45].
- *`JuliaWorkspaces.jl`* — an internal library of LanguageServer.jl (Objective 1); despite the name it is unrelated to Pkg workspaces — it is the server's model of which environments/files it watches.
- *Editor workspaces* — the folders your editor opens. They matter because they determine which `Project.toml` files the language server discovers.

Objective 3 — Language integration

Package choices and justification

Direction	Recommended	Alternatives
R → Julia	`JuliaConnectoR` (CRAN 1.1.6, Sep 2026, JSS paper)[46]	`JuliaCall` (CRAN, embeds libjulia int
Julia → Python	`PythonCall` (Julia side of the JuliaPy pair)[48]	`PyCall` (the older, mature interface; a
Python → Julia	`juliacall` 0.9.36 (PyPI, Sep 2026)[50]	`pyjulia` (PyCall-based, development

[44] Pkg.jl guide, *Creating packages* — https://pkgdocs.julialang.org/v1/creating-packages/ (test-specific dependencies, workspaces with `test/Project.toml`, `pkg> test` convention)

[45] Pkg.jl guide, *Project.toml and Manifest.toml* — https://pkgdocs.julialang.org/v1/toml-files/ (UUID semantics, `[compat]`, `[sources]`, `[workspace]`, `[extras]` legacy, version-specific manifests)

[46] JuliaConnectoR repository — https://github.com/stefan-m-lenz/JuliaConnectoR

Use the *pair* PythonCall/juliacall on both Python-facing bridges so the conversion rules are the same in both directions.

R to Julia (JuliaConnectoR)

Environment discovery: `JuliaConnectoR` finds the Julia binary through the system `PATH`, through `juliaup`, or via `JULIA_BINDIR`[52]. The Julia *environment* is whatever Julia would activate — so setting `JULIA_PROJECT` to the repo root before loading the package makes the spawned Julia process use the repository's `Project.toml`/`Manifest.toml` directly. No Julia packages are duplicated: Julia reads the same depot (`~/.julia/packages`) and the same manifest as the Julia-native workflow. renv only manages the R side (`JuliaConnectoR` belongs in `Suggests:`).

First-run bootstrapping:

```
$ juliaup add 1.13.1 && juliaup default 1.13.1
$ julia --project=. -e 'using Pkg; Pkg.instantiate()'   # downloads Julia deps once
$ Rscript -e 'renv::restore()'                          # installs JuliaConnectoR (Suggests)
```

```
# scripts/r2julia-demo.R
## Make the spawned Julia process activate the repo environment.
Sys.setenv(JULIA_PROJECT = normalizePath("."))    # must run before library(JuliaConnectoR)
library(JuliaConnectoR)
stopifnot(juliaSetupOk())

## --- primitives ------------------------------------------------------
## R length-1 vectors map to Julia scalars (double → Float64, integer → Int,
## logical → Bool, character → String) and back:
juliaCall("sqrt", 2.0)                 #> [1] 1.414214
Dve <- juliaImport("Dve")               # import the project's own package
Dve$hello_world("Hello", "Julia")      #> [1] "Hello, Julia!"

## --- data frames -----------------------------------------------------
planets <- data.frame(
  name    = c("Earth", "Mars"),
  gravity = c(9.807, 3.721),
  stringsAsFactors = FALSE)

DataFrames <- juliaImport("DataFrames")              # third-party Julia package
```

[47] JuliaCall (R) — https://cran.r-project.org/package=JuliaCall

[48] PythonCall guide (environment variables `JULIA_PYTHONCALL_EXE`, `JULIA_CONDAPKG_BACKEND`; preferences) — https://juliapy.github.io/PythonCall.jl/stable/pythoncall/ and FAQ — https://juliapy.github.io/PythonCall.jl/stable/faq/

[49] PyCall.jl — https://github.com/JuliaPy/PyCall.jl

[50] juliacall on PyPI (non-copying numeric-array conversion both directions; version/requirements) — https://pypi.org/project/juliacall/ ; juliacall reference — https://juliapy.github.io/PythonCall.jl/stable/juliacall/

[51] pyjuliapkg (functional API, `juliapkg.json`, Julia discovery order, `PYTHON_JULIAPKG_EXE`/`PYTHON_JULIAPKG_PROJECT`) — https://github.com/JuliaPy/pyjuliapkg

[52] JuliaConnectoR package documentation (setup, `JULIA_BINDIR`, type-translation tables, proxies, `as.data.frame.JuliaProxy`) — https://rdrr.io/cran/JuliaConnectoR/man/JuliaConnectoR-package.html ; Lenz, Hackenberg & Binder (2022), *Journal of Statistical Software* 101(6) — https://doi.org/10.18637/jss.v101.i06

```r
out <- juliaCall("DataFrames.DataFrame", planets)    # Tables-compatible object → DataFrame
class(out)                                           #> "JuliaProxyOf<DataFrames.DataFrame>" ...
res <- as.data.frame(out)                            # Tables proxy → R data.frame
res                                                  #>    name gravity
                                                     #> 1 Earth   9.807
                                                     #> 2  Mars   3.721

stopJulia()
```

How the conversion works[53]: every value crosses the boundary by *serialization over a local TCP connection* — all primitives and arrays are **copies** by construction; there is no shared memory. An R `data.frame` is translated into an object implementing the Julia `Tables` interface, which `DataFrames.DataFrame` accepts directly. In the other direction, structs such as `DataFrame` are transferred *by reference* as `JuliaProxy` objects (mutable access via `[`, `[[`, `$`), and `as.data.frame()` / `juliaGet()` copy the contents back into R. This is deliberately different from R's usual by-value illusion: proxies give you Julia-side mutation from R.

In `tests/testthat/`, wrap any Julia test with `skip_on_cran()` (CRAN has no Julia) — same discipline as `NOT_CRAN` examples.

Julia to Python (PythonCall)

Environment discovery without duplicating the Python install: PythonCall by default builds a private Conda environment per project via `CondaPkg`. Disable that and pin it to the *uv-managed* interpreter with two environment variables[54]:

```
$ uv sync                                              # creates .venv with dvepy + numpy + panda
$ export JULIA_CONDAPKG_BACKEND=Null                    # do not manage Python via CondaPkg
$ export JULIA_PYTHONCALL_EXE="$(uv run python -c 'import sys; print(sys.executable)')"
$ julia --project=. scripts/j2python-demo.jl
```

(Preferences can make this permanent per-machine: `pkg> preference add CondaPkg backend=Null` then `preference add PythonCall exe=/path/to/python`[55].)

```
# scripts/j2python-demo.jl
using PythonCall                # from the repo environment
using DataFrames

const np = pyimport("numpy")

## --- primitives ------------------------------------------------------------
```

[53] JuliaConnectoR package documentation (setup, `JULIA_BINDIR`, type-translation tables, proxies, `as.data.frame.JuliaProxy`) — https://rdrr.io/cran/JuliaConnectoR/man/JuliaConnectoR-package.html ; Lenz, Hackenberg & Binder (2022), *Journal of Statistical Software* 101(6) — https://doi.org/10.18637/jss.v101.i06

[54] PythonCall guide (environment variables `JULIA_PYTHONCALL_EXE`, `JULIA_CONDAPKG_BACKEND`; preferences) — https://juliapy.github.io/PythonCall.jl/stable/pythoncall/ and FAQ — https://juliapy.github.io/PythonCall.jl/stable/faq/

[55] PythonCall guide (environment variables `JULIA_PYTHONCALL_EXE`, `JULIA_CONDAPKG_BACKEND`; preferences) — https://juliapy.github.io/PythonCall.jl/stable/pythoncall/ and FAQ — https://juliapy.github.io/PythonCall.jl/stable/faq/

```
r = np.sqrt(2.0)                  # returns a Py object: NO conversion, NO copy
pyconvert(Float64, r)              # → 1.4142135623730951  (explicit materialization)
a = np.asarray([1.0, 2.0, 3.0])
v = pyconvert(Vector{Float64}, a)  # explicit conversion → new Julia Vector (copy)
sum(v)                            # → 6.0

## --- DataFrames <→> pandas ---------------------------------------------
df = DataFrame(name = ["Earth", "Mars"], gravity = [9.807, 3.721])
pdf = PythonCall.pytable(df)          # returns a Py object that *is* a pandas.DataFrame
                                    # (format=:pandas is the default)
println(pdf.shape)                    # → (2, 2)  (pandas attribute access from Julia)
df2 = DataFrame(PythonCall.PyTable(pdf))   # pandas → PyPandasDataFrame → Julia DataFrame
println(df2)                          # 2×2 DataFrame; gravity Float64 column

## --- third-party and project Python packages ---------------------------
println(pyconvert(String, np.version.__version__))
dve = pyimport("dvepy.dve")            # the project's own package, importable because the
println(pyconvert(String,              # uv environment has it installed (uv sync)
    dve.hello_world("Hello", "from Julia")))  # → "Hello, from Julia!"
```

How the conversion works[56]: PythonCall's philosophy is *wrap, don't copy*. Mutable Python objects cross into Julia as `Py` wrappers (method calls, attributes — no conversion, zero copy); immutable results are also left as `Py` until you call `pyconvert`, which materializes Julia values (**copies**). Numeric arrays can be wrapped zero-copy in both directions (`PyArray` gives a Julia `AbstractArray` view of a `numpy` array). For tables: `pytable(df)` hands the Julia columns to the pandas constructor as Python-visible (wrapped, non-copying) arrays — treat the resulting DataFrame as a new table and avoid mutating shared columns; `PyTable(pdf)` is a lazy zero-copy wrapper (`PyPandasDataFrame`) over the pandas object, and `DataFrame(...)` materializes new Julia columns from it. This is the exact opposite of JuliaConnectoR's always-copy serialization, and of PyCall's old convert-everything default — pick semantics consciously per bridge.

First run is slow: PythonCall precompiles, and the first `pyimport` loads the interpreter into the Julia process.

Python to Julia (juliacall + juliapkg)

Environment discovery: `juliacall` loads libjulia into the Python process, so the Python side is exactly the interpreter you launch — run everything under `uv run` and dvepy/numpy/pandas are importable with no second installation. The Julia side is located by `juliapkg` in order: `PYTHON_JULIAPKG_EXE` → a compatible `julia` on `PATH` → `juliaup` installs a compatible version → direct download[57]. With juliaup already managing 1.13.1 on `PATH`, the existing installation is reused. Julia packages are installed by `juliapkg` into `<venv>/julia_env` — a thin project that reuses the same depot package

[56] PythonCall compatibility tools (`PyTable`/`PyPandasDataFrame`, `pytable(src; format=:pandas)`) — https://juliapy.github.io/PythonCall.jl/stable/compat/

[57] pyjuliapkg (functional API, `juliapkg.json`, Julia discovery order, `PYTHON_JULIAPKG_EXE`/`PYTHON_JULIAPKG_PROJECT`) — https://github.com/JuliaPy/pyjuliapkg

store, so nothing is downloaded twice; `PYTHON_JULIAPKG_PROJECT` can point it at a shared project instead.

Declare Julia dependencies next to the code that uses them, so `juliapkg` finds them wherever the package is installed[58]:

```
# inst/python/dve/_julia.py — bootstrap called by dvepy.dve before touching Julia
from pathlib import Path
import juliapkg

# repo root = <root>/inst/python/dve/_julia.py
REPO = Path(__file__).resolve().parents[3]

juliapkg.require_julia("1.13")
juliapkg.add("Dve",
             uuid="4f81d0a5-2f9d-487d-8fe0-4f9f77b1ac2b",
             dev=True, path=str(REPO))   # dev-mode: edits to src/Dve.jl are picked up
juliapkg.add("Statistics", uuid="10745b16-79ce-11e8-11f9-7d13ad32a3b2")
juliapkg.add("DataFrames", uuid="a93c6f00-e57d-5684-b7b6-d8193f3e46c0", version="1")
juliapkg.add("PythonCall", uuid="6099a3de-0909-5bc5-a295-3f136d948b72", version="0.9")
juliapkg.resolve()   # first call installs; later calls are a no-op
```

```
# tests/pytest/test_julia_integration.py
import pandas as pd
import pytest

pytest.importorskip("juliacall")
from dve import _julia            # runs the juliapkg bootstrap
_julia()                          # resolve before importing juliacall

from juliacall import Main as jl

jl.seval("using Statistics, DataFrames, Dve")


def test_primitives():
    # Strings: Julia → Python is a plain str
    assert jl.Dve.hello_world("Hello", "Julia") == "Hello, Julia!"
    # juliacall converts Python arguments to Julia where a conversion exists,
    # and Julia scalars come back as Python scalars (docs: juliacall reference)
    assert float(jl.Statistics.mean([1.0, 2.0, 3.0])) == 2.0


def test_dataframes_roundtrip():
```

[58] pyjuliapkg (functional API, `juliapkg.json`, Julia discovery order, `PYTHON_JULIAPKG_EXE`/`PYTHON_JULIAPKG_PROJECT`) — https://github.com/JuliaPy/pyjuliapkg

```
    pdf = pd.DataFrame({"name": ["Earth", "Mars"],
                        "gravity": [9.807, 3.721]})
    # pandas → Julia DataFrame: PyTable wraps the pandas object lazily,
    # DataFrame(...) materializes Julia columns
    df = jl.DataFrame(jl.PythonCall.PyTable(pdf))
    assert float(jl.Statistics.mean(df.gravity)) == pytest.approx(6.764)
    # Julia DataFrame → pandas: pytable (format=:pandas by default) is a copy
    pdf2 = jl.PythonCall.pytable(df)
    pd.testing.assert_frame_equal(pdf2.reset_index(drop=True), pdf)
```

```
$ uv add juliacall juliapkg      # if not yet declared
$ uv run pytest tests/pytest/test_julia_integration.py
..                                                                       [100%]
2 passed
```

Copy semantics summary for this bridge: numeric arrays convert fast and non-copying in both directions[59]; mutable objects that have no Julia equivalent stay wrapped (`Py`-style) rather than copied; scalars and strings are small values materialized on call. Argument conversion is juliacall's convenience layer over PythonCall's explicit `pyconvert` — for anything performance-critical, do the conversion explicitly on the Julia side.

Acceptance tests

- ☐ `julia --version` prints `julia version 1.13.1` (via `juliaup default 1.13.1`)
- ☐ `julia --project=. -e 'using Pkg; Pkg.instantiate(); using Dve; println(Dve.hello_world("Hel "Julia"))'` prints `Hello, Julia!`
- ☐ `julia --project=. -e 'using Pkg; Pkg.test()'` ends with `Testing Dve tests passed`
- ☐ `./scripts/hello.jl --help` prints usage; `./scripts/hello.jl -s Ciao -w Giovanni` prints `Ciao, Giovanni!` from a different CWD (e.g. `cd /tmp && <repo>/scripts/hello.jl ...`)
- ☐ Jupyter lists a `Julia-dve` kernel; a notebook using it can run `using Dve` successfully
- ☐ VS Code / Emacs: opening `src/Dve.jl` yields completions and go-to-definition for `DataFrames` symbols (server found the workspace `Project.toml`)
- ☐ `Rscript -e 'renv::restore()'` then `Rscript scripts/r2julia-demo.R` prints `1.414214`, `Hello, Julia!` and the planets frame
- ☐ `uv sync && uv run pytest tests/pytest/test_julia_integration.py` prints `2 passed`
- ☐ `R CMD build .` produces a tarball without `Project.toml`, `Manifest.toml`, `test/`, `scripts/`, or `.jl` files (check with `R CMD build . && tar tzf dvesimpler_*.tar.gz`)

Common pitfalls

- Running `julia script.jl` without `--project` (or without in-script `Pkg.activate`): Julia falls back to the global environment, `using Dve` fails with `ArgumentError: Package Dve not found`.

[59] juliacall on PyPI (non-copying numeric-array conversion both directions; version/requirements) — https://pypi.org/project/juliacall/ ; juliacall reference — https://juliapy.github.io/PythonCall.jl/stable/juliacall/

- Editing `Manifest.toml` by hand — it is regenerated wholesale by Pkg and your edits will be lost.
- Forgetting `[compat]`: resolution floats to latest compatible versions only if bounds exist; CI reproducibility comes from the committed manifest plus `Pkg.instantiate()` (the analogue of `uv sync --frozen`).
- Moving the repo after `IJulia.installkernel`: the kernelspec hard-codes the project path; re-install the kernel.
- Expecting `Pkg.test()` to find tests in `tests/`: only `test/runtests.jl` runs — keep the shim.
- Expecting JuliaConnectoR results to alias Julia objects: everything is serialized (copies); proxies are the only by-reference channel.
- Letting PythonCall/CondaPkg silently install a second Python interpreter: set `JULIA_CONDAPKG_BACKEND=Null` and `JULIA_PYTHONCALL_EXE`, or you end up with two pandas installs and broken `import dvepy`.
- First-run lateness mistaken for failure: LanguageServer download, PythonCall precompile, and juliapkg resolve are all slow once, then cached.
- CRAN submission with Julia calls active: guard every R test/example touching Julia with `skip_on_cran()` / `if (Sys.getenv("NOT_CRAN") == "true")`.
- Mixing `juliaup` channels: pin `juliup default 1.13.1` so editors, kernels and bridges all embed the same `libjulia`.

Unverified items

- [unverified] `lsp-julia` (Emacs) maintenance cadence — check the repository history at `github.com/gdkrmr/lsp-julia` before standardizing on it.
- [unverified] `pyjulia` dormancy — check `github.com/JuliaPy/pyjulia` releases before considering it.
- [unverified] `JuliaCall` (R, CRAN) current release status — check `cran.r-project.org/package=JuliaCall` if you evaluate it as an alternative.
- [unverified] juliacall's exact automatic argument-conversion rules (which Python types become which Julia types when calling `jl.f(...)` from Python) — check the juliacall reference in the PythonCall documentation before relying on it in production code.
- [unverified] Whether `juliapkg` `path` entries in a `juliapkg.json` file resolve relative to the JSON file's location — check the pyjuliapkg documentation; the functional API with an absolute `path` used above avoids the question.
- [unverified] The exact depot location the VS Code extension uses for its LanguageServer installation, and the current name of its environment-path setting (`julia.environmentPath`) — check the julia-vscode.org extension documentation.

Additional notes

- Julia 1.13.1 is confirmed current; Julia 1.10 remains the LTS line, so CI may reasonably test 1.10 + 1.13 if the package avoids 1.13-only features[60].

[60] Julia 1.13 release notes and highlights — https://docs.julialang.org/en/v1/NEWS/ and https://julialang.org/blog/2026/09/julia-1.13-highlights/ ; 1.13.1 patch release announcement — https://discourse.julialang.org/t/julia-1-13-1-has-been-released/139692

- Time-to-first-plot: first `using` of a package triggers compilation; `Pkg.precompile()` in CI images and juliacall's cached resolution mitigate it. Do not judge Julia speed from the first call.
- `Revise.jl` (in the default environment) redefines changed functions in a live session — the analogue of Python's autoreload, and the standard way to co-develop `src/Dve.jl` while the R/Python bridges keep long-lived processes open.
- The three lock files (`uv.lock`, `renv.lock`, `Manifest.toml`) describe three disjoint toolchains; the only shared state is the machine-level runtimes (Python interpreter, R, Julia) plus `libjulia` embedding. Pin the runtime versions in CI (`uv python`, `rig`/renv for R, `juliaup`) so all three bridges resolve identically.
- When a paper artifact cites the stack, Julia packages carry DOIs via the General registry/Zenodo; `Pkg.status()` prints a citation-ready manifest summary.

Q:2

Q:2 - TODO:(q2-title)

^

Question Prompt 2

TODO:(q1-prompt) ...

A:2 (Claude)

^ ***TODO:(a1-ref-claude)***

TODO:(a2-claude) ...

A:2 (Gemini)

^ ***TODO:(a2-ref-gemini)***

TODO:(a2-gemini) ...

A:2 (ChatGPT)

^ ***TODO:(a2-ref-chatgpt)***

TODO:(a2-chatgpt) ...

A:2 (DeepSeek)

^ ***TODO:(a2-ref-deepseek)***

TODO:(a2-deepseek) ...

A:2 (Perplexity)

^ ***TODO:(a2-ref-perplexity)***

TODO:(a2-perplexity) ...

A:2 (Vibe)

^ ***TODO:(a2-ref-vibe)***

TODO:(a2-vibe) ...

A:a

A:a - TODO:(aa-title)

^

Appendix a

TODO:(aa-text) ...